

Nexis: A Multi-Agent Autonomous Engineering Platform with Closed-Loop Fault Recovery

by

Ratul Sikder

Submitted to the Institute of Information Technology (IIT)

in partial fulfillment of the degree requirements

at the

UNIVERSITY OF DHAKA

August 2026

© University of Dhaka 2026. All rights reserved.

The author hereby grants the Institute of Information Technology (IIT), University of Dhaka, permission to reproduce and distribute copies of this project report, in whole or in part, for academic purposes.

Nexis: A Multi-Agent Autonomous Engineering Platform with Closed-Loop Fault Recovery

by

Ratul Sikder

Submitted to the Institute of Information Technology (IIT) on 11 August 2026,
in partial fulfillment of the degree requirements

This is to declare that this project is the author's original work. No part of it has been submitted elsewhere, in whole or in part, for the award of any other degree or diploma, and the plagiarism policy stated by the supervisor has been maintained.

Author

Institute of Information Technology (IIT), University of Dhaka

11 August 2026

Certified by

Project Supervisor

Accepted by

Director, Institute of Information Technology (IIT)

Acknowledgement

I would like to express my sincere gratitude to the faculty of the Institute of Information Technology (IIT), University of Dhaka, for the academic environment and the guidance that made this project possible. The idea behind Nexis grew out of a frustration that I believe most engineering teams face at some point: an AI tool may write a function well enough, but it rarely closes the loop all the way to a production failure being fixed and verified. I am grateful that I was given the freedom to pursue this problem in real depth, building nine cooperating agents, a genuine multi-tenant control plane, and a working closed-loop deployment engine, instead of settling for a smaller proof of concept. I would also like to thank my family for their patience through the long build-and-verify cycles that this project demanded.

Abstract

Modern software engineering teams spend an estimated 40 to 60 percent of their time on maintenance, debugging, and manual incident response, rather than on building new capability. A number of AI-assisted developer tools already exist in the market today, such as code completion assistants, automated test generators, and deployment dashboards, but each of these addresses only an individual task in isolation. None of them model the collaborative structure of a real engineering team, and none of them close the loop between detecting a production failure and actually repairing it. **Nexis** has been designed to address both of these gaps together. The system models a software engineering organisation as nine role-specialised autonomous agents, arranged across two cooperating layers: a five-agent Execution Team (Architect, Backend, Quality Assurance (QA), DevOps, and Data Engineer) and a four-agent Self-Healing Loop (Sentinel, Pathfinder, Synthesiser, and Validator). These agents are durably orchestrated using Temporal, and every automated repair is gated by a severity-routed, human-in-the-loop Approval Gate that supports Approve, Reject, and Modify (Reinforcement Learning from Human Feedback (RLHF)) decisions. This report documents the complete, working system, which includes a multi-tenant control plane built with Go (Golang), PostgreSQL with Row-Level Security (RLS), Neo4j, and Redis; a Docker shadow-validation sandbox; a real GitHub-App-mediated GitOps deployment path; and a from-scratch Docker preview-deploy engine that clones, builds, and serves an arbitrary connected repository, feeding its own build and runtime failures back into the same self-healing pipeline. It also includes a Next.js console that spans authentication, Role-Based Access Control (RBAC), intelligent role recommendation, incident observability, billing, and multi-provider integrations. Two live end-to-end recovery runs against a local Large Language Model (LLM) have been reported verbatim in this document: one correctly timed out at the human approval gate, while the other succeeded fully, from fault detection through to an approved and deployed patch, in under two minutes. Alongside these, a live and verified proof of the preview-deploy engine building and serving a real public repository has also been included. This report has been written as a candid engineering account. What works has been demonstrated with reproducible evidence, and what has been deliberately left out of scope, namely a fully credentialed production deployment to the registered `nexis.sh` domain

and the human-subject NASA Task Load Index (NASA-TLX) study, has been stated plainly rather than left implied.

Keywords: Multi-agent systems, autonomous software engineering, self-healing systems, closed-loop fault recovery, root-cause analysis, LLM-guided program repair, DevOps automation, Temporal workflow orchestration, human-in-the-loop AI, retrieval-augmented generation.

Table of Contents

Chapter 1: Introduction.....	8
1.1 Background.....	8
1.2 Motivation.....	9
1.3 Problem Statement.....	9
1.4 Project Objectives.....	10
1.5 Scope of the Project.....	12
1.6 Significance of the Project.....	13
Chapter 2: Literature Review.....	14
2.1 Overview.....	14
2.2 Research Gap.....	17
2.3 Conclusion.....	18
Chapter 3: System Analysis and Requirements.....	20
3.1 Requirement Analysis.....	20
3.2 Functional Requirements.....	21
3.3 Non-Functional Requirements.....	26
3.4 System Feasibility.....	28
Chapter 4: System Design and Architecture.....	30
4.1 System Context.....	30
4.2 Component Architecture.....	32
4.3 Data and Storage Architecture.....	34
4.4 Self-Healing Loop Pipeline.....	35
4.5 Deployment and Infrastructure.....	37
4.6 Use Case Diagrams.....	38
4.7 Data Flow Diagrams.....	42
4.8 Self-Healing Loop Sequence Diagram.....	49
Chapter 5: Implementation.....	53
5.1 Tools and Technology.....	53
5.2 Project View.....	56
5.3 Database Schema.....	90
Chapter 6: Testing and Results.....	93
6.1 Static Verification.....	93

6.2 Live End-to-End Recovery Runs.....	94
6.3 Live Deploy-Engine Verification.....	96
6.4 Defects Found and Fixed During This Engineering Cycle.....	97
6.5 Testing Summary.....	99
Chapter 7: Conclusion and Future Work.....	100
7.1 Overview.....	100
7.2 Limitations of the Project.....	101
7.3 Future Work.....	102
7.4 Conclusion.....	104
References.....	105

Chapter 1: Introduction

This chapter lays out why **Nexis** was built in the first place: the pain points that drove it, where existing tools fall short, the objectives set for the project, and who, human and non-human alike, it actually serves. It ends by explaining why a genuinely closed loop, running from fault detection through to verified deployment, actually matters.

| 1.1 Background

Software rarely fails at the moment it is written. It fails later, in production, under load, against data and dependencies that its authors never anticipated. It may be noted that this cost is not merely anecdotal: the original proposal for this project draws on industry experience placing it at 40–60% of an engineering team’s time, spent on maintenance, debugging, and incident response rather than on building anything new. Consequently, for a small team this means that for every engineer creating value, there is roughly one more engineer employed full-time just to keep it working.

Artificial Intelligence (AI)-assisted software engineering has grown rapidly over the last few years. Code-completion assistants such as GitHub Copilot [1] now write a meaningful share of new code, and the tools built around them generate tests, review pull requests, and summarise logs. Yet each of these remains a point solution: it accelerates one task performed by one human, and nothing more. The production incident lifecycle makes the point clearly. It runs through seven stages: detect a fault, diagnose its root cause, write a fix, validate it, obtain approval, deploy it, and verify the deployment. At every hand-off between these stages, a human engineer remains the transport mechanism; thus, the chain stays manual from end to end.

Nexis exists to close that chain. Rather than assisting one engineer at one task, it models an entire software engineering organisation as nine role-specialised autonomous agents, arranged in two cooperating layers: a five-agent Execution Team and a four-agent Self-Healing Loop, both durably orchestrated by the Temporal workflow engine [2]. The human approval decision, in turn, is reduced to a single severity-routed click rather than an open-ended manual process.

| 1.2 Motivation

The pain points behind this project are not abstract; they show up in some form in almost any engineering organisation:

- **No tool closes the loop.** Monitoring products such as Sentry [14], Datadog [16], and PagerDuty [15] can detect a failure and page a human about it; code assistants such as GitHub Copilot [1] write code, but only once a human has asked for it. Nothing on the market currently connects the two, so that a detected failure leads on its own to an autonomous repair.
- **The engineer is the bottleneck for every incident.** Even a routine alert requires a human to read the logs, localise the fault, write a patch, run the tests, request review, deploy, and then watch the rollout. That human sits on the critical path for all seven stages, even for fault classes that recur in an almost identical form each time.
- **Root-cause analysis is unaided.** Localising a fault, in practice, means tracing call chains and service dependencies by hand and from memory, since no mainstream incident tool walks the code dependency graph or ranks candidate root causes against any stated evidence.
- **Repair without validation is dangerous, so it is not attempted.** Because there is no safe, sandboxed path for a machine-generated patch to prove itself before a human ever sees it, organisations reasonably refuse to let machines patch anything at all; as such, the loop stays open.
- **Approval is all-or-nothing.** Deployment gates are typically either absent (which is risky automation) or heavyweight (a full review cycle for what may be a one-line fix), and no existing tool scales the depth of human involvement to the measured severity of the change.

| 1.3 Problem Statement

Existing approaches to Artificial Intelligence (AI)-assisted engineering and incident response, taken together, share a set of structural limitations:

- They are **isolated tools**: detection, diagnosis, code generation, testing, and deployment each live in separate products, with no shared state or protocol connecting them.
- They provide **no closed loop**: a detected fault never automatically becomes a validated, deployable repair.
- They keep a **human as the bottleneck for every decision**, regardless of how small, repetitive, or low-risk the change is.
- They offer **no causal root-cause tooling**. Nothing traverses the actual dependency structure of the code the way a graph database such as Neo4j [3] makes possible, and nothing applies causal reasoning of the kind formalised by libraries such as DoWhy [4] to explain *why* a fault occurred in the first place.
- They provide **no safe autonomous deploy path**. Even in cases where a machine can propose a patch, there is no sandboxed stage to validate it, no severity-aware approval gate, and nothing to verify or roll back the deployment automatically, so shipping that patch can never really be defended.

In this regard, the problem this project addresses can be stated as follows: *how can the full production-fault lifecycle (detect, diagnose, repair, validate, approve, deploy, and verify) be closed autonomously by a team of cooperating role-specialised agents, while still keeping a human decision in the loop exactly where, and only where, the severity of the change warrants it?*

| 1.4 Project Objectives

What Nexis set out to build, and what this report documents as actually built, is the following:

1. **A nine-agent, two-layer architecture.** The first layer is a five-agent Execution Team, made up of the Architect, Backend, Quality Assurance (QA), DevOps, and Data Engineer agents, which plans, generates minimal-diff patches, writes property-based tests, produces deployment manifests, and proposes database migrations. The second is a four-agent Self-Healing Loop, made up of Sentinel, Pathfinder, Synthesiser, and Validator, which detects faults statistically, walks the Neo4j [3] code dependency graph toward candidate root causes, routes the incident only to the agents it actually needs, and

shadow-executes every candidate patch inside a locked-down Docker [10] sandbox before any human ever sees it. The entire pipeline runs as a durable Temporal workflow [2], so that every step survives process restarts and can be replayed.

2. **Severity-routed human approval.** An Approval Gate classifies each validated patch as HIGH (mandatory human review, for example schema changes, security-sensitive paths, or a contract violation by an agent), MEDIUM (a 120-second window in which a human may intervene before auto-approval), or LOW (immediate auto-approval for trivial User Interface (UI)-only diffs). The human is then left with three decision options: Approve, Reject, or Modify, the last of which lets the engineer edit the diff before it ships. Every terminal decision is persisted afterward as feedback data, meant to support future model improvement.
3. **Real GitOps closure.** Once approved, the platform opens a genuine GitHub pull request through a GitHub App installation. A post-deploy rollback probe then watches for fresh fatal errors and, when the recovery policy calls for it, invokes Argo CD's rollback client [6] to fall back to the prior known-good revision.
4. **A Docker preview-deploy engine.** A dedicated service clones any connected repository, detects its technology stack, generates a working Dockerfile when none already exists, builds and runs the container under strict resource limits, health-checks it over Hypertext Transfer Protocol (HTTP), and returns a live preview Uniform Resource Locator (URL). Crucially, it also feeds its own build or health-check failures back into the same self-healing pipeline, treating them as first-class incidents in their own right.
5. **Intelligent role recommendation.** A heuristic engine, working over real audit-log and login signal, recommends privilege escalations and downgrades for organisation members, each accompanied by a plain-language, evidence-backed rationale that an administrator is free to accept or dismiss.
6. **A complete multi-tenant platform around the loop.** This includes a control plane built on PostgreSQL Row-Level Security (RLS) [7] for tenant isolation, Role-Based Access Control (RBAC), full authentication (including Multi-Factor Authentication (MFA) and Single Sign-On (SSO) via WorkOS [12]), billing through Stripe [13], integrations with GitHub, Slack [17], Sentry [14], Datadog [16], and PagerDuty [15], a tamper-evident

audit log, and an administrative console that covers incidents, approvals, agents, cost, and system health.

| 1.5 Scope of the Project

Nexis serves three categories of human user, each distinguished by Role-Based Access Control (RBAC):

- **Owner / Administrator:** provisions the organisation, connects integrations, sets recovery policy, decides on HIGH-severity approvals, manages members, billing, and Application Programming Interface (API) keys, and acts on role recommendations.
- **Engineer / Member:** works with projects and incidents on a day-to-day basis, observes live pipeline runs, and exercises the Approve / Reject / Modify decision on patches within their remit.
- **Viewer:** a read-only console experience, enforced by the same Role-Based Access Control gating, for stakeholders who need visibility into incidents, recovery outcomes, and cost without the ability to change anything.

Beyond these three roles, Nexis makes a deliberate conceptual departure from conventional systems analysis, in that the **nine autonomous agents are modelled as a distinct actor category in their own right**, and not merely as internal implementation detail. In a typical information system, every actor in the use-case model is human. In Nexis, however, the Sentinel raises incidents, the Architect plans, the Backend agent writes code, and the Validator renders verdicts. Each of these agents initiates and completes use cases exactly as a human actor would, subject to the same audit logging and the same policy constraints. Moreover, treating agents as first-class actors is precisely what allows the human roles above to shrink down to supervisory decisions, and this framing carries through consistently across the requirement analysis and design chapters of this report.

The scope of the delivered system covers the full platform described under the objectives above, evaluated end-to-end in a local environment against a locally hosted Large Language Model (LLM) via Ollama [20]. The `nexis.sh` domain has been registered for the platform, though a

credentialed public production deployment falls outside the scope of this report, a point discussed honestly in the limitations.

| 1.6 Significance of the Project

The significance of Nexis lies chiefly in closing a loop that existing tools leave open. Detection without repair produces pager fatigue; code generation without validation produces risk; and automation without a human gate produces distrust that is entirely justified. Every individual stage of the lifecycle has been automated somewhere before. What has been missing is a system in which detect → diagnose → repair → validate → approve → deploy → verify runs as one continuous, durable, and auditable workflow, with the human reduced from being the transport mechanism between stages to making a single, well-placed decision.

This report does not argue that claim in the abstract, but presents live evidence for it instead. Chapter 6 reports two genuine end-to-end recovery runs, both executed against a real, locally hosted language model [20]. In one, the approval window elapsed and the pipeline correctly rejected the patch on timeout (the safety mechanism working exactly as designed); in the other, a fault travelled the entire loop, from detection to an approved and deployed patch, in under two minutes, with per-agent token and cost figures recorded at every step. The same chapter also presents a live, HTTP-verified run of the preview-deploy engine, building and serving a real public repository that contained no Dockerfile to begin with. A platform that can demonstrably turn a production fault into a validated, human-approved repair in roughly the time it takes an on-call engineer to open their laptop marks a meaningful step toward engineering teams spending their time building software, rather than forever nursing it.

Chapter 2: Literature Review

| 2.1 Overview

Nexis is, at its core, a systems-integration project: its contribution comes not from a single new algorithm but from composing several established, well-engineered tools into a closed loop that none of them forms alone. Since intellectual honesty demands a clear account of what each tool already does, the discussion below goes tool by tool, distinguishing plain dependency from genuine extension.

Large Language Model (LLM) coding assistants. GitHub Copilot [1] is the most widely deployed Artificial Intelligence (AI) pair-programming tool: it completes code and, in its agent modes, drafts multi-file changes inside an editor session. Yet all of this happens inside the developer’s authoring context. Copilot never observes a production incident, never checks its own output against a failing system, and never deploys anything. Nexis’s Backend agent occupies that same “generate a minimal code change” niche, though what invokes it is a fault-detection pipeline rather than a keystroke, and its output is machine-validated before any human sees it.

Durable workflow orchestration. Temporal [2] provides durable, replayable workflow execution with automatic retry and state persistence. Nexis takes Temporal as-is, as a dependency: the entire nine-agent RecoveryPipeline runs as a single Temporal workflow, which is why every agent step survives a process restart and is recorded as a durable ActivityEvent. What Temporal does not provide is the engineering semantics layered on top: role routing, contract-violation detection, and severity classification are all Nexis’s own logic.

Graph databases for dependency analysis. Neo4j [3] is a native graph database, and Nexis stores its code-dependency graph inside it. The Pathfinder agent walks that graph from a fault symptom to candidate root-cause nodes, forwarding hop distance and node degree as evidence. Neo4j supplies storage and traversal, nothing more; the evidence-ranking applied to the traversal results is Nexis’s own code.

Causal inference tooling. DoWhy [4], Microsoft Research’s library for explicit causal modelling with refutation tests, is the methodological reference point behind Nexis’s causal-inference sidecar. It must be stated plainly, however, that Nexis does *not* fit a DoWhy structural causal model, since no interventional production data exists to fit one against. As such, the sidecar implements a documented proxy instead, ranking candidates by evidence-chain specificity, graph-structural proximity, textual overlap, and a scenario prior, with a transparent per-component confidence breakdown.

Observability standards. OpenTelemetry [5] standardises traces, metrics, and logs; Nexis adopts it, unmodified, as its instrumentation layer, exporting to Prometheus, Loki, and Tempo behind Grafana. It is a straightforward dependency: Nexis adds nothing to the standard itself.

GitOps deployment. Argo CD [6] reconciles a cluster against a Git repository and exposes Sync and Rollback operations. Nexis’s DevOps agent emits Argo CD manifests, and whenever a policy-gated Service Level Objective (SLO) breach is detected, the post-deploy rollback probe calls Argo CD’s real Rollback client. That is integration on Nexis’s part, not an extension of Argo CD itself.

Container runtimes. Docker [10] is the target Application Programming Interface (API) for two components inside Nexis: the Validator’s sandbox, a network-isolated, read-only, capability-dropped shadow execution of every candidate patch, and the preview-deploy engine. Podman [11] stood in for Docker in the evaluation environment, as a Docker-API-compatible runtime, and behaved identically; the substitution is reported honestly here, not hidden.

Property-based testing. Hypothesis [9] generates test inputs from declared properties. Nexis’s Quality Assurance (QA) agent puts it to a genuinely new use, auto-generating pytest/Hypothesis suites from the incident and the Backend diff while Hypothesis itself stays unmodified. A background QA loop then replays the stored suites and raises a fresh incident on any pass-to-fail regression.

Data lineage. OpenLineage [8] defines an open specification for run-level lineage events. Nexis’s Data Engineer agent emits OpenLineage-compatible RunEvents for every migration it proposes or applies; here too, Nexis conforms to the specification rather than extending it.

The incident-response ecosystem. Sentry [14], Datadog [16], and PagerDuty [15] respectively capture application errors, aggregate observability data, and route alerts to an on-call human. Nexis integrates all three as webhook-driven incident *sources* feeding the same Sentinel pipeline. The Slack API [17] is used in the opposite direction: the Approval Gate pushes a decision into Slack, where an engineer can approve or reject a validated patch with one click. A further set of platform dependencies is used exactly as documented, without modification: WorkOS for Single Sign-On (SSO) [12], Stripe for billing [13], PostgreSQL Row-Level Security (RLS) for tenant isolation [7], pgvector for knowledge-base retrieval [19], and Ollama [20], the local Large Language Model provider behind this report’s live evidence. Table 2.1 sets out this dependency-versus-contribution boundary.

Table 2.1: Position of surveyed tools relative to Nexis.

Tool	What it provides	Relation to Nexis
GitHub Copilot [1]	Editor-time code generation	Comparable to one agent (Backend) only
Temporal [2]	Durable workflow execution	Dependency; hosts the RecoveryPipeline
Neo4j [3]	Graph storage and traversal	Dependency; Pathfinder walks it
DoWhy [4]	Structural causal models	Reference point; proxy method used instead
OpenTelemetry [5]	Telemetry standard	Dependency; instrumentation layer
Argo CD [6]	GitOps sync/rollback	Integrated by DevOps agent and rollback probe
Docker [10] / Podman [11]	Container runtime	Dependency; sandbox and deploy engine
Hypothesis [9]	Property-based testing	Novel use: Quality Assurance agent generates suites
OpenLineage [8]	Lineage specification	Conformance by Data

Tool	What it provides	Relation to Nexis
		Engineer agent
Sentry / Datadog / PagerDuty [14, 16, 15]	Error/alert capture	Webhook incident sources
Slack API [17]	Team messaging	One-click approval channel

| 2.2 Research Gap

Each tool surveyed above is strong within its own boundary, and several of them are routinely chained together in industry: Sentry alerting into PagerDuty, an on-call engineer opening an editor with GitHub Copilot, Argo CD eventually deploying the merge that follows. Moreover, what no single tool, nor any combination in common use, provides is a system that (a) models a software engineering *team* with explicit role specialisation, and (b) closes the loop from automatic fault detection to a deployed, verified fix, with human judgement concentrated at one severity-routed decision point.

This gap shows up at every seam of the conventional tool chain. GitHub Copilot generates code but never sees a production incident; it has no notion of a fault, a blast radius, or a deployment [1]. Observability platforms such as Datadog and Sentry detect and display faults with great fidelity, yet they never generate a candidate fix, let alone validate one [16, 14]. PagerDuty routes a human to the problem, but the repair itself stays entirely manual [15]. Argo CD deploys whatever a repository contains, though it authors nothing [6]. Temporal can durably orchestrate any of these steps; still, it carries no engineering semantics of its own [2]. In every case the loop breaks at the same place: between “a fault is known” and “a validated fix exists,” a human performs the unaided diagnosis, authoring, testing, review, and release.

Nexis closes precisely that span, end to end. Sentinel detects a fault using a statistical-process-control rule, an Exponentially Weighted Moving Average (EWMA) baseline with a mean-plus-three-sigma breach condition, rather than a fixed threshold. Pathfinder then walks the real Neo4j dependency graph to candidate root causes, which the causal-inference sidecar ranks with a transparent confidence breakdown. The Synthesiser classifies the scenario and selects which Execution Team agents are actually needed, and the workflow enforces that routing by visibly

skipping the agents not selected. The Architect, for its part, produces a structured plan whose `affected_files` contract is mechanically checked against the Backend agent’s actual diff; a violation forces HIGH severity so the change can never auto-deploy unreviewed. Quality Assurance generates property-based tests from the incident and the diff [9], and the Validator then executes the patch and those tests inside a locked-down container sandbox, so no unvalidated patch ever reaches a human. Only at this final stage does the single human decision point engage, and only when severity warrants it: HIGH-severity changes require explicit review, MEDIUM-severity changes auto-approve after a 120-second window unless overridden, and trivial LOW-severity changes pass immediately. The decision itself is richer than a binary gate, since the engineer may Approve, Reject, or Modify the diff before it ships, with every terminal decision persisted for future preference learning. On approval, a real GitHub pull request is opened, and a post-deploy rollback probe watches for the regression that follows [6].

Two further properties set this apart from a merely scripted pipeline of existing tools. It may be noted, first, that role specialisation here is structural, not cosmetic: the five Execution Team agents (Architect, Backend, Quality Assurance, DevOps, Data Engineer) and the four Self-Healing Loop agents (Sentinel, Pathfinder, Synthesiser, Validator) have distinct inputs, outputs, and contracts, and the Architect’s plan constrains the others. Second, the loop closes over the platform’s own infrastructure too: when Nexis’s preview-deploy engine fails to build or health-check a repository, that failure is inserted as a first-class incident into the very same pipeline, and “write a working Dockerfile” becomes an ordinary repair task. To the author’s knowledge, no tool in common use today combines these properties in a single system.

| 2.3 Conclusion

The landscape surveyed in this chapter supplies every ingredient Nexis needs: durable orchestration [2], graph traversal [3], sandboxed execution [10, 11], property-based testing [9], GitOps delivery [6], and a mature incident-capture ecosystem [14, 16, 15]. What it does not supply is the loop between detection and repair; that is left open, for a human to close. Hence, Nexis’s contribution lies not in any single new algorithm but in the demonstrated composition of these tools into a role-specialised multi-agent team that detects, diagnoses, repairs, validates, and deploys autonomously, engaging human judgement once, at a severity-routed gate. How that

composition was specified, designed, implemented, and tested is the subject of the chapters that follow.

Chapter 3: System Analysis and Requirements

| 3.1 Requirement Analysis

Nexis is a role-based, multi-tenant platform, and this single fact shapes almost every requirement that follows. Every piece of data in the system belongs to exactly one organisation (tenant), and every request is evaluated against both the caller's organisation and the caller's role within it. The requirement analysis accordingly begins by identifying the actors who interact with the system boundary.

Three *human* actor types were identified in the course of this analysis:

- **Owner/Administrator** — the organisation's administrator, responsible for membership, roles, billing, policy, integrations, and the final human decision on high-stakes automated repairs.
- **Engineer/Member** — the working engineer who investigates incidents, exercises the evaluation tooling, and interacts with the recovery pipeline day to day.
- **Viewer** — a read-only observer (for example a stakeholder or an auditor) who may inspect the system's state but may not change it.

The role model is *additive*: an Administrator holds every permission an Engineer holds, and an Engineer in turn holds every permission a Viewer holds. As such, the functional requirements in Section 3.2 list only the permissions each role *adds* over the tier below it, so that nothing is repeated across the three lists. (Internally the database role enumeration is `owner/admin/member`; the console additionally presents the read-only Viewer experience through Role-Based Access Control (RBAC) gating on the frontend and Application Programming Interface (API).)

Unusually for a requirement analysis, a fourth actor category has to be modelled here that is not human at all: the **nine-agent self-healing fleet**. Most requirement-analysis chapters enumerate only human actors, since in a conventional system every state change can ultimately be traced back to a person clicking something; that assumption simply does not hold for Nexis. The agent fleet acts autonomously inside the system boundary. It detects faults without being asked, reads

and writes the same PostgreSQL, Neo4j, and object-storage data stores that the human-facing console uses, and triggers real external side effects: opening GitHub pull requests, applying rollbacks through Argo CD [6], and redeploying preview containers. An actor model that left the fleet out would misrepresent the system, given that roughly half of all writes to the incident, patch, and audit tables originate from agents rather than from people. Treating the fleet as a first-class actor also makes the safety analysis explicit, since every autonomous capability listed in Section 3.2.4 is bounded by a corresponding control (sandboxing, severity classification, or the human approval gate) listed in Section 3.3.

The Level 0 and Level 1 use case diagrams for these four actors appear together with the rest of the system’s Unified Modeling Language (UML) design in Section 4.6 (Figures 4.6 and 4.7), alongside the data flow diagrams in Section 4.7, which show how the same actors move data through the platform.

| 3.2 Functional Requirements

The functional requirements that follow are grouped by actor. Since the role model is additive, each human tier is listed only as a delta over the tier beneath it.

3.2.1 Owner/Administrator can:

- Do everything an Engineer/Member and a Viewer can do (role additivity).
- Manage organisation membership: generate invite codes, send email invites, and remove members.
- Assign and change member roles, and act on the intelligent role-recommendation banner on the Members settings page: accept a recommendation (which applies the role change under an optimistic-concurrency guard) or dismiss it (which starts a 30-day per-rule cool-down).
- Edit organisation settings (name, profile, and workspace-level configuration).
- Manage billing through Stripe [13]: view usage, manage payment methods, and review invoices produced by the usage-ticker and invoice-roller cron jobs; set the organisation’s daily token budget on the cost tracker.

- View and export the append-only, hash-chained audit log (Comma-Separated Values (CSV) export) and invoke its cryptographic verification endpoint to prove the chain has not been tampered with.
- Decide pending approvals from the recovery pipeline: **Approve** a validated patch, **Reject** it, or **Modify** it by editing the unified diff in place before it ships, with the edited diff persisted as a Reinforcement Learning from Human Feedback (RLHF) feedback example.
- Configure each project's recovery policy, including whether the platform rolls back automatically on a Service-Level Objective (SLO) breach.
- Operate the Docker preview-deploy engine from a project's *Ops* tab: trigger Deploy and Stop, watch live status, open the returned preview Uniform Resource Locator (URL), inspect build and container logs, and review deployment history.
- Install and manage integrations: the GitHub App (installation, repository listing, webhook-driven incident creation on merged pull requests), Sentry [14], Datadog [16], and PagerDuty [15] webhook sources, Slack [17] (OAuth install plus interactive one-click approve/reject directly from a Slack message), and Argo CD [6].
- Export evaluation-harness results as CSV (an administrator-only action).
- Receive the daily administrator digest email summarising the last 24 hours of incidents, repairs, approvals, and Mean Time To Recovery (MTTR) for the organisation.
- Monitor platform operations pages: System Health (parallel dependency probes of PostgreSQL, Redis, Neo4j, MinIO, and Temporal), webhook activity, and integration connection health.

3.2.2 Engineer/Member can:

- Do everything a Viewer can do (role additivity).
- Sign up and sign in with email and password (a JSON Web Token (JWT) session in an HttpOnly cookie), sign in passwordlessly via magic link, or sign in through WorkOS Single Sign-On (SSO) [12]; recover access through the password-reset flow (hashed, time-limited reset tokens).

- Enrol Multi-Factor Authentication (MFA): Time-based One-Time Password (TOTP) with Quick Response (QR) code provisioning.
- Manage their own active sessions: list signed-in devices with Internet Protocol (IP) address, user agent, and last-seen time (the caller’s own session carries a “This device” badge), and revoke any session remotely.
- Create, list, and revoke scoped API keys (`nx_live_...` bearer tokens) for programmatic access.
- Create projects through the new-project wizard and connect them to GitHub repositories.
- Browse the incidents list and open an incident’s detail page, where the nine-agent pipeline timeline streams live over Server-Sent Events (SSE), including per-agent status, token counts, and cost.
- Inspect each agent’s structured payloads (for example the Backend agent’s raw generated diff) from the timeline.
- Trigger fault-injection runs from the Live-Demo console page, which presents 27 curated scenario cards drawn from a broader fixture-scenario taxonomy; only one scenario is fully wired end-to-end at present, and the remainder are labelled “Coming soon”.
- Run the evaluation harness: side-by-side OpenAI-versus-Ollama provider comparisons over fixture incidents, with persisted transcripts and cost figures.
- Query the knowledge base (pgvector-backed retrieval [19]) and view its retrieval status.
- Complete the post-incident NASA Task Load Index (NASA-TLX) workload survey [18], the platform’s instrument for measuring operator cognitive load.
- Use the operational consoles: the agents fleet page and per-agent detail views, the workflows list, the activity stream, the validator sandbox page, and the recovery-pipeline view.

3.2.3 Viewer can:

- Sign in and view the dashboard: Key Performance Indicator (KPI) tiles (MTTR, recovery success rate, open incidents, mean tokens per run), the incidents-over-time chart, the system status panel, the projects health grid, and the recent activity feed.

- Read the incidents list and incident detail timelines, including the live SSE stream, without any decision or mutation controls.
- Read project pages (overview, activity, and status) and the performance page.
- View the agents, workflows, and activity-stream pages in read-only form.
- Take no mutating action of any kind: no approvals, no deployments, no settings changes, no key or session management beyond their own account.

3.2.4 The Nine-Agent Fleet autonomously:

- **Detects** faults (Sentinel): an always-on detector applies two rules, a per-row fatal-level trigger and a statistical-process-control spike rule that maintains a per-organisation Exponentially Weighted Moving Average (EWMA) mean and variance baseline and fires on a breach of $\text{mean} + 3\sigma$, with a Poisson floor for low-count noise and an absolute-floor warm-up fallback.
- **Diagnoses** root causes (Pathfinder): walks the Neo4j code dependency graph [3] outward from the fault symptom and forwards candidate root-cause nodes, annotated with hop distance and degree, to the causal-inference sidecar, which ranks candidates by evidence-chain specificity, structural proximity, textual overlap, and a scenario prior, returning a per-component confidence breakdown [4].
- **Classifies and routes** (Synthesiser): assigns the incident a scenario, using a regular-expression (regex) fast path with a Large Language Model (LLM) fallback, and computes an ordered `selected_agents` list; the workflow enforces this routing, skipping non-selected execution agents with a visible “skipped” timeline state.
- **Plans** the repair (Architect): decomposes the incident into `plan_steps`, `affected_files`, and a `risk_level`, and serves as a contract other agents may not silently violate: if the generated diff touches a file outside `affected_files`, the workflow flags a contract violation and forces HIGH severity, so the patch can never auto-deploy unreviewed.
- **Codes** the fix (Backend): generates an LLM-guided minimal unified diff, the smallest valid change rather than a full-file rewrite.
- **Tests** the fix (QA, the Quality Assurance agent): auto-generates property-based test suites (pytest/Hypothesis [9]) from the incident and the diff; a background continuous-QA loop

replays previously generated suites against the sandbox and raises a fresh incident on any pass-to-fail regression, closing a second, independent loop.

- **Packages** the deployment (DevOps): emits Continuous Integration / Continuous Deployment (CI/CD) and Argo CD manifests [6] and drives post-deployment health monitoring.
- **Handles data changes** (Data Engineer): proposes paired forward and reverse Structured Query Language (SQL) migrations, runs a proactive schema-drift detector that introspects `information_schema` against a captured baseline and raises a real incident on drift, and emits OpenLineage-compatible run events [8] for every migration proposal and application.
- **Validates** every candidate patch (Validator): executes it in a locked-down Docker sandbox [10] (no network access, a read-only filesystem, all capabilities dropped, a Process Identifier (PID) limit, and `no-new-privileges`), runs the QA-generated property tests, and reports pass/fail and coverage; no unvalidated patch ever reaches the approval gate.
- **Classifies severity** at the approval gate: HIGH for schema-drift scenarios, sensitive-path matches (SQL, migrations, authentication, security, and cryptography paths), unknown scenarios, or contract violations; MEDIUM otherwise, with a 120-second auto-approve countdown unless a human intervenes; LOW (tiny User Interface (UI)-only diffs) auto-approves immediately.
- **Deploys** approved patches (GitOps): opens a real GitHub pull request through a GitHub App installation (blob → tree → commit → ref → pull request); when the triggering incident originated from the preview-deploy engine, a successful merge additionally triggers an automatic redeploy.
- **Rolls back** on regression: a post-deploy probe watches for a fresh fatal incident on the same organisation and service within a bounded window and, when the project policy permits, invokes Argo CD's rollback client to restore the prior known-good revision.
- **Records** every step as a Temporal-durable activity event [2] and streams it live to the incident detail page over Server-Sent Events.

- **Feeds the deploy engine's failures back into itself:** when a preview deployment fails to build or pass its health check, the deploy engine files a real incident (`source: "deploy_engine"`) into the same Sentinel-to-pipeline path as every other incident source, making a broken build a self-healing target like any production fault.

| 3.3 Non-Functional Requirements

Table 3.1: Summary of non-functional requirements and their realisation.

Requirement	Realisation in Nexis
Multi-tenancy	PostgreSQL Row-Level Security (RLS) on essentially every table [7]
Durability	Temporal-orchestrated workflows survive crashes and restarts [2]
Security	Sandboxed patch execution, encrypted patch storage, hashed tokens, hash-chained audit log
Scalability	Stateless Go (Golang) services, queue-decoupled workers, parallel dependency probes
Responsiveness	Server-Sent Events stream the live incident timeline
Usability	Role-scoped console, one-click approvals, plain-language rationales

Multi-tenancy and isolation. Tenant isolation cannot be left to application code remembering to add a `WHERE org_id = ...` clause every time; that approach fails the moment a single query is missed. Nexis instead enforces isolation inside the database itself, using PostgreSQL Row-Level Security policies [7]. A request-scoped transaction binds `app.current_org_id` for every request, and the database refuses to return or mutate rows belonging to any other organisation, no matter what the application layer asks for. This applies uniformly to human requests and to agent writes alike.

Durability of long-running work. A recovery run can span many minutes, crosses several services along the way, and often waits on a human decision, so it must survive process crashes and restarts without losing its place. For this reason all pipeline state lives in Temporal [2], a durable workflow engine, where every agent step is recorded as a journaled activity and a resumed worker replays deterministically to exactly where it stopped. An in-memory orchestrator was considered and rejected, since a single crash mid-repair would strand an incident in an unknown state.

Security. Autonomously generated code is treated as untrusted by construction, and nothing written by an LLM is allowed to run on the assumption that it is safe. Candidate patches therefore execute only inside a Docker sandbox [10], with networking disabled, a read-only filesystem, all Linux capabilities dropped, a process-count limit, and `no-new-privileges`. Patch blobs are kept encrypted in object storage, and password-reset and API tokens are stored only as hashes (reset tokens are 32-byte random values hashed with Secure Hash Algorithm 256 (SHA-256), carrying a 30-minute time-to-live (TTL) and a non-leaky 202 response). The audit log itself is append-only and hash-chained, which means any tampering with a past entry breaks the chain and can be detected through the verification endpoint.

Scalability. The control plane, validator, GitOps, and deploy-engine services are independent, stateless Go (Golang) modules, which lets each of them scale horizontally on its own. Temporal decouples workflow load from any single process, Redis handles rate limiting and caching, and the System Health page’s dependency probes fan out in parallel under a bounded timeout, so that a single slow dependency cannot stall the whole page.

Real-time responsiveness. An engineer watching a live repair needs to see agent transitions as they happen, not after a polling delay. The incident detail page therefore subscribes to a Server-Sent Events stream, and every Temporal-recorded activity event reaches the browser within moments of occurring; the live-demo runs described in Chapter 6 were in fact observed this way.

Usability. The console has to make a genuinely complex system operable by an ordinary user. Role-scoped navigation shows each person only what they can act on, approvals are reduced to a one-click decision with the full diff in view, and severity together with countdown state stays

visible at all times; role recommendations, in addition, carry a plain-language rationale that names the concrete evidence behind them.

| 3.4 System Feasibility

3.4.1 Technical Feasibility

Nexis is deliberately polyglot, with each language chosen for the workload it carries rather than for uniformity across the codebase. The four backend services (control-plane, validator, gitops, deploy-engine) are independent Go (Golang) modules, and Go was chosen here because its static binaries, low-overhead concurrency, and first-class Docker API client suit long-lived services that juggle webhooks, container lifecycles, and parallel health probes. The console, by contrast, is Next.js/React with TypeScript, which is the practical choice for a large role-gated UI carrying live SSE streams, embedded Monaco diff editing, and chart-heavy dashboards. The causal-inference sidecar is written in Python/FastAPI, since the scientific-Python ecosystem, and the Hypothesis property-testing stack [9] that the QA agent depends on, both live there.

The data layer pairs PostgreSQL 17 with pgvector [19], so that a single engine provides transactional tenant data, Row-Level Security [7], and vector retrieval for the knowledge base, which avoids standing up a separate vector database. Neo4j [3], meanwhile, holds the code dependency graph, because root-cause diagnosis is fundamentally a graph-traversal problem (hop distances, degrees, paths from symptom to cause) that a relational schema expresses poorly. Temporal [2] was proven in practice, since both live recovery runs reported in Chapter 6 executed durably end-to-end. Every component chosen is technology the team could run locally, and the entire platform was in fact built and verified on a single development machine, with Podman [11] standing in for Docker [10] as an API-compatible runtime wherever Docker itself was unavailable.

3.4.2 Economic Feasibility

The platform's dependency set is open source end to end: PostgreSQL, Neo4j Community, Redis, MinIO, Temporal, OpenTelemetry [5], and the Grafana observability stack, so that no licence cost attaches to development or evaluation. The one economically significant input is

LLM inference, and this is dual-path by design. Production targets OpenAI’s hosted models, while Ollama [20] serves fully supported local models (`gpt-oss:20b`, `qwen2.5-coder:14b`) as a zero-marginal-cost development and evaluation path. This is not merely a theoretical fallback, since every live end-to-end run reported in this document was driven by the local Ollama provider, which means the complete system, including its most expensive component, was exercised without incurring any per-token cost at all. The cost tracker, per-organisation token ledger, and daily budget controls exist precisely so that the paid path remains bounded and observable whenever it is used.

3.4.3 Operational Feasibility

Operationally, the system is designed to be run by the people it already serves, rather than by a separate specialist team. The console presents role-based dashboards: an administrator lands on organisation-wide KPIs, approvals, and health; an engineer lands on incidents and the live pipeline; a viewer sees a read-only picture of the same truth. High-stakes autonomy is reduced to a one-click human decision (Approve/Reject/Modify), which can optionally be taken directly from Slack [17] without opening the console at all. Severity classification, in turn, means that routine low-risk fixes demand no attention while sensitive changes always do. The NASA-TLX instrument [18] has been built into the platform precisely so that the operator-workload claim can eventually be measured rather than simply asserted. Taken together, these choices mean adopting Nexis does not require a dedicated operations team; the intended day-to-day operator is simply an ordinary engineer with a browser.

Chapter 4: System Design and Architecture

This chapter sets out the design of Nexis across several complementary levels of abstraction. The structural view occupies Sections 4.1 through 4.5, spanning five diagrams: the system context, the component architecture, the data and storage architecture, the self-healing agent pipeline, and the deployment and infrastructure architecture. Section 4.6 then turns to the functional view, using Level 0 and Level 1 use case diagrams to cover the three human roles together with the nine autonomous agents. The data view follows in Section 4.7, built around a context-level Data Flow Diagram (DFD) and two Level 1 decompositions, one for the Owner/Administrator control surface and the other for the self-healing loop itself. Section 4.8 closes the chapter with a Unified Modeling Language (UML) sequence diagram tracing a single complete recovery run, message by message, from fault detection through to post-deploy verification. Since the self-healing loop is the central contribution of this project, it has deliberately been given the most space of all the views presented here.

| 4.1 System Context

Figure 4.1 places Nexis, the multi-agent autonomous engineering platform, at the centre of its environment and shows every actor and external system with which it exchanges data. On the human side stand the three platform roles, namely Owner/Administrator, Engineer/Member, and Viewer, joined by the nine-agent self-healing fleet. The diagram draws this fleet as a fourth, autonomous actor that operates inside the platform boundary on the humans' behalf.

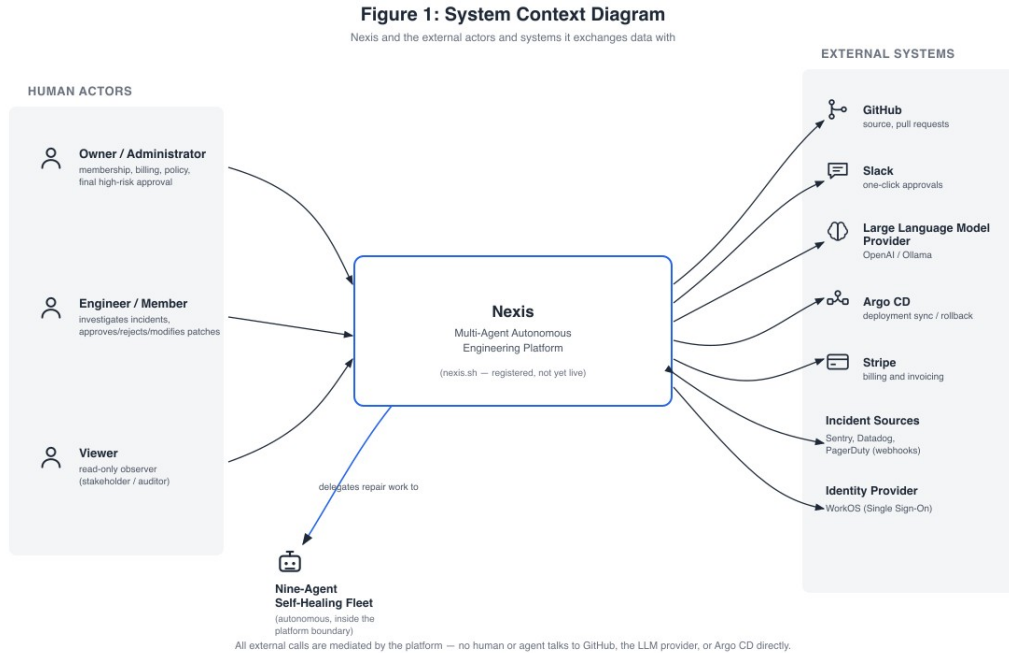


Figure 4.1: System Context Diagram. (Nexis and the external actors and systems it exchanges data with).

Every entry point into the platform converges on the same boundary: the web console (`apps/web`, Next.js 16), inbound GitHub webhooks, inbound alert webhooks from Sentry [14], Datadog [16], and PagerDuty [15], Slack interactivity callbacks [17], and plain Hypertext Transfer Protocol (HTTP) clients, that is, Command-Line Interface (CLI) tools such as `curl`, which address the deploy-engine and the public Application Programming Interface (API) directly. All of these are routed into the control-plane, since no client is permitted to talk to a data store or an external integration on its own.

On the external side, the diagram lists every third-party system that Nexis touches: GitHub for source repositories and pull requests, Slack for one-click approvals, the Large Language Model (LLM) provider (OpenAI or Ollama), Argo CD for deployment synchronisation and rollback, Stripe billing and invoicing [13], the incident sources (Sentry, Datadog, and PagerDuty) arriving as webhooks, the WorkOS identity provider for Single Sign-On (SSO) [12], and Amazon Simple Email Service (SES) or Simple Mail Transfer Protocol (SMTP) mail delivery. The property the diagram makes explicit in its footer is mediation: all external calls pass through the platform, and neither a human nor an agent is permitted to talk to GitHub, the LLM provider, or Argo CD

directly. It may be noted that the diagram also carries an honest status note: the `nexis.sh` domain has been registered, but the platform is not yet publicly live, a point to which Section 4.5 returns.

4.2 Component Architecture

Figure 4.2 opens up the platform boundary to show the components that sit inside it: the web console, the control-plane at the centre, the sidecar services around it, and the LLM provider on which they draw.

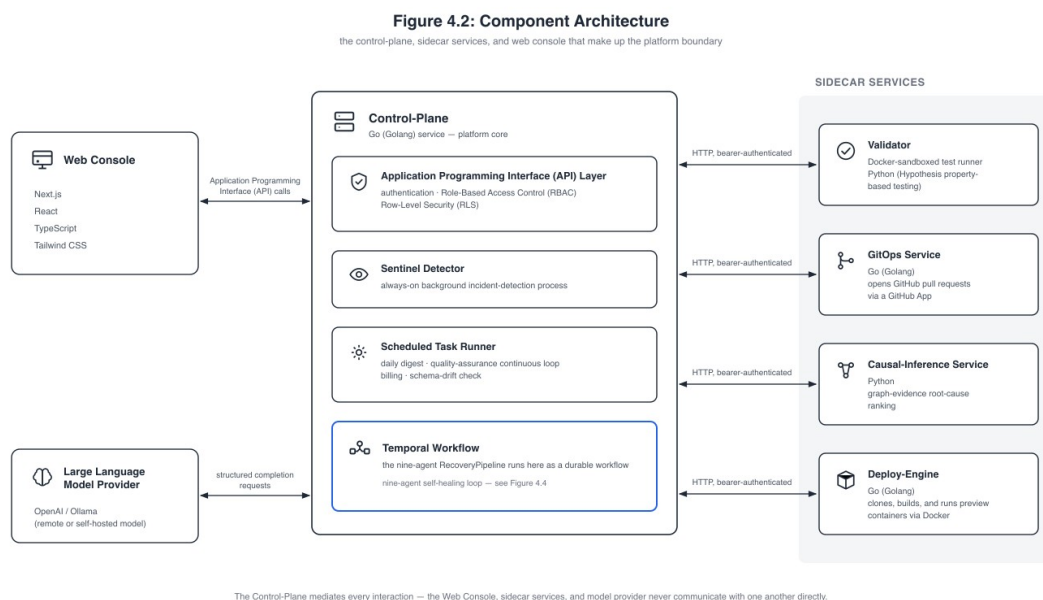


Figure 4.2: Component Architecture. (the control-plane, sidecar services, and web console that make up the platform boundary).

Web console. Built with Next.js 16, React, TypeScript, and Tailwind CSS, the console (`apps/web`) speaks to the control-plane exclusively through Application Programming Interface calls.

Control-plane. At the heart of the system sits a single Go (Golang) service (`:8080`) that hosts three things: the HTTP API layer (the chi router, with authentication, Role-Based Access Control (RBAC), Row-Level Security (RLS) binding, and rate limiting), the always-on Sentinel detector goroutine, which applies an exponentially weighted moving average (EWMA) $+3\sigma$ spike-detection rule, and the scheduled task runner (the daily digest, the Quality Assurance (QA)

continuous loop, billing tickers, and schema-drift detection). Inside this same service, the Temporal RecoveryPipeline workflow [2] orchestrates the nine-agent self-healing loop as a durable workflow: detect → diagnose → plan → design → code → test → package/migrate → approve (severity-routed) → deploy → verify → rollback if needed.

Large Language Model spine. Alongside the workflow sits the model layer. This comprises OpenAI (the gpt-4o family) as the production default and Ollama [20] (local gpt-oss:20b and qwen2.5-coder:14b) as a fully supported alternate provider; a pgvector retrieval store [19] that supplies code-context passages to the Architect, Backend, and Quality Assurance agents; and a structured-output stage that validates every agent completion against a per-agent JavaScript Object Notation (JSON) Schema, retrying on mismatch.

Sidecar services. Around the control-plane sit several independently deployable modules, each one reached over HTTP with bearer-token authentication from the control-plane:

- **Validator (:8081):** the Docker-sandboxed test runner [10] that shadow-executes every candidate patch under `--network=none`, `--read-only`, and `--cap-drop=ALL`, and that runs the Quality Assurance-generated Hypothesis property tests [9];
- **GitOps service (:8082):** GitHub App pull request (PR) automation (blob → tree → commit → ref → PR), bearer-authenticated from the control-plane;
- **Causal-inference service (:8090):** the Python/FastAPI graph-evidence ranker that scores Pathfinder’s candidate root causes by evidence-chain specificity, structural proximity, and textual overlap, an honest statistical proxy rather than a fitted structural causal model [4];
- **Deploy-engine (:8091):** clone, stack detection, Dockerfile generation, build, and health-checked container run for preview deployments.

Alongside these four, an Argo CD client module provides the Sync and Rollback integration [6] used by the post-deploy Service-Level Objective (SLO) probe. The diagram’s footer states the same mediation rule as the system context: the control-plane mediates every interaction, and the web console, the sidecar services, and the model provider never communicate with one another directly.

This component architecture is a direct reflection of the polyglot stack described in Chapter 3: Go (Golang) for every latency- and concurrency-sensitive service (the control-plane, Validator, GitOps service, and deploy-engine), Python for the statistics-oriented causal-inference sidecar, and TypeScript for the web console.

4.3 Data and Storage Architecture

Figure 4.3 shows the four storage systems, plus the workflow persistence layer, and sets out what each one is responsible for. The diagram also distinguishes between the two classes of write initiator, namely human-initiated writes arriving through the web console and the Application Programming Interface, and agent-initiated writes from the autonomous fleet. Its footer then makes a strong isolation claim: the control-plane is the only component holding credentials to any store, humans and agents never connect to a database directly, and reads flow back over these same mediated paths.

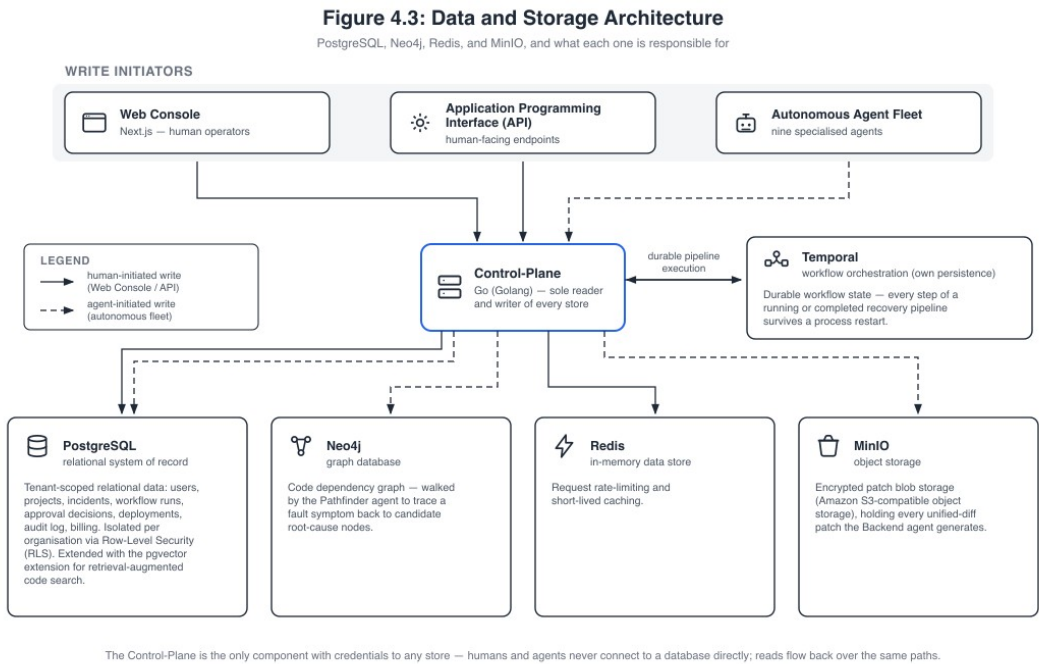


Figure 4.3: Data and Storage Architecture. (PostgreSQL, Neo4j, Redis, and MinIO, and what each one is responsible for).

PostgreSQL 17, with the **pgvector** extension, is the relational system of record, made multi-tenant through Row-Level Security policies [7] across 33 migrations. The key tenant-scoped tables (**users**, **org_members**, **projects**, **incidents_raw**, **workflow_runs**, **approval_decisions**, **deployments**, **lineage_events**, **audit_log**, **feedback_examples**, **role_recommendations**, and **digest_reports**) reappear as the data stores of the Data Flow Diagrams in Section 4.7.

Furthermore, **Neo4j** [3] holds the code dependency graph of each connected repository, which the Pathfinder agent walks to trace a fault symptom back to candidate root-cause nodes.

Temporal, for its part, maintains its own persistence for workflow durability, so that every step of a running or completed recovery pipeline survives a process restart. **Redis** serves request rate limiting and short-lived caching, while **MinIO** functions as the encrypted patch store: Amazon Simple Storage Service (S3)-compatible object storage that holds every unified-diff patch the Backend agent generates.

| 4.4 Self-Healing Loop Pipeline

Figure 4.4 presents the central contribution of the project as a single flow, namely the nine-agent detect-to-deploy pipeline, running as the durable Temporal RecoveryPipeline workflow introduced in Section 4.2. Reading the numbered stages in order: **Sentinel** detects the fault (a statistical spike or a fatal-level event); **Pathfinder** walks the Neo4j code graph to find candidate root causes; **Synthesiser** classifies the incident and selects which agents run; **Architect** writes the repair plan and the file-level contract; **Backend** generates the minimal code patch; **Quality Assurance** writes property-based tests for the patch; **DevOps** prepares the deployment manifests (the Continuous Integration / Continuous Deployment (CI/CD) and Argo CD manifests) in parallel with **Data Engineer**, which prepares any schema migration; and **Validator** runs the patch inside an isolated Docker sandbox together with the generated tests.

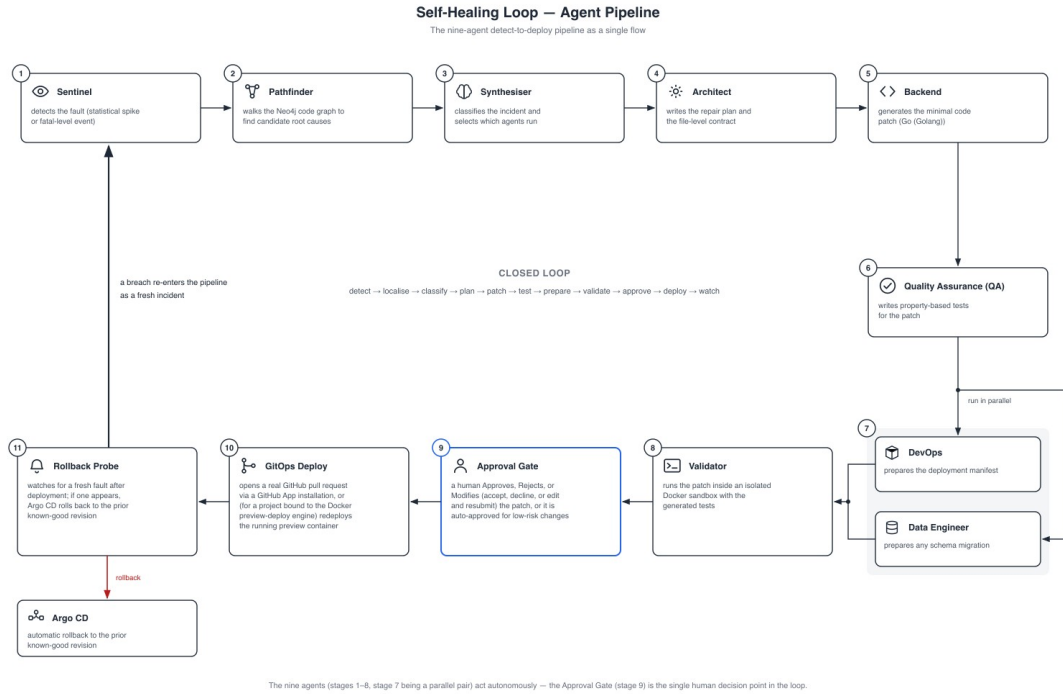


Figure 4.4: Self-Healing Loop — Agent Pipeline. (the nine-agent detect-to-deploy pipeline as a single flow).

Only then does a person enter the flow. At the **Approval Gate**, a human approves, rejects, or modifies the patch (that is, accepts it, declines it, or edits and resubmits it), unless it qualifies for auto-approval as a low-risk change. On approval, **GitOps Deploy** opens a real GitHub pull request via a GitHub App installation, or, for a project bound to the Docker preview-deploy engine, redeploys the running preview container. Finally, the **Rollback Probe** watches for a fresh fault after deployment; should one appear, Argo CD rolls back to the prior known-good revision, and the breach re-enters the pipeline as a fresh incident. The nine agents therefore act autonomously, with the Approval Gate standing as the single human decision point in the loop. The closing arc of the figure captures the property that gives the project its title: detect → localise → classify → plan → patch → test → prepare → validate → approve → deploy → watch, with the watch stage feeding back into detection. Sections 4.7.3 and 4.8 go on to decompose this flow into its data stores and its message-by-message timing.

4.5 Deployment and Infrastructure

Figure 4.5 shows how the platform is built, delivered, and observed. On the delivery side, GitHub Actions provides Continuous Integration, building and testing every commit and producing the container images; Docker, a Podman-compatible runtime, serves every service as well as the preview-deploy engine; and Argo CD supplies GitOps-style continuous delivery, syncing desired state and rolling back on failure. Terraform, in turn, declares the infrastructure as code, provisioning the production environment on Amazon Web Services (AWS).

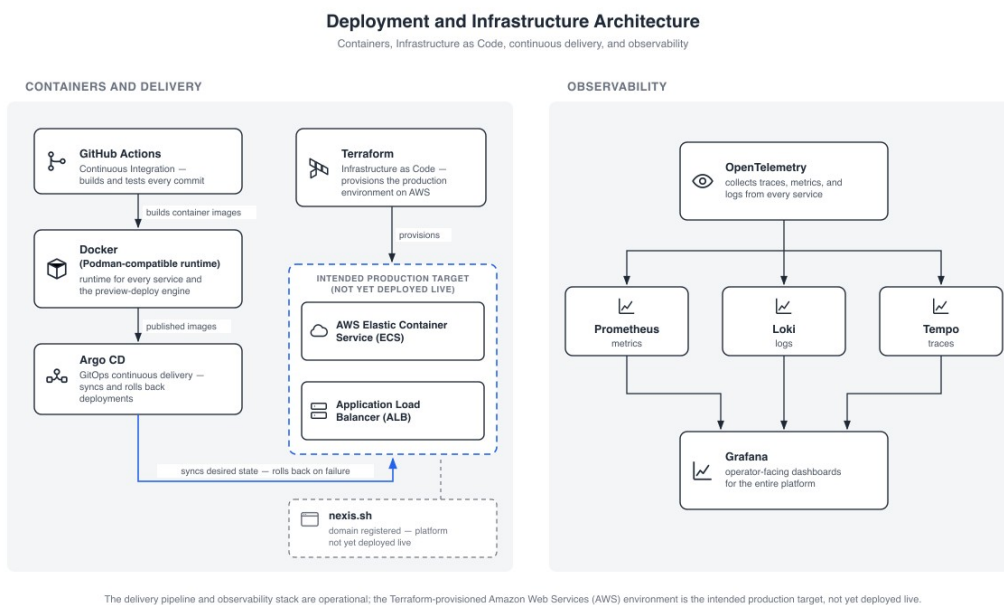


Figure 4.5: Deployment and Infrastructure Architecture. (containers, Infrastructure as Code, continuous delivery, and observability).

The diagram is deliberately honest about deployment status: the AWS Elastic Container Service (ECS) cluster behind an Application Load Balancer (ALB) is the *intended* production target declared in the Terraform code, but it has not yet been deployed live, and the `nexis.sh` domain, while registered, does not yet serve the platform. Observability across all layers follows the OpenTelemetry pipeline [5]: traces, metrics, and logs are collected from every service and flow into Prometheus for metrics, Loki for logs, and Tempo for traces, all of which are visualised in Grafana as the operator-facing dashboards for the entire platform.

The container runtime boundary is the Docker Engine Application Programming Interface, and Podman [11] satisfies this identically. This was more than a theoretical claim: the evaluation environment used for this report actually ran the Validator sandbox and the deploy-engine on Podman.

4.6 Use Case Diagrams

4.6.1 Use Case Diagram, Level 0

Figure 4.6 shows the system at its coarsest functional granularity, with a single system boundary, the three human actors and the autonomous nine-agent fleet placed on the left, and the external systems on the right.

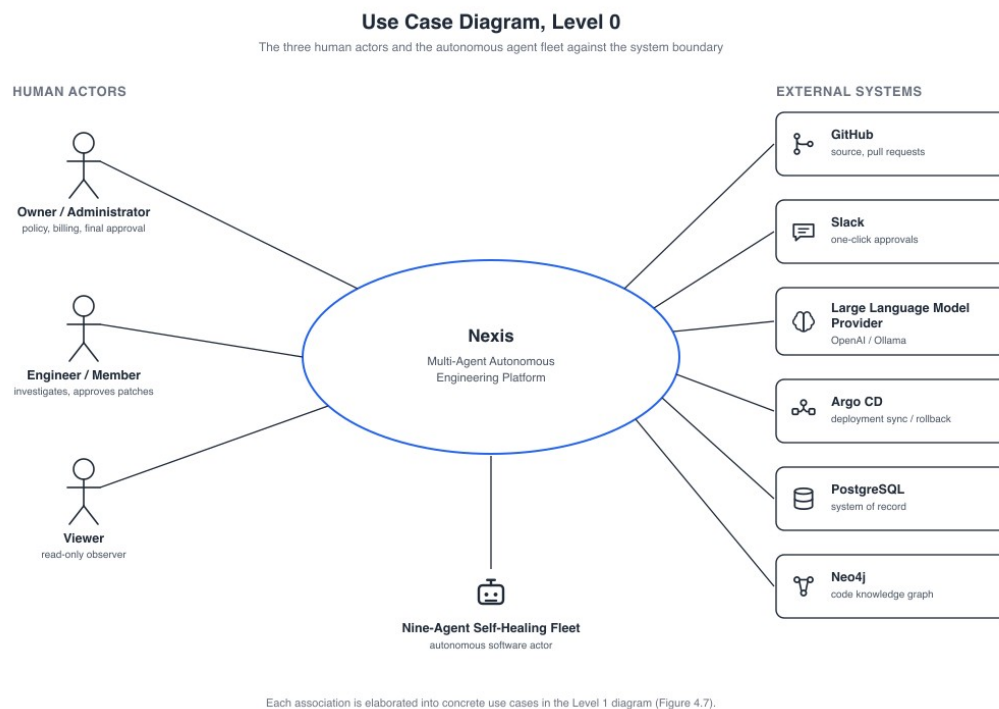


Figure 4.6: Use Case Diagram, Level 0. (the three human actors and the autonomous agent fleet against the system boundary).

The three human actors correspond to the platform’s role-based access model.

Owner/Administrator is the role that administers the organisation, covering membership, roles, integrations, recovery policy, and billing, and, most importantly, this role owns the approval gate

through which every autonomous repair must pass. Day-to-day engineering work, on the other hand, falls to the **Engineer/Member**: running demo scenarios, browsing incidents, inspecting agent runs, and participating in the approval flow. A read-only experience is what the **Viewer** gets, surfaced through Role-Based Access Control gating in the console, so that dashboards, incidents, and system health can be observed but nothing can be changed. The fourth actor, the **nine-agent self-healing fleet**, is an autonomous software actor to which the human roles delegate repair work, and it surfaces back to a person only at the approval gate.

On the right-hand side sit the external systems that the platform mediates: GitHub (source and pull requests), Slack (one-click approvals), the Large Language Model provider (OpenAI or Ollama), Argo CD (deployment sync and rollback), and the platform's own stores, PostgreSQL and Neo4j. (The remaining integrations, that is, the alert sources Sentry, Datadog, and PagerDuty, along with Stripe billing and the MinIO patch store, appear instead in the system context of Section 4.1 and in the Data Flow Diagrams of Section 4.7.) In this regard, the essential property that the diagram expresses is mediation: every association from an actor terminates at the system boundary, and every line to an external system originates from it. No human ever calls GitHub, the Large Language Model provider, or Argo CD directly; rather, it is the nine autonomous agents that act on the external systems on the humans' behalf. Each association is elaborated further into concrete use cases in the Level 1 diagram.

4.6.2 Use Case Diagram, Level 1

Figure 4.7 decomposes the single Level 0 bubble into the concrete use cases belonging to each actor. The diagram groups these use cases by the minimum role required: an administrative tier for the Owner/Administrator, a shared approval flow, an engineering tier for the Engineer/Member, a read-only tier for the Viewer, and the autonomous nine-agent fleet placed on the right. Hollow-triangle generalisation arrows make the role inheritance explicit throughout.

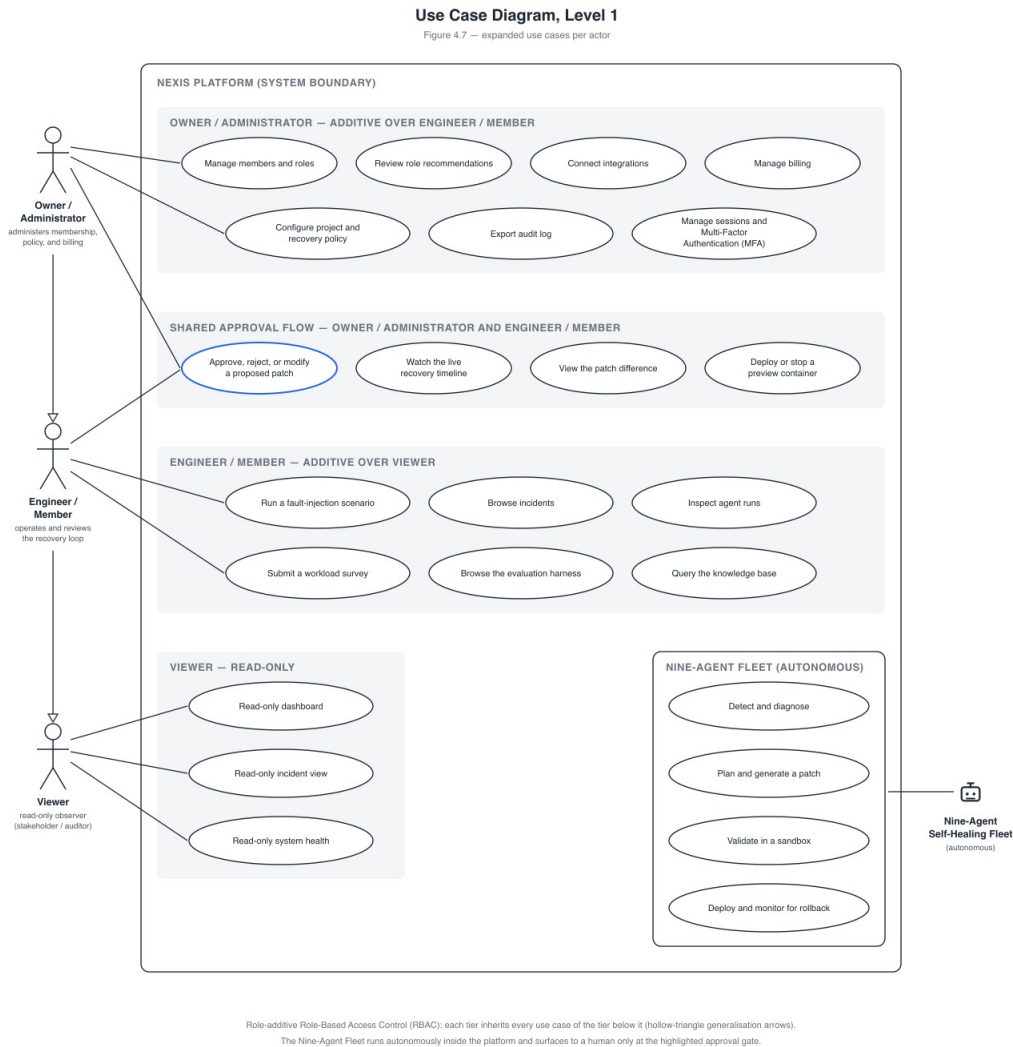


Figure 4.7: Use Case Diagram, Level 1. (expanded use cases per actor).

Owner/Administrator. The administrative surface covers fifteen use cases in total, which the diagram condenses into its grouped administrative tier: managing members and roles; accepting or dismissing role recommendations produced by the heuristic recommender; connecting integrations; creating and configuring projects; setting the recovery policy and Service-Level Objective; toggling the kill switch; issuing invites and Application Programming Interface keys; managing billing and plan; exporting the Reinforcement Learning from Human Feedback (RLHF) training data (the JSON Lines (JSONL) feedback export); viewing the audit log; triggering a manual Sentinel run; exporting evaluation-harness and audit Comma-Separated

Values (CSV) files; revoking sessions; resetting passwords; and enrolling or verifying Multi-Factor Authentication (MFA).

Shared approval flow. Since both roles participate in reviewing autonomous work, four use cases are shared between Owner/Administrator and Engineer/Member: approving, rejecting, or modifying a proposed patch; watching the live recovery timeline over Server-Sent Events; viewing the patch difference (diff) in the Monaco viewer; and deploying or stopping a preview container from the project's Operations (Ops) tab.

Engineer/Member. The engineering surface covers running a Live-Demo fault-injection scenario, browsing the incidents list, inspecting agent runs, submitting the NASA Task Load Index (NASA-TLX) workload survey [18], browsing evaluation harness runs, querying the knowledge base, viewing cost and performance pages, managing the member's own Application Programming Interface keys, updating profile and preferences, managing the member's own sessions and devices, and, finally, the account lifecycle itself (sign up, sign in, Multi-Factor Authentication verification).

Viewer. Exactly three use cases belong to the Viewer: read-only dashboard, read-only incident view, and read-only system health.

The nine-agent fleet. The fleet appears as a single autonomous actor whose grouped use cases summarise the recovery pipeline: detecting and diagnosing an incident (Sentinel and Pathfinder); planning and synthesising the repair (Synthesiser and Architect); generating code and tests (Backend and Quality Assurance) while packaging deployment and migrations (DevOps in parallel with Data Engineer); shadow-validating the patch (the Validator sandbox); and opening the pull request, redeploying, or rolling back on a Service-Level Objective breach (GitOps deploy and the rollback probe). Its only connection to a human use case is the approval gate, since the fleet otherwise executes autonomously inside the boundary and surfaces to a person exactly once per run.

The permission model is *role-additive*: Owner/Administrator holds every Engineer and Viewer permission, and Engineer holds every Viewer permission. Hence, the diagram draws the tiers as a generalisation chain, with each tier inheriting every use case of the tier below it; a higher role never loses a capability that a lower role has, it only gains administrative ones on top. The read-

only Viewer experience is enforced by Role-Based Access Control gating in the console, even though the underlying database role enumeration is `owner/admin/member`, which is itself a design decision discussed in Chapter 3.

4.7 Data Flow Diagrams

4.7.1 Data Flow Diagram, Level 0 (Context)

Figure 4.8 gives the context diagram: the entire platform, that is, the control-plane plus the nine-agent loop, drawn as a single process 0, together with its external entities and the three top-level data stores.

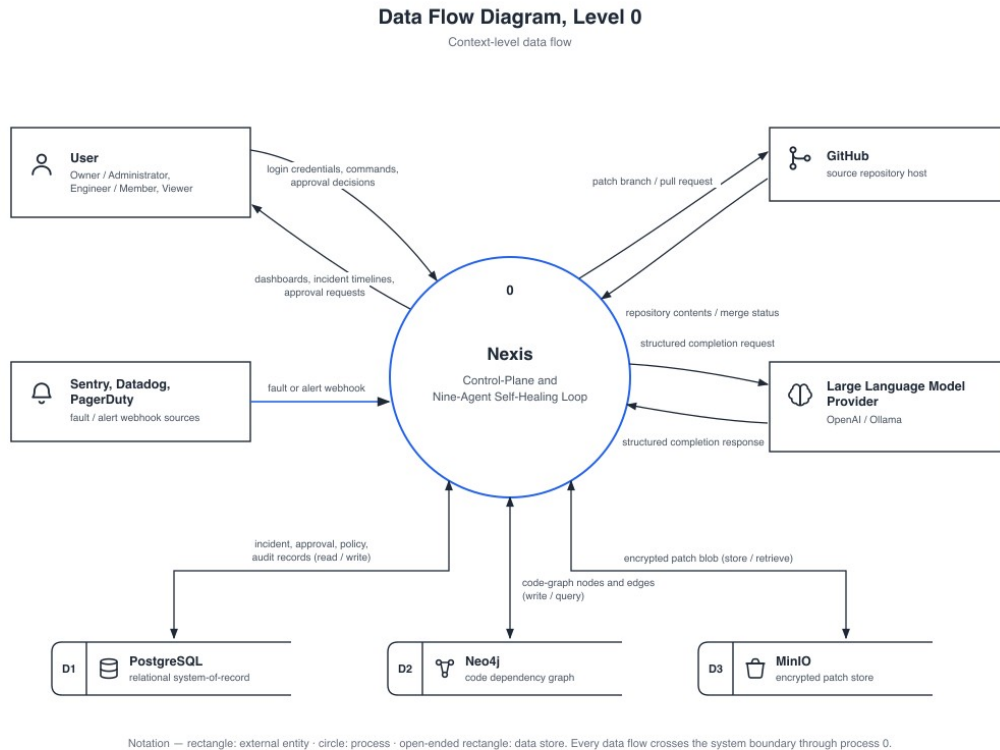


Figure 4.8: Data Flow Diagram, Level 0. (context-level data flow).

The data stores are labelled D1 to D3. **D1, the tenant database (PostgreSQL)**, receives every organisational record, including users, memberships, projects, incidents, workflow runs, decisions, and audit entries, all under Row-Level Security. **D2, the code graph (Neo4j)**, holds

the dependency graph of each connected repository and is read by Pathfinder during diagnosis. Finally, **D3, the patch store (MinIO)**, receives the encrypted diff blob produced by each recovery run.

Four external entity flows cross the boundary. The **user**, standing in for any of the three roles, sends login credentials, commands, and approval decisions inward, and receives dashboards, incident timelines, and approval requests in return. From **GitHub**, the platform receives patch branches and pull request creation requests (along with repository content reads, including the Dockerfile reads performed for the deploy-engine), and it returns pull request Uniform Resource Locators (URLs), commit identifiers, and merge status. The **alert sources** (Sentry, Datadog, and PagerDuty) send fault and alert webhooks inward only; they are producers of incidents, not consumers of data. Structured completion requests go out to the **Large Language Model provider** (OpenAI or Ollama), which returns patch and plan JavaScript Object Notation in response. As with Level 0 of the use case model, every one of these flows is mediated by the platform process. The diagram's notation note records that each data flow crosses the system boundary through process 0. As such, no direct path exists from a human to an integration.

4.7.2 Data Flow Diagram, Level 1 — Owner/Administrator

Figure 4.9 decomposes the administrative side of the platform into ten numbered processes, grouped into three clusters (identity and access administration, platform administration, and incident and approval operations), together with the tenant-scoped PostgreSQL stores that each cluster touches: D1 (identity and access: `users`, `org_members`, `sessions`, `role_recommendations`), D2 (platform data: `projects`, `integrations`, `billing`, `audit_log`), and D3 (incident and approval data: `incidents_raw`, `approval_decisions`, `workflow_runs`).

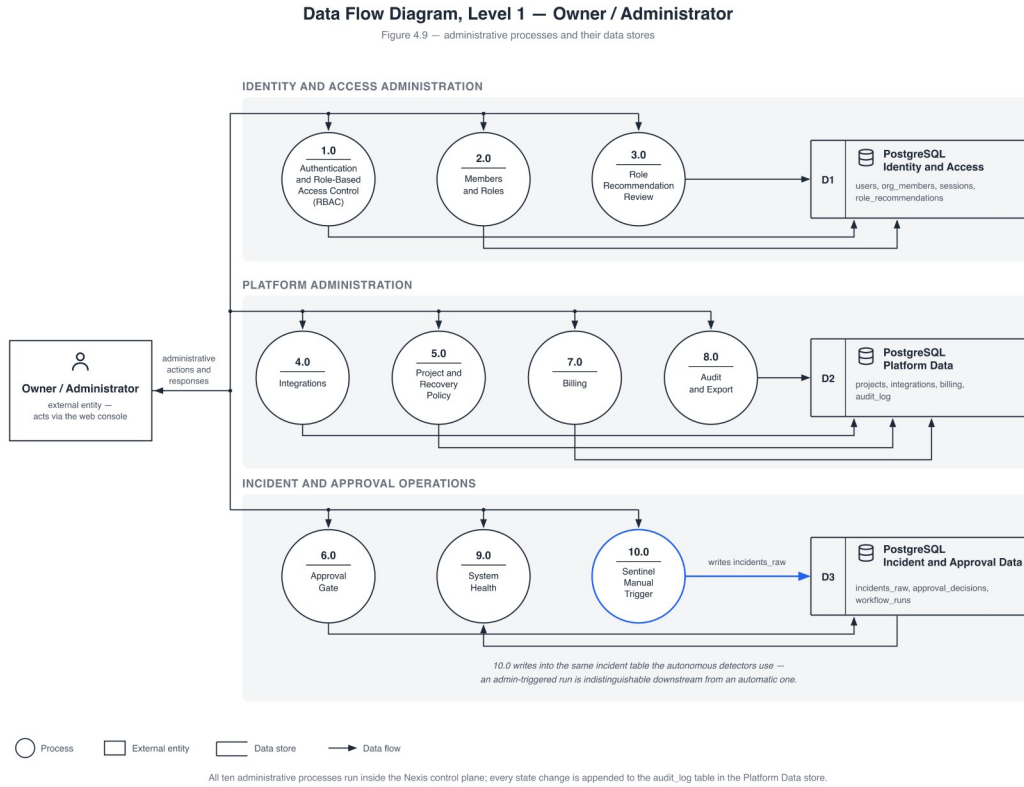


Figure 4.9: Data Flow Diagram, Level 1 — Owner / Administrator. (administrative processes and their data stores).

Table 4.1 summarises each of these processes, together with the primary data store each one writes to.

Table 4.1: Owner/Administrator Data Flow Diagram Level 1 processes and their primary data stores.

Process	Responsibility	Primary store written
1.0 Authentication and Role-Based Access Control	login, JSON Web Token (JWT) session issue, role checks	users, sessions
2.0 Members and Roles	invite, promote, demote, remove	org_members

Process	Responsibility	Primary store written
3.0 Role Recommendation Review	heuristic promote/downgrade suggestions	<code>role_recommendations</code>
4.0 Integrations	connect GitHub / Slack / Sentry / Argo CD	<code>integrations</code>
5.0 Project and Recovery Policy	project configuration, recovery policy, Service-Level Objective, kill switch	<code>projects</code>
6.0 Approval Gate	approve / reject / modify decisions	<code>approval_decisions</code>
7.0 Billing	Stripe usage ticker, invoices, plan	billing tables (Stripe)
8.0 Audit and Export	hash-chained audit writes, Comma-Separated Values export	<code>audit_log</code>
9.0 System Health	parallel dependency probes (read-only)	— (no persistent write)
10.0 Sentinel Manual Trigger	administrator-initiated detection run	<code>incidents_raw</code>

A brief walk-through of the diagram is useful here. Process 1.0 authenticates the administrator, binds the request-scoped Row-Level Security context, and owns the session surface, since session listing and revocation, along with Multi-Factor Authentication enrolment via Time-based One-Time Password (TOTP), both operate on the `sessions` table in the identity store. Processes 2.0 and 3.0 together form the membership surface, with the role recommender writing its suggestions to `role_recommendations` and the accept action flowing back through 2.0 into `org_members`. Moreover, processes 4.0 and 5.0 configure what the autonomous loop is allowed to act on; 5.0 also carries the kill switch, the safety brake that suspends autonomous recovery for a project by halting `workflow_runs` and writing the action to `audit_log`. Process 6.0 records every terminal approval decision, while 7.0 and 8.0 handle billing and the append-only, hash-chained

audit trail respectively. Process 9.0 fans out health probes to PostgreSQL, Redis, Neo4j, MinIO, and Temporal in parallel, and persists nothing itself. The diagram’s footer records the enclosing guarantee: all ten administrative processes run inside the Nexis control plane, and every state change is appended to the `audit_log` table.

The most important flow in the diagram is process **10.0, the Sentinel Manual Trigger**. When an administrator manually triggers a detection run, the process writes directly into `incidents_raw`, the *same* table that the self-healing loop’s automatic detectors write to. There is no separate “manual” code path, no parallel queue, and no privileged shortcut involved: an administrator-triggered run is indistinguishable downstream from an automatic one, and it flows through the identical Temporal pipeline, Validator sandbox, and approval gate. This design decision keeps the system’s behaviour uniform, and it means that every guarantee established for the autonomous path, namely durability, validation, severity routing, and auditability, holds for manual runs as well, at no extra engineering cost.

4.7.3 Data Flow Diagram, Level 1 — Self-Healing Loop

Figure 4.10 decomposes the nine-agent loop itself into twelve numbered processes, arranged by layer: the detection and decision agents at the head of the pipeline, the Layer-1 execution team in the middle, and the validation, approval, deployment, and verification tail. Around them sit the fleet’s five data stores, comprising D1 (`incidents_raw`), D2 (the Neo4j code graph), D3 (the MinIO patch store), D4 (`approval_decisions` and `feedback_examples`), and D5 (`deployments` and `lineage_events`), together with its four external entities: the fault telemetry sources, the Large Language Model provider, the Owner/Administrator, and GitHub.

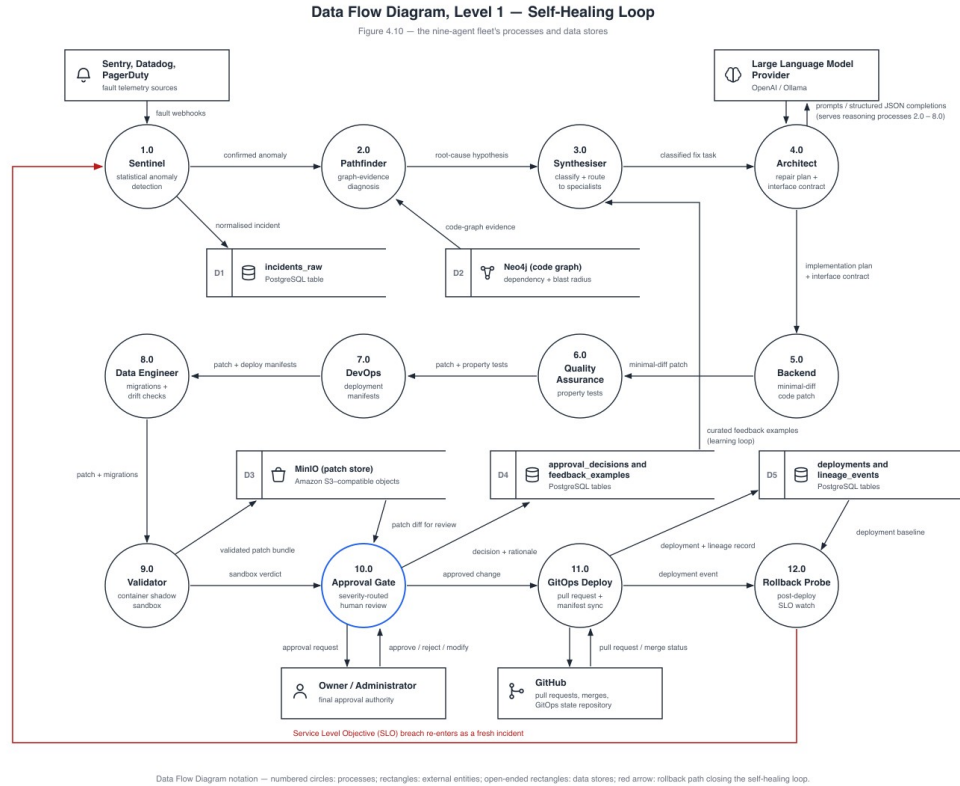


Figure 4.10: Data Flow Diagram, Level 1 — Self-Healing Loop. (the nine-agent fleet’s processes and data stores).

Read end-to-end, the pipeline proceeds as follows:

1. **1.0 Sentinel** receives fault signals, whether webhook payloads from Sentry, Datadog, or PagerDuty, or from its own exponentially weighted moving average $+3\sigma$ spike rule over the incident stream, and writes the normalised incident into `incidents_raw` (D1).
2. **2.0 Pathfinder** walks the Neo4j code graph (D2) outward from the fault symptom, and forwards the candidate root-cause nodes, complete with hop distance and degree metadata, to the causal-inference sidecar for graph-evidence ranking.
3. **3.0 Synthesiser** classifies the incident into a scenario, using a regular-expression fast path with a Large Language Model fallback, and computes the ordered `selected_agents` list that the workflow subsequently enforces.

4. **4.0 Architect** produces the structured plan, consisting of `plan_steps`, `affected_files`, and `risk_level`, which acts as the contract that every downstream agent is checked against.
5. **5.0 Backend** generates the minimal unified diff, guided by the Large Language Model provider, shown in the diagram as a dashed structured-JSON completion flow serving the reasoning processes 2.0 through 8.0.
6. **6.0 Quality Assurance** generates property-based tests directly from the incident and the Backend diff.
7. **7.0 DevOps** emits the Continuous Integration / Continuous Deployment and Argo CD manifests needed for the change.
8. **8.0 Data Engineer** proposes forward and reverse migrations wherever schema changes are involved, and emits OpenLineage-compatible run events [8] into `lineage_events` (D5).
9. **9.0 Validator** shadow-executes the patch inside the Docker sandbox, runs the Quality Assurance-generated test suite, and stores the encrypted candidate diff in the MinIO patch store (D3).
10. **10.0 Approval Gate** routes the validated patch, by severity, to the Owner/Administrator external entity (the approve/reject/modify flow in the diagram), and records the terminal decision in `approval_decisions` and `feedback_examples` (D4).
11. **11.0 GitOps Deploy** opens the real GitHub pull request, writing to the `deployments` store (D5) and out to the GitHub external entity (pull requests, merges, and the GitOps state repository), and it redeploys through the deploy-engine whenever the incident's source was a preview deployment failure.
12. **12.0 Rollback Probe** watches `incidents_raw` during a bounded post-deploy window, looking for a fresh fatal on the same organisation and service.

The diagram's closing annotation states the property that gives the project its title: **the loop is closed**. If process 12.0 observes a Service-Level Objective breach after a deployment, it does not raise an alert into some separate escalation channel; instead, the breach lands in `incidents_raw` as a fresh fatal, exactly as any external fault would, and re-enters the pipeline at process 1.0 as a new Sentinel-detected incident (with an Argo CD rollback to the prior known-good revision

when the project's `RollbackOnSLOBreach` policy is set). Recovery, verification, and re-detection thus form a single cycle over one shared data path.

4.8 Self-Healing Loop Sequence Diagram

Figure 4.11 is the Unified Modeling Language sequence diagram of one complete recovery run. All nine agents appear as lifelines, together with the fault source, the approval gate, the Owner/Administrator, and the GitOps and Deploy-Engine services. Eighteen numbered messages trace the run in temporal order, and each one corresponds to a named Temporal activity. Every one of them, moreover, is recorded as a durable ActivityEvent and streamed to the incident timeline over Server-Sent Events, which is why the live screenshots in Chapter 5 show this exact message order as a per-agent timeline.

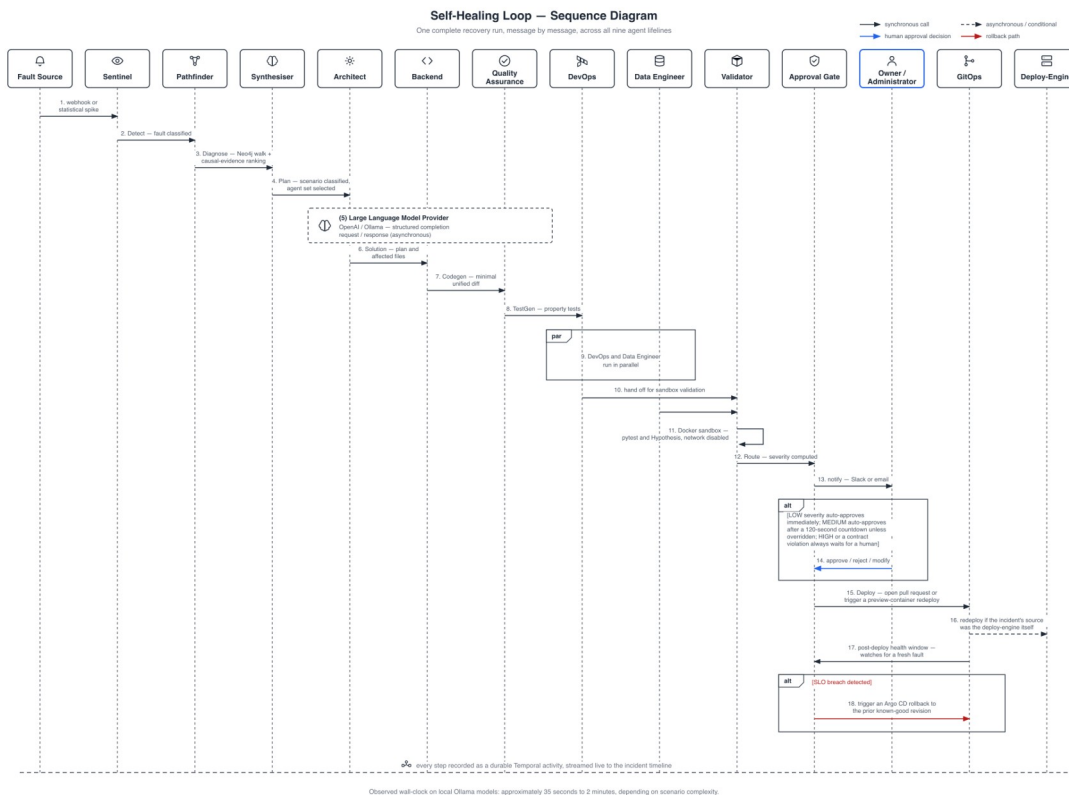


Figure 4.11: Self-Healing Loop — Sequence Diagram. (one complete recovery run, message by message, across all nine agent lifelines).

The run proceeds as follows:

1. **Message 1: fault arrival.** A webhook or statistical spike, fanned in from any incident source, reaches Sentinel. The resulting `incidents_raw` row is what the RecoveryPipeline trigger polls, so that workflow start is decoupled from signal arrival.
2. **Message 2: Sentinel.Detect.** The first workflow activity re-records the detection inside the durable workflow. This is, in effect, a stub, since the detection logic itself already ran in the always-on detector goroutine; the activity exists only so that the workflow timeline owns the provenance of the run.
3. **Message 3: Pathfinder.Diagnose.** Pathfinder walks the Neo4j dependency graph from the symptom and submits the candidates to the causal-inference sidecar for ranking.
4. **Message 4: Synthesiser.Plan.** The incident is classified into a scenario, and the agent set for this run is selected. Non-selected Layer-1 agents are not simply left out; they later appear as explicitly “skipped” in the timeline.
5. **Message 5: Large Language Model completion.** This is drawn as a dashed asynchronous flow spanning the agent lifelines. Every reasoning agent obtains its output as a structured JavaScript Object Notation completion, which is validated against that agent’s schema and retried on mismatch.
6. **Message 6: Architect.Solution.** The structured plan, with `plan_steps` and `affected_files`, is produced here, and this becomes the contract for the rest of the run.
7. **Message 7: Backend.Codegen.** The minimal unified diff is generated at this step. Should the diff touch any file outside the Architect’s `affected_files`, the workflow flags a contract violation, which in turn forces HIGH severity at the approval gate.
8. **Message 8: QA.TestGen.** Property-based tests are generated from the incident and the diff.
9. **Messages 9a/9b: DevOps.Pipeline in parallel with DataEngineer.Migrations.** This is the only parallel pair in the pipeline, drawn inside the diagram’s parallel (par) block. The Continuous Integration / Continuous Deployment and Argo CD manifests are prepared concurrently with the forward and reverse migration proposal, since neither depends on the other’s output.
10. **Message 10: hand-off to Validator.Validate.** The Backend result, together with the Quality Assurance suite, is passed on to the Validator.

11. **Message 11: sandboxed execution.** The Validator runs the patch inside the locked-down container (`--network=none`, `--read-only`, `--cap-drop=ALL`, `--pids-limit=128`, `--security-opt=no-new-privileges`), executes the `pytest/Hypothesis` suite, and reports back pass/fail status along with coverage. No unvalidated patch is able to proceed past this point.
12. **Message 12: ApprovalGate.Route.** Severity here is computed as a function of the scenario, sensitive-path matches, and contract violations. It is set to HIGH for the schema-drift scenario, for any diff touching a sensitive path glob (`*.sql`, `migrations/`, `auth/`, `security/`, `crypto/`), for an unknown scenario, or for a contract violation; MEDIUM for everything else; and LOW for small User Interface (UI)-only diffs.
13. **Message 13: notification.** For HIGH and MEDIUM runs, the gate notifies humans over Slack and email, and then waits.
14. **Message 14: human decision.** The Owner/Administrator's approve, reject, or modify action arrives as a Temporal signal. The subsequent alternative (alt) block encodes the automatic paths: LOW severity auto-approves immediately, while MEDIUM auto-approves after a 120-second countdown, unless a human overrides it first. HIGH severity, or any contract violation, always requires an explicit human decision. Indeed, one of the two live runs evidenced in Chapter 5 genuinely timed out at this gate, which is simply the safety mechanism working as designed.
15. **Message 15: GitOpsDeploy.** On approval, the GitOps service performs the blob → tree → commit → ref → pull request sequence against GitHub.
16. **Message 16: conditional DeployEngine.Redeploy.** This step executes only when the triggering incident's source was the deploy-engine, in which case a merged fix to a broken preview build triggers an automatic rebuild and redeploy of the preview container.
17. **Message 17: post-deploy health window.** The rollback probe watches `incidents_raw` for a fresh fatal on the same organisation and service, within the bounded window.
18. **Message 18: conditional ArgoCD.Rollback.** Inside the closing alternative (alt) block, a Service-Level Objective breach detected during the window triggers the Argo CD Rollback client to restore the prior known-good revision. As established in Section 4.7.3, the breach itself then re-enters the pipeline as a new incident.

The diagram's footer records the observed steady-state wall-clock time for a full run: approximately 35 seconds to 2 minutes on local Ollama models, depending on scenario complexity and on Large Language Model latency. This range is corroborated by the real timings visible in the Chapter 5 screenshots, one of which, the fully successful run, completed in 1 minute 40 seconds from detection through to verified deployment.

The chapter's views, taken as a whole, describe one coherent design. The platform boundary is mediated throughout (the system context and use case Level 0); the control-plane is the sole gateway to every component and store (the component and data architectures); a role-additive human surface sits beside an autonomous fleet (use case Level 1); manual and automatic operation share a single data path (Data Flow Diagram Level 1, Owner/Administrator); and a detection–repair–verification cycle feeds its own failures back into itself (the agent pipeline, Data Flow Diagram Level 1 for the self-healing loop, and the sequence diagram). Chapter 5 goes on to present the implementation of each of these elements, while Chapter 6 sets out the evidence that the loop actually closes in practice.

Chapter 5: Implementation

The system presented in this chapter is the one that actually runs today, not merely one that was designed on paper. Section 5.1 sets out the tools and technologies used at each layer of the stack, Section 5.2 walks through the platform screen by screen, and Section 5.3 closes with the database schema beneath the multi-tenant control plane. Every screenshot included here is an unretouched capture of the running development environment; where a screen shows an error, a pending state, or a feature that is not yet finished, that has been left in rather than swapped for a more flattering capture.

| 5.1 Tools and Technology

Nexis is organised as a polyglot pnpm + Turborepo monorepo, with each subsystem built in whichever language suits its role. Go (Golang) handles the latency- and concurrency-sensitive control plane and its sidecar services, Python handles the statistical causal-inference component, and TypeScript handles the web console.

5.1.1 Backend

- **Go 1.25** — four independent Go modules make up the server side of the platform: `services/control-plane` (the multi-tenant Application Programming Interface (API) and business logic), `services/validator` (the sandboxed patch-validation service), `services/gitops` (the GitHub-App deployment path), and `services/deploy-engine` (the Docker preview-deploy engine, described in Section 5.2.5).
- **Python / FastAPI** — `services/causal-inference` is a sidecar on port 8090 that ranks candidate root-cause nodes forwarded by the Pathfinder agent, using a graph-evidence scoring algorithm.
- **Temporal [2]** — the durable workflow engine orchestrating the nine-agent RecoveryPipeline. Every agent step runs as a Temporal activity, so that a crashed worker resumes from exactly where it left off, and every step is durably recorded as an `ActivityEvent`.

5.1.2 Frontend

- **Next.js 16 / React 19 / TypeScript** — the web console, `apps/web`, covering authentication, the administrator console, incident observability, settings, and the project Operations (Ops) tab.
- **Tailwind CSS version 4** and **Radix UI** — used for styling and accessible component primitives.
- **Monaco editor** — the in-browser editor component that renders and edits unified diffs inside the approval dialogs.
- **Recharts** — powers the dashboard, performance, and cost charts.
- **Server-Sent Events (SSE)** — used by the incident detail page to subscribe to a live event stream, so pipeline progress renders in real time without polling.

5.1.3 Data and Messaging

- **PostgreSQL 17** — the system of record, made multi-tenant via Row-Level Security (RLS) [7]; the schema has grown across 33 sequential migrations (Section 5.3).
- **pgvector** [19] — provides vector similarity search inside PostgreSQL, backing the knowledge-base retrieval feature.
- **Neo4j** [3] — holds the code dependency graph that the Pathfinder agent walks, from a fault symptom toward candidate root-cause nodes.
- **Redis** — handles rate limiting and caching.
- **MinIO** — Amazon Simple Storage Service (S3)-compatible storage for encrypted patch blobs.
- **Temporal persistence** — Temporal keeps its own durable event history alongside the application database.

5.1.4 Artificial Intelligence (AI) / Large Language Model (LLM)

- **OpenAI (GPT-4o family)** — the production-configured provider for agent reasoning and code generation.

- **Ollama** [20] — a fully supported alternate provider that runs local models (`gpt-oss:20b` and `qwen2.5-coder:14b`). All live-run evidence presented in this chapter was produced end-to-end through Ollama; no cloud LLM credentials were used at any point.
- **Hypothesis** [9] — the property-based testing library the Quality Assurance (QA) agent uses to auto-generate test suites; these are the same suites the Validator executes inside its sandbox.
- **Causal-inference sidecar** — a statistical/graph ranking algorithm, not a fitted structural causal model. A full DoWhy [4] model would need interventional production data that does not yet exist, and this limitation is stated openly here rather than being papered over.

5.1.5 DevOps and Infrastructure

- **Docker API** [10] — the container runtime for both the Validator sandbox and the preview-deploy engine. This integration is Podman-compatible [11]; in the evaluation environment used for this report, Podman was substituted for Docker and shown to work identically.
- **Terraform** — infrastructure-as-code targeting Amazon Web Services (AWS) Elastic Container Service (ECS) behind an Application Load Balancer.
- **OpenTelemetry** [5] — exports traces, metrics, and logs to Prometheus, Loki, and Tempo, visualised in Grafana.
- **Argo CD** [6] — the DevOps agent’s Sync/Rollback client integration for GitOps deployment and post-deploy rollback.

5.1.6 Integrated Development Environment (IDE) / Version Control

- **Git** — version control for the entire monorepo, including the 33 sequential database migrations tracked in the repository.
- **GitHub** — repository hosting; beyond hosting, though, GitHub is a first-class runtime dependency, since a GitHub App installation supplies short-lived tokens for repository cloning and for the GitOps service’s blob → tree → commit → ref → Pull Request (PR) deployment path.

- **pnpm workspaces + Turborepo** — handles monorepo task orchestration (build, typecheck, lint) across the web app and shared packages.

| 5.2 Project View

The screens that follow are grouped by functional area, in the order a new organisation would encounter them: authentication first, then the administrator console, member and session management, the self-healing loop itself (the centrepiece of this report), the Docker preview-deploy engine, the project and observability views, and integrations last.

5.2.1 Authentication and Onboarding

Account access is handled by a local JavaScript Object Notation (JSON) Web Token (JWT)-based session system, held in an HttpOnly cookie, with optional WorkOS [12] Single Sign-On (SSO) available as well. Figure 5.1 is the sign-up page, where the initial account gets created, and Figure 5.2 is sign-in, which establishes the session. The forgot-password flow (Figure 5.3) deserves a closer look: it issues a 32-byte random reset token, stored hashed with the Secure Hash Algorithm 256 (SHA-256) and given a 30-minute time-to-live (TTL), and it deliberately returns the same 202 response whether or not the e-mail exists, so that the endpoint leaks no account information.



Create your NEXIS account
Start with a personal org. Invite teammates later.

Email
demo@nexus.local

Password
.....

Org name

[Sign up](#)

Already have an account? [Sign in](#)



Figure 5.1: Sign-Up Screen. (new account registration).



Sign in to NEXIS
Welcome back. Use your email and password to continue.

Email
demo@nexus.local

Password
.....

[Forgot password?](#)

[Sign in](#)

Don't have an account? [Sign up](#)



Figure 5.2: Sign-In Screen. (JWT session login with HttpOnly cookie).

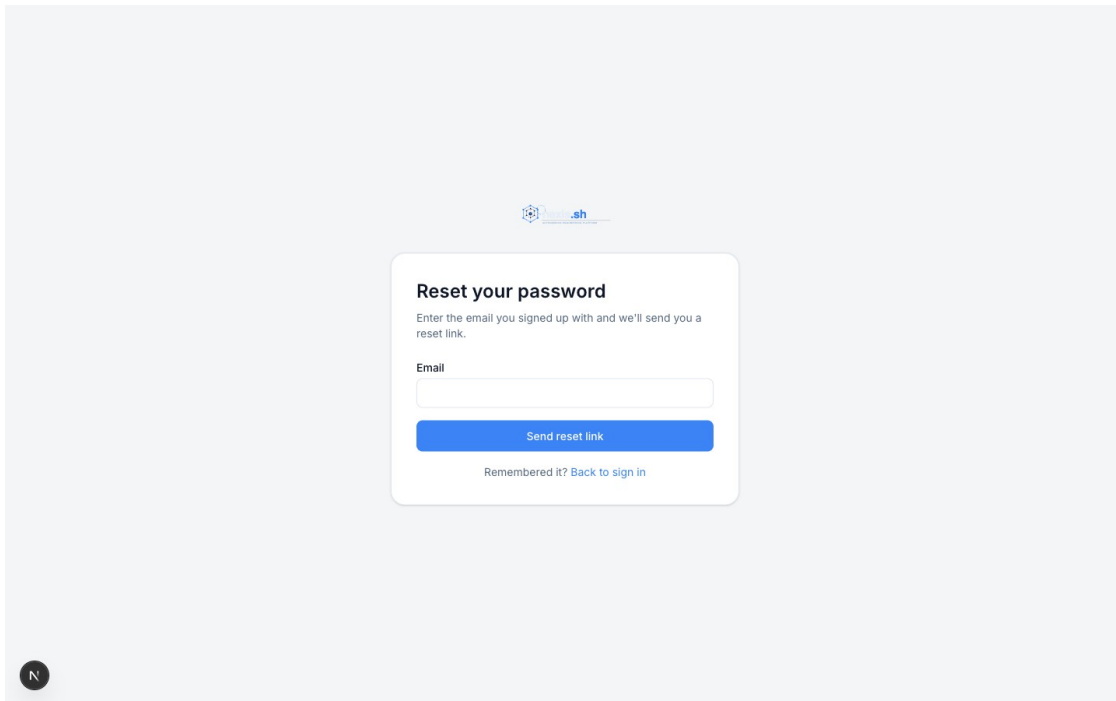


Figure 5.3: Forgot-Password Screen. (non-leaky password-reset request).

5.2.2 Administrator / Owner Console

The console home, captured in Figure 5.4, gives the organisation’s operational state at a glance: Key Performance Indicator (KPI) tiles for Mean Time To Recovery (MTTR), recovery success rate, open incidents, and mean tokens per run, alongside an incidents-over-time chart, a system status panel, a projects health grid, and a recent activity feed.

NEXIS

<<

Production

us-east-1

Home

Jump to...

HK

demo

Production

us-east-1

WORKSPACE

Dashboard

Projects

Incidents

Approvals

Audit Log

FLEET

Agents

9

Workflows

Activity Stream

Validator Sandbox

OBSERVABILITY

Performance

Cost Tracker

System Health

INTEGRATIONS

All Integrations

0/6

Webhook Activity

Settings

LIVE

Workspace Created

9h ago

91c3-ch1ac1c6..

1 issue

OPERATIONS

Good morning, demo

Demo Org · demo-org · OWNER

OPEN INCIDENTS

→ 0.0%

0

0 total last 14d

MTTR · 7D

→ 0.0%

—

Mean time to recover

ACTIVE WORKFLOWS

→ 0.0%

0

Idle

MTD SPEND

—

No cost data yet

TREND

Incidents over time

LAST 7D

Hourly buckets across the last 7 days.

No incidents in the last 7 days.

DISTRIBUTION

Severity breakdown

OPEN

Open incidents grouped by severity.

No open incidents.

OUTCOMES

Recovery success rate

LAST 14D

Daily breakdown of recovery pipeline outcomes.

No completed pipelines in the last 14 days.

COST TELEMETRY

Token usage by agent

LAST 14D

Synthesised daily distribution from the 14d rollout.

No agent telemetry yet.

Per-day shape is interpolated from the 14d total — exact daily series ships with the cost endpoint.

PATTERNS

Activity heatmap

LAST 30D

Incident volume by hour-of-day and day-of-week, last 30 days.

No incidents in the last 30 days.

System status

Full health board →

Sub-systems + integration health, polled every 30s.

CORE SUB-SYSTEMS

Control plane

API + auth + RLS

Temporal

Workflow engine

Postgres

OLTP + pgvector

Redis

Cache + pubsub

Edge / Proxy

No integrations

INTEGRATIONS

No integration connections yet. Connect one from [Integrations](#) to see live probe health.

Projects health

View all →

Auto-healing targets in this workspace.

No projects yet

Connect a service to start routing incidents.

+ Connect first project

Recent activity

View all →

Latest 10 audit events across the org.

ORGANISATION

Demo Ora

59

Figure 5.4: Administrator Dashboard. (KPI tiles, incidents-over-time chart, and system status panel).

Across both layers, the Agents page (Figure 5.5) lists the full nine-agent fleet: the five Execution Team agents (Architect, Backend, QA, DevOps, Data Engineer) and the Self-Healing Loop agents (Sentinel, Pathfinder, Synthesiser, Validator), each shown with its role and current status.

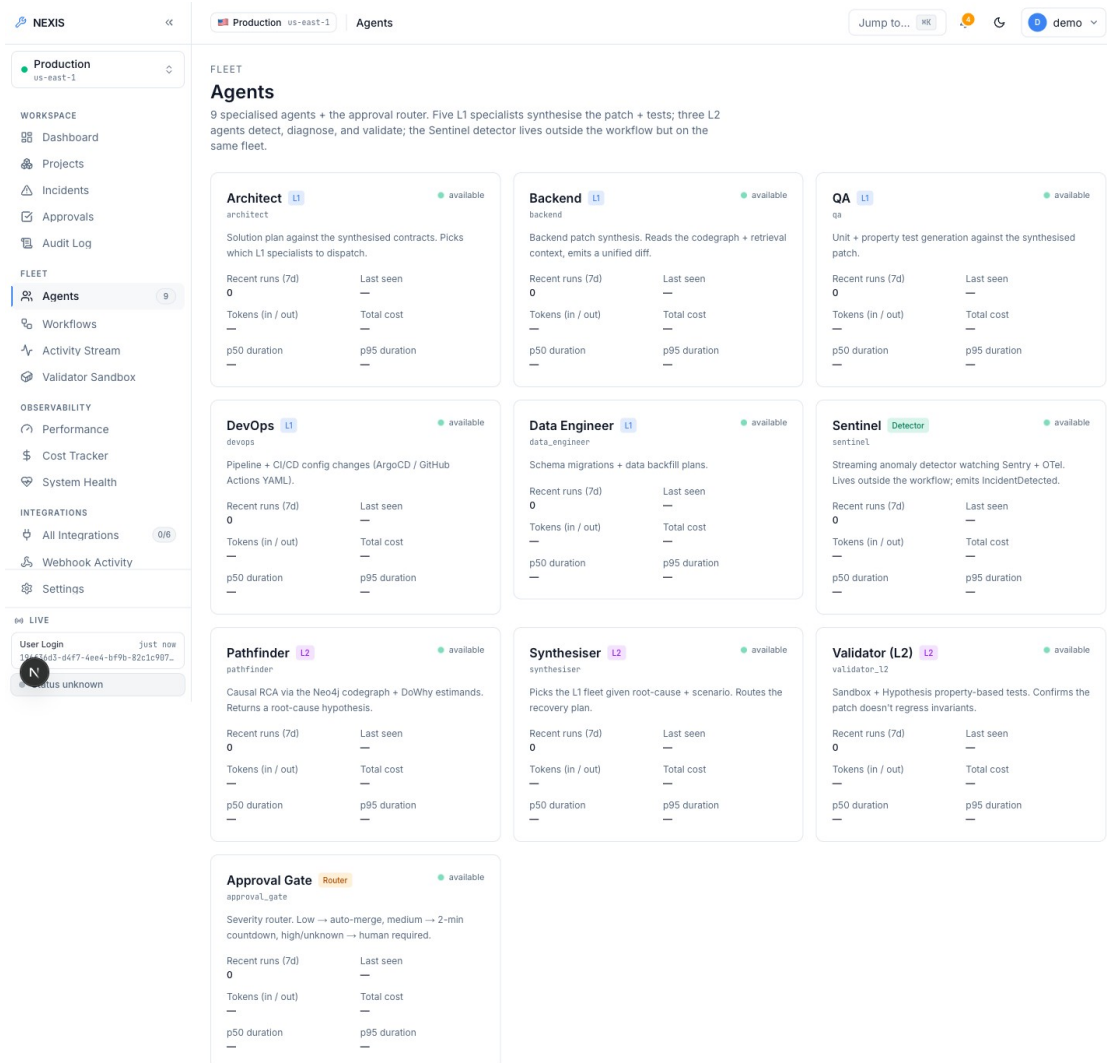


Figure 5.5: Agents Screen. (the nine-agent fleet roster across both layers).

Figure 5.6, the Workflows page, lists RecoveryPipeline executions. The Activity Stream (Figure 5.7) is the organisation-wide live event feed, and the Validator Sandbox console (Figure 5.8)

surfaces the Docker-sandboxed shadow-execution service where every candidate patch is run against its QA-generated property tests before it can reach the approval gate.

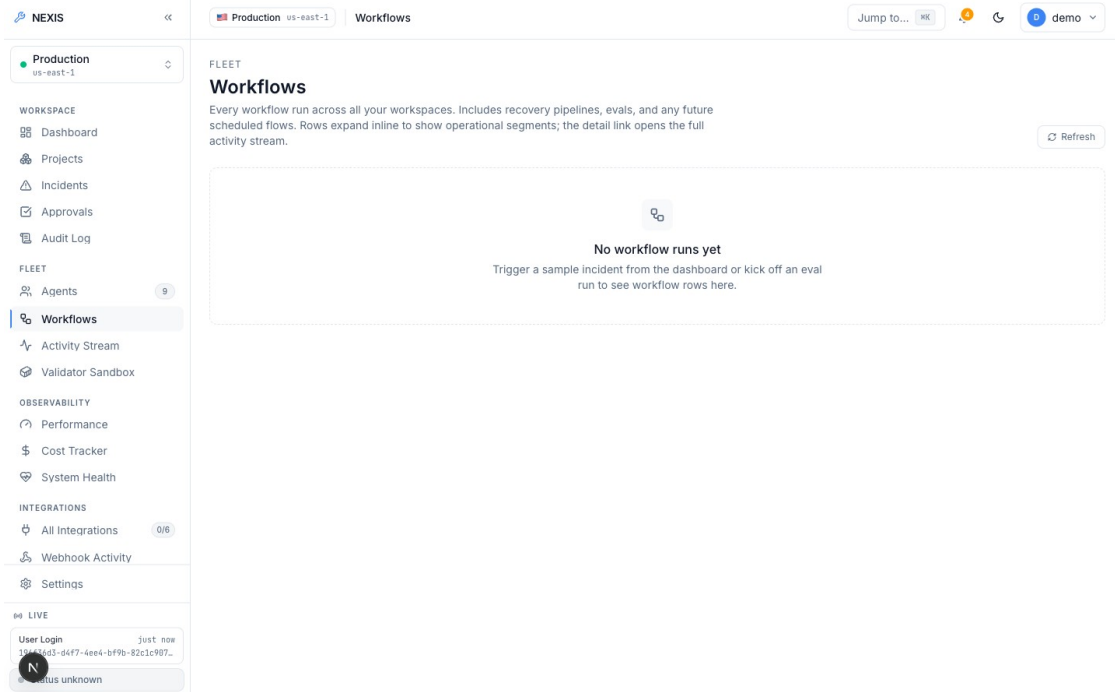


Figure 5.6: Workflows Screen. (RecoveryPipeline run listing).

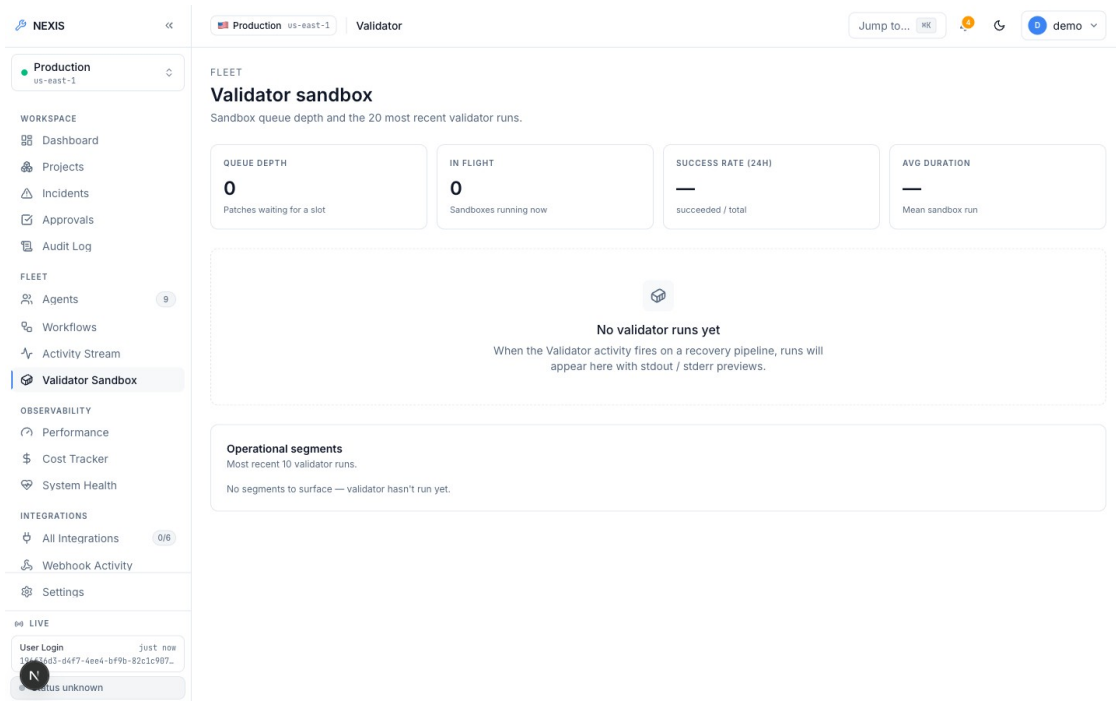


Figure 5.8: Validator Sandbox Console. (sandboxed shadow execution of candidate patches).

Append-only and hash-chained, the Audit Log (Figure 5.9) is built so that each entry incorporates the hash of the one before it; tampering with a past row therefore breaks the chain and becomes detectable through the cryptographic verify endpoint. It also supports Comma-Separated Values (CSV) export for offline review.

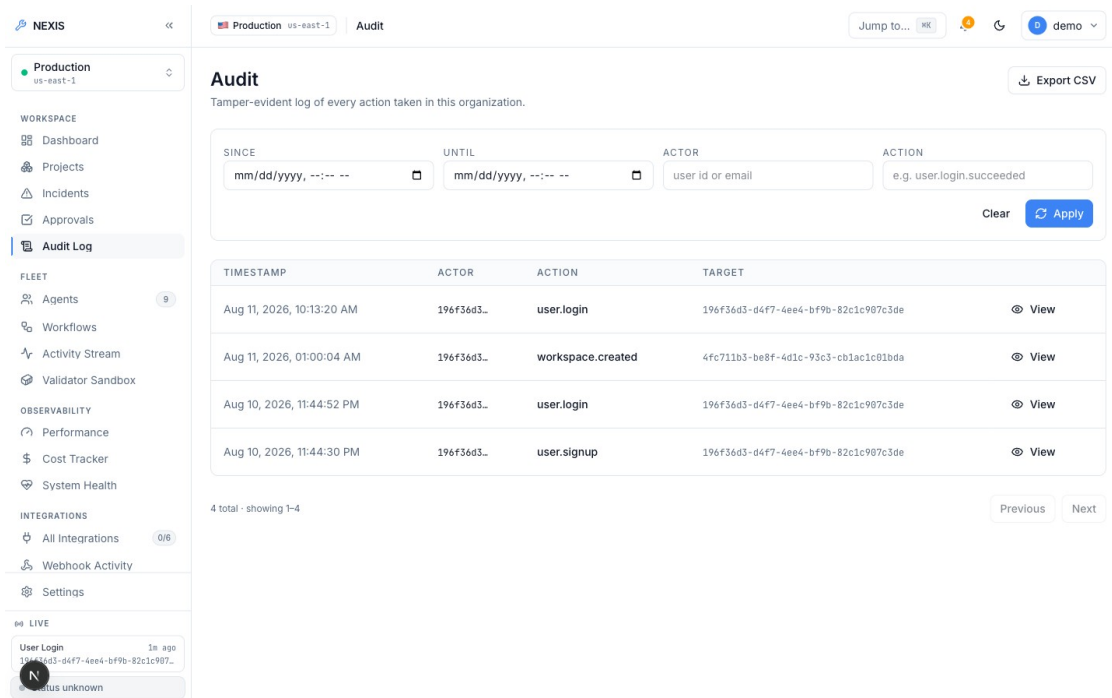


Figure 5.9: Audit Log. (append-only, hash-chained entries with CSV export).

5.2.3 Members, Roles, and Sessions

The Members settings page (Figure 5.10) lists the organisation’s members with their roles (owner, admin, member) and includes the role-recommendation panel. This is backed by the actual heuristic engine described in Chapter 4 rather than a mock, since it works from concrete signals already sitting in the schema. Three conditions trigger a recommendation: a member who is not yet an administrator but has logged five or more administrator-gated audit-log actions in the past 30 days; an administrator who has not logged in for 90 days or more; and an active administrator with zero administrator-gated actions to their name. Each recommendation comes with a plain-language rationale naming the specific evidence behind it, and an administrator may accept it, applying the role change under an optimistic-concurrency guard, or dismiss it, which starts a 30-day per-rule cool-down.

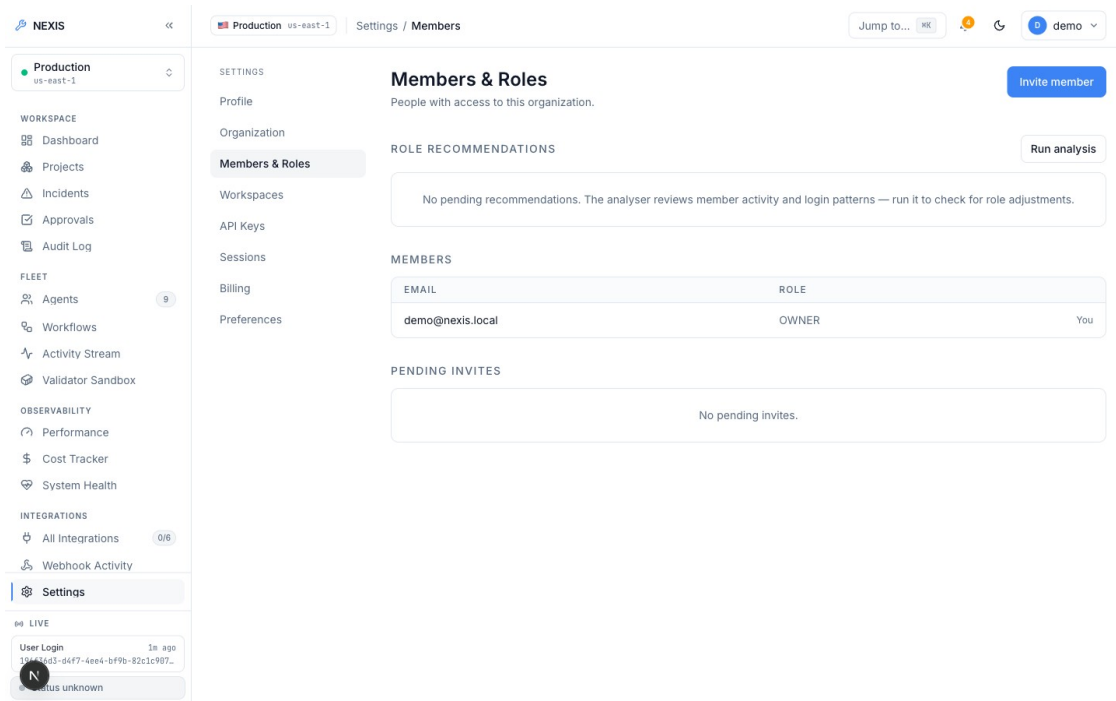


Figure 5.10: Members Settings. (members list with the heuristic role-recommendation panel).

Figure 5.11 covers session management: every active session is listed with its device, Internet Protocol (IP) address, user agent, and last-seen time, any session can be revoked remotely, and the caller’s own session carries a “This device” badge. API keys, shown in Figure 5.12, are scoped bearer tokens (nx_live_...) that can be created, listed, and revoked. Stripe [13] usage and payment-method management appear under Billing (Figure 5.13), while the Organization page (Figure 5.14) holds organisation-level profile settings.

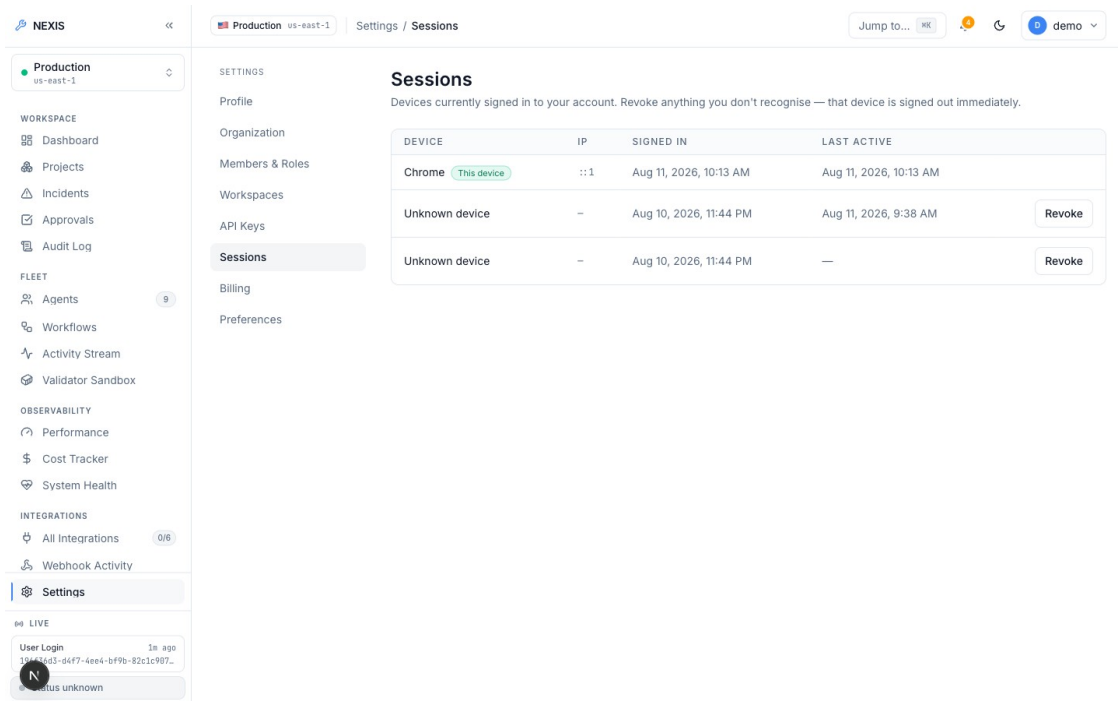


Figure 5.11: Sessions Settings. (active sessions with device, IP, and remote revoke).

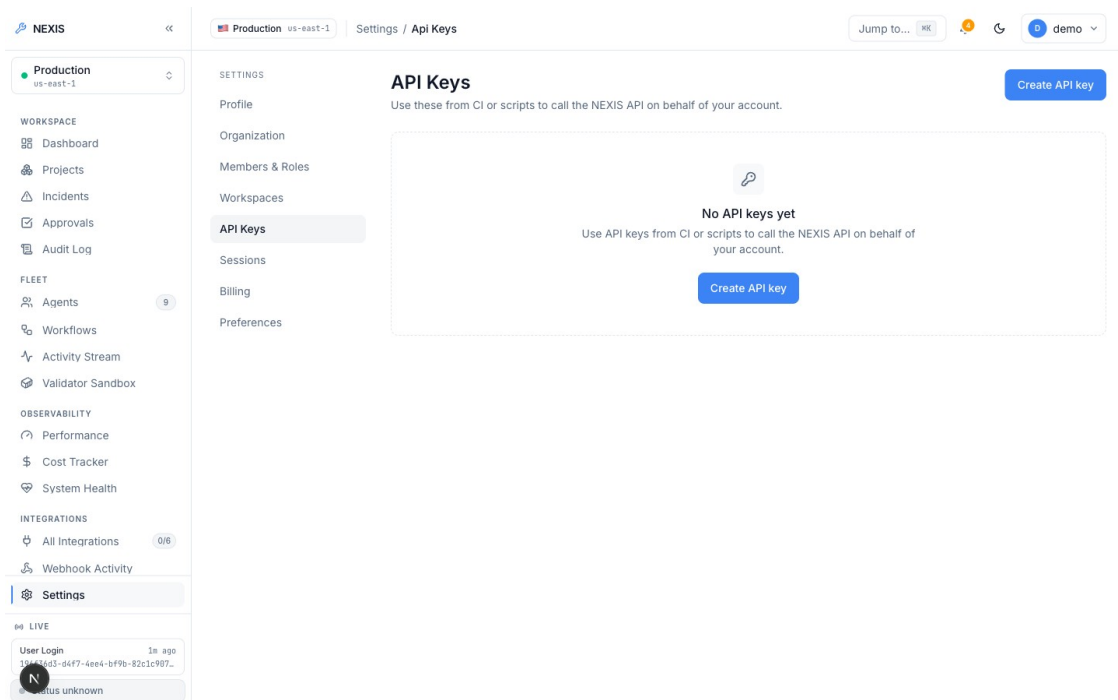


Figure 5.12: API Keys Settings. (scoped bearer key creation, listing, and revocation).

Production us-east-1

Workspace: Dashboard, Projects, Incidents, Approvals, Audit Log

Fleet: Agents (9), Workflows, Activity Stream, Validator Sandbox

Observability: Performance, Cost Tracker, System Health

Integrations: All Integrations (0/6), Webhook Activity

Settings

Profile, Organization, Members & Roles, Workspaces, API Keys, Sessions, **Billing**, Preferences

Billing

Manage your payment method, review usage, and download invoices.

PAYMENT METHOD

CARD NUMBER: 4242 4242 4242 4242

EXP MONTH: 12, EXP YEAR: 30, CVC: 123

POSTAL CODE: 10001, BILLING EMAIL: billing@example.com

[Use test card](#) [Save payment method](#)

USAGE

USAGE THIS PERIOD

\$0.20

BY WORKSPACE	
4fc711b3-be8f-4d1c-93c3-cb1ac1c01bda	\$0.20

BY KIND	
runtime_hours	\$0.20

INVOICES

No invoices yet — they'll appear at the end of your first billing cycle.

Figure 5.13: Billing Settings. (Stripe usage and payment-method management).

Production us-east-1

Workspace: Dashboard, Projects, Incidents, Approvals, Audit Log

Fleet: Agents (9), Workflows, Activity Stream, Validator Sandbox

Observability: Performance, Cost Tracker, System Health

Integrations: All Integrations (0/6), Webhook Activity

Settings

Profile, **Organization**, Members & Roles, Workspaces, API Keys, Sessions, Billing, Preferences

Organization

The tenant your account belongs to. Renaming + transfers coming soon.

NAME	Demo Org
SLUG	demo-org
ORGANIZATION ID	1d81b8e0-0172-4e07-9c17-fa99aad0aeb
YOUR ROLE	owner

Figure 5.14: Organization Settings. (organisation profile management).

5.2.4 Self-Healing Loop — Live Evidence

The evidence in this subsection carries the most weight in the report, since it comes from two real, complete, end-to-end executions of the nine-agent RecoveryPipeline against a live local LLM (Ollama, `qwen2.5-coder:14b` and `gpt-oss:20b`). One run timed out at the human approval gate and was safely auto-rejected; the other was approved within the window and succeeded in full. It may be noted that both outcomes are shown exactly as they occurred.

Everything starts from the Live Demo console (Figure 5.15), which presents 27 curated fault-scenario cards drawn from a broader fixture taxonomy spanning application, database, deploy, observability, and security categories. To put it precisely, of these 27 scenarios, exactly one is fully wired end-to-end at present; the remaining cards are User Interface (UI)-complete but backend-pending, labelled “Coming soon” in the interface itself. The runs documented in the rest of this subsection were triggered through that one wired path.

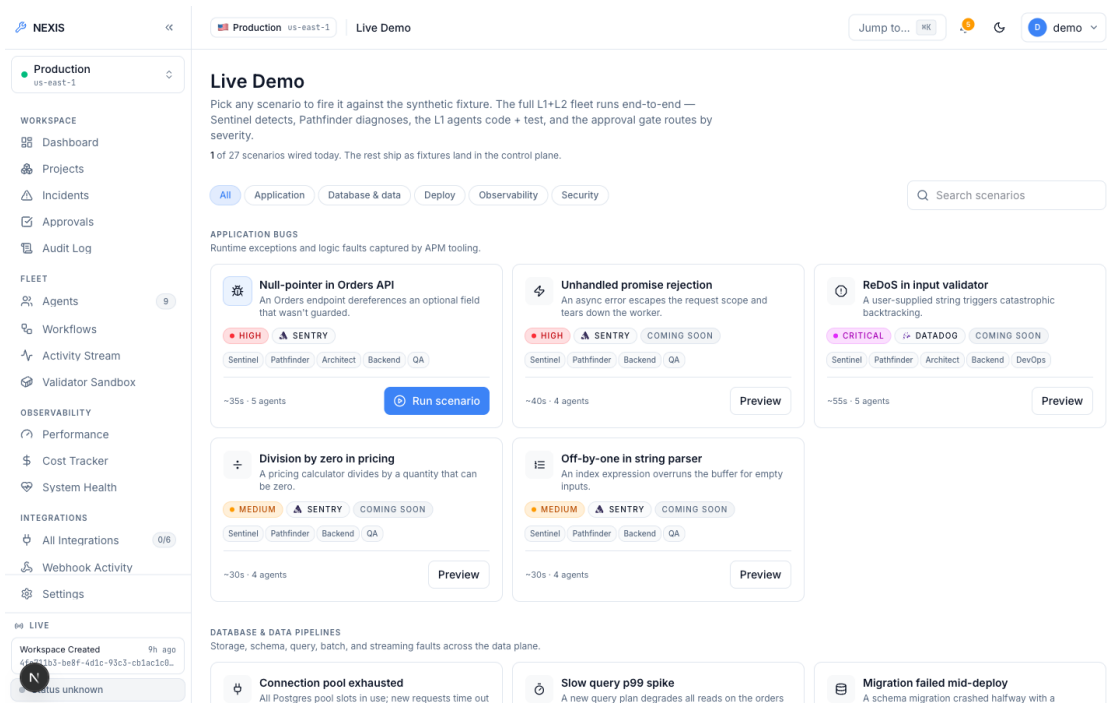


Figure 5.15: Live Demo Console. (27-scenario fault-injection picker; one scenario fully wired, the rest backend-pending).

Figure 5.16 catches the first run genuinely mid-flight: the Architect, Backend, and QA steps have already completed, each showing a real, non-zero token count and cost figure built up from

actual LLM calls, while the DevOps and Data Engineer steps run in parallel behind them. These visible token and cost numbers are evidence in their own right, since a stubbed or templated pipeline would report zero tokens, whereas every completed step here carries the token consumption of the model call that actually produced its output.

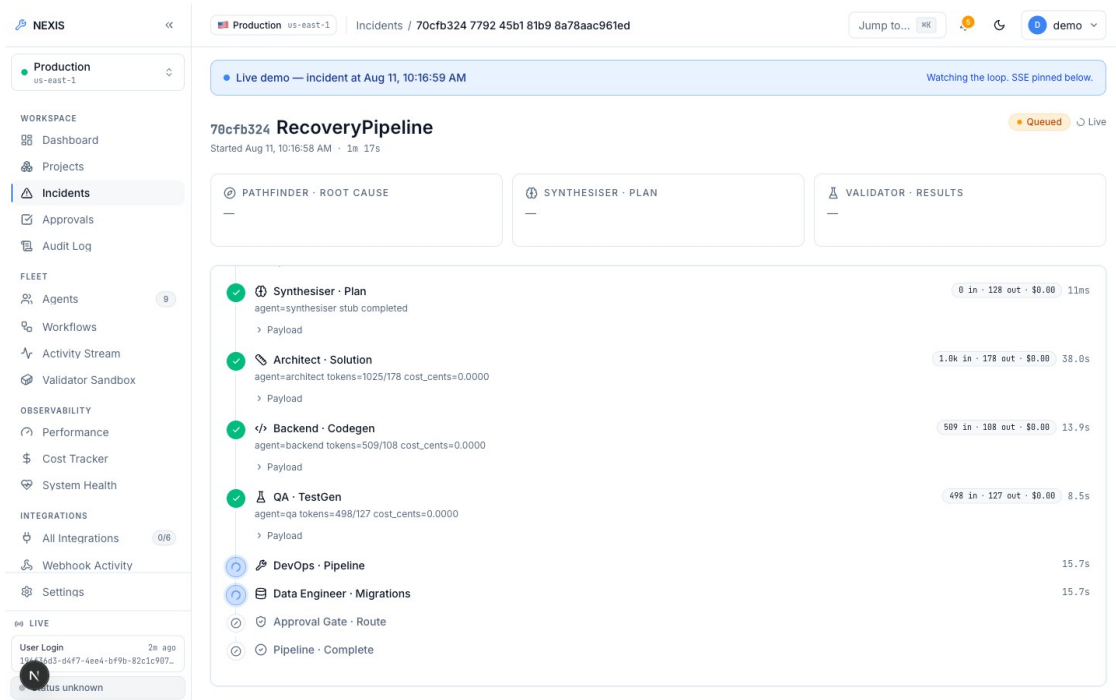


Figure 5.16: Incident Detail, Mid-Flight. (live pipeline with completed Architect/Backend/QA steps showing real token counts; DevOps and Data Engineer running in parallel).

Figure 5.17 shows the same run a little later. Every Layer-1 and Layer-2 step has now completed green: the patch has been generated, property-tested, and validated in the sandbox, and the run sits at severity “Medium” with approval status “Pending”, waiting on a human decision inside the 120-second window.

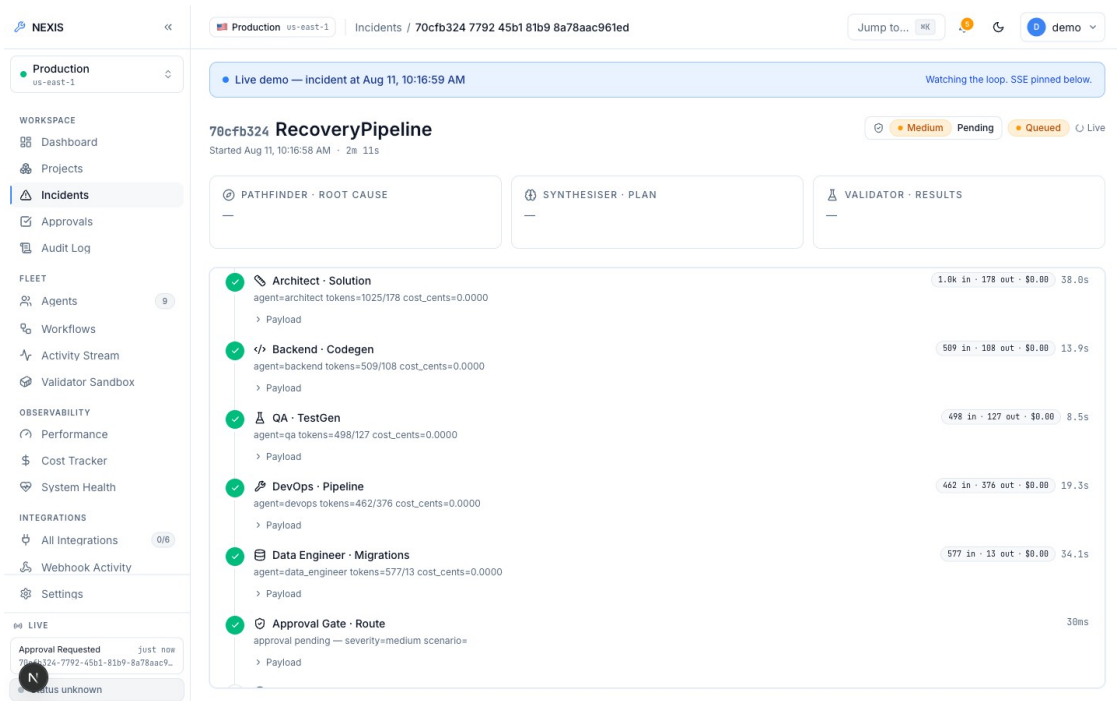


Figure 5.17: Incident Detail, Pipeline Complete. (all Layer-1 and Layer-2 steps green; severity Medium, approval Pending).

That pending state carries through to the Approvals queue (Figure 5.18), which shows a real pending-approval row with the three human decision actions: Reject, Modify, and Approve.

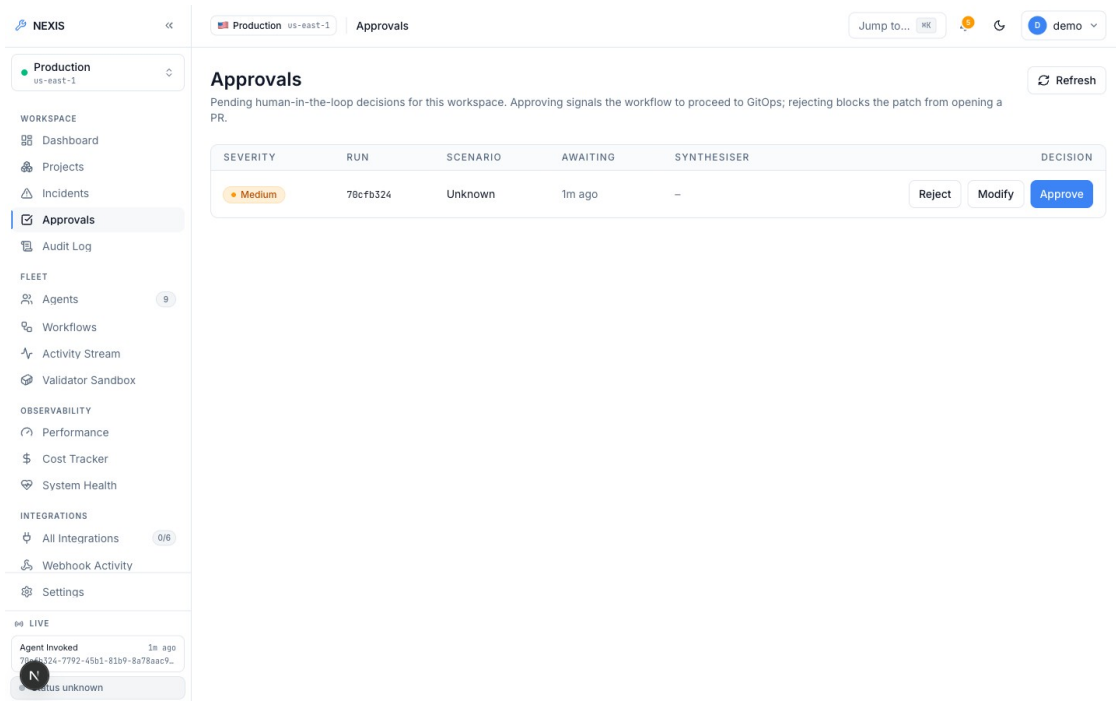


Figure 5.18: Approvals Queue. (real pending approval row with Reject / Modify / Approve actions).

Clicking Modify opens the dialog shown in Figure 5.19, and this is where the Reinforcement Learning from Human Feedback (RLHF) mechanism described in Chapter 4 becomes concrete: the engineer edits the LLM-generated unified diff directly inside the embedded editor before approving it. That edited diff, along with every other terminal approval decision, is persisted as a row in the `feedback_examples` table, and those rows feed a JSON Lines (JSONL) export endpoint intended for future fine-tuning. Actually submitting a paid fine-tuning job against the exported data remains a deliberate manual step rather than an automated one.

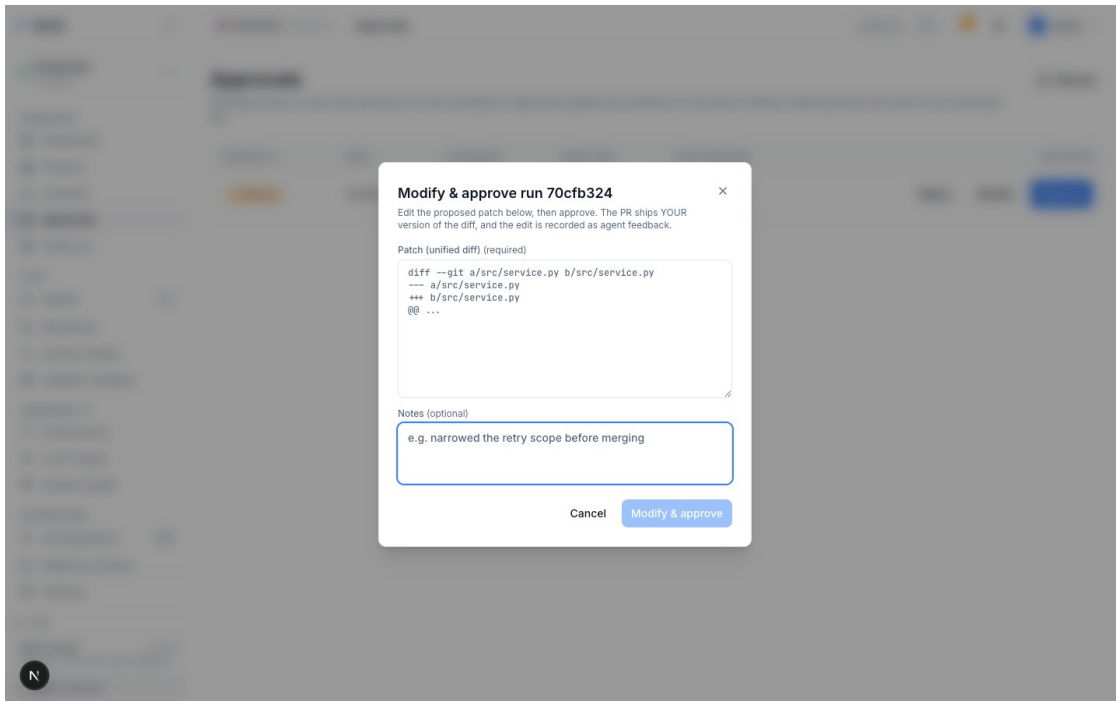


Figure 5.19: Modify-and-Approve Dialog. (RLHF diff editing; the engineer’s edit becomes a `feedback_examples` training row).

An honest negative result appears in Figure 5.20, included here deliberately rather than omitted. In this first run, the 120-second approval window elapsed before any decision was recorded, so the workflow terminated the run as `timeout_rejected`. This is the timeout safety mechanism functioning precisely as it was designed to: when no human decision arrives in time, the system’s default is to refuse deployment of unreviewed code rather than to proceed anyway. As such, a pipeline that silently auto-deployed on timeout would itself be a defect; here, that behaviour is the intended safe failure mode.

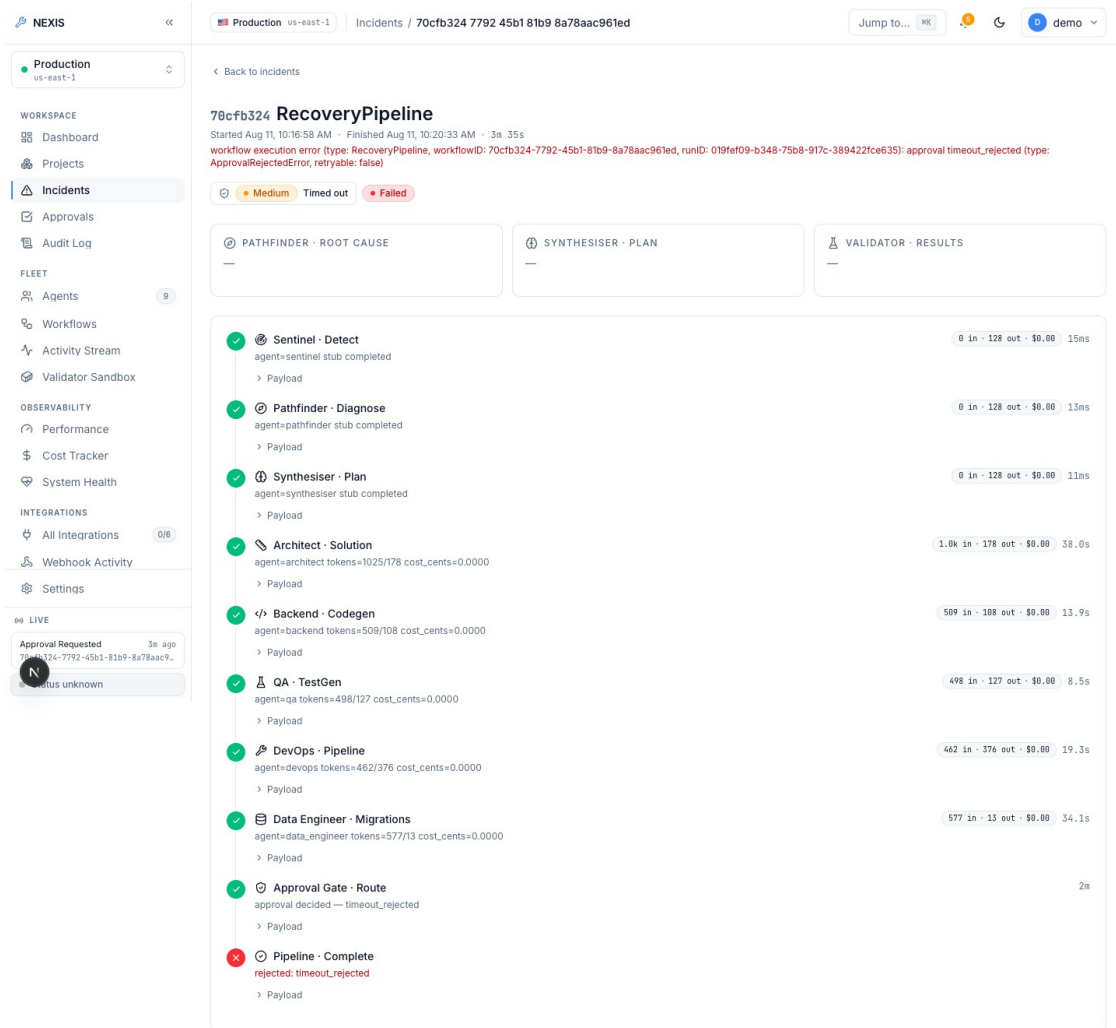


Figure 5.20: Timeout-Rejected Run. (the 120-second approval window elapsed; the workflow safely auto-rejected as `timeout_rejected` rather than deploying unreviewed code).

The key positive result of the report sits in Figure 5.21: a second live run, approved by a human inside the window. The incident shows severity “Medium”, decision “Approved”, and outcome “Succeeded”. Its full nine-step timeline, running from detection through plan, patch generation, test generation, validation, approval, and deployment, completed end-to-end in 1 minute 40 seconds, with real per-agent token and cost figures recorded throughout.

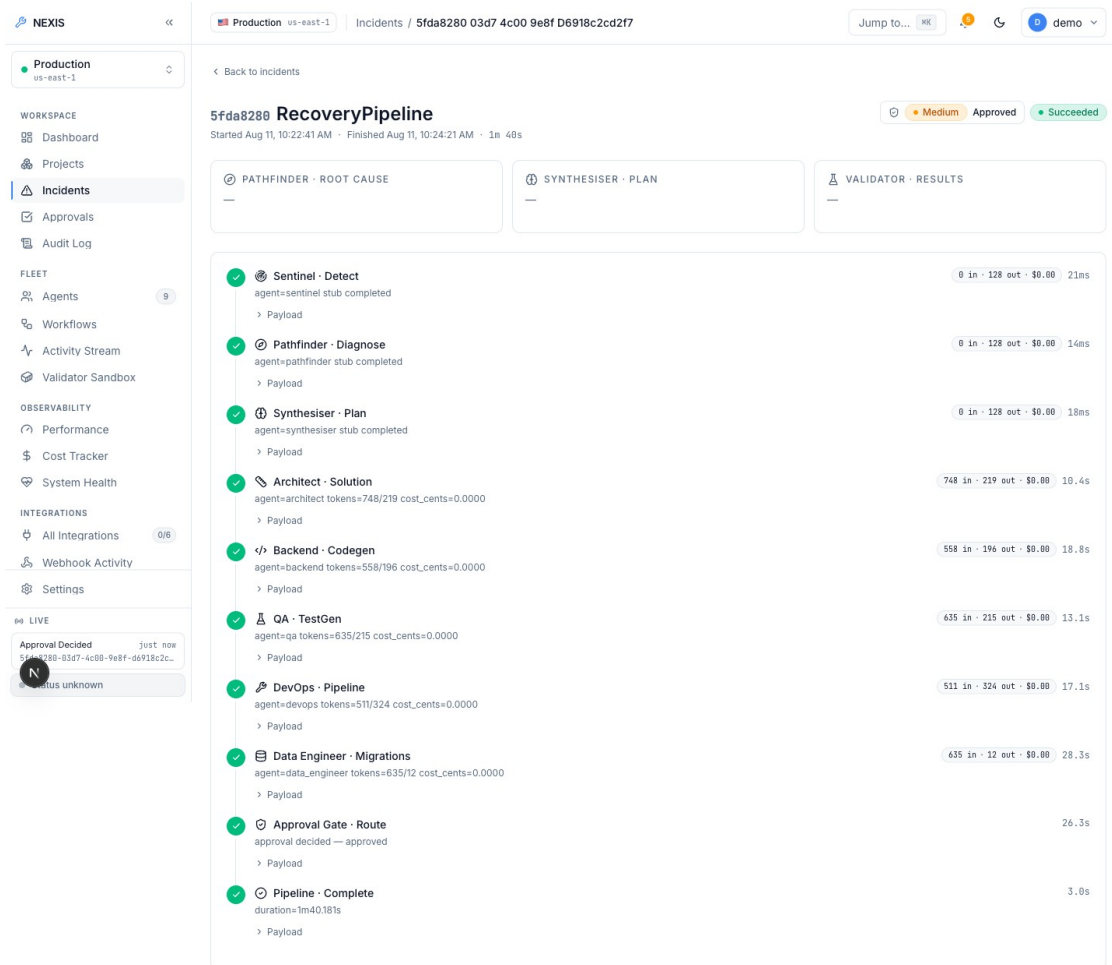


Figure 5.21: Fully Approved Run. (Medium severity, Approved, Succeeded; complete nine-step timeline in 1 minute 40 seconds with real per-agent token and cost figures).

One question remains: whether the patch itself is genuinely LLM-generated, and Figure 5.22 settles it. It shows the expanded raw JSON payload of the Backend agent’s step, including the actual unified diff produced by `qwen2.5-coder:14b`, with `provider="ollama"` recorded in the payload itself. The patch is model output parsed from a unified diff, not a template retrieved from a fixture.

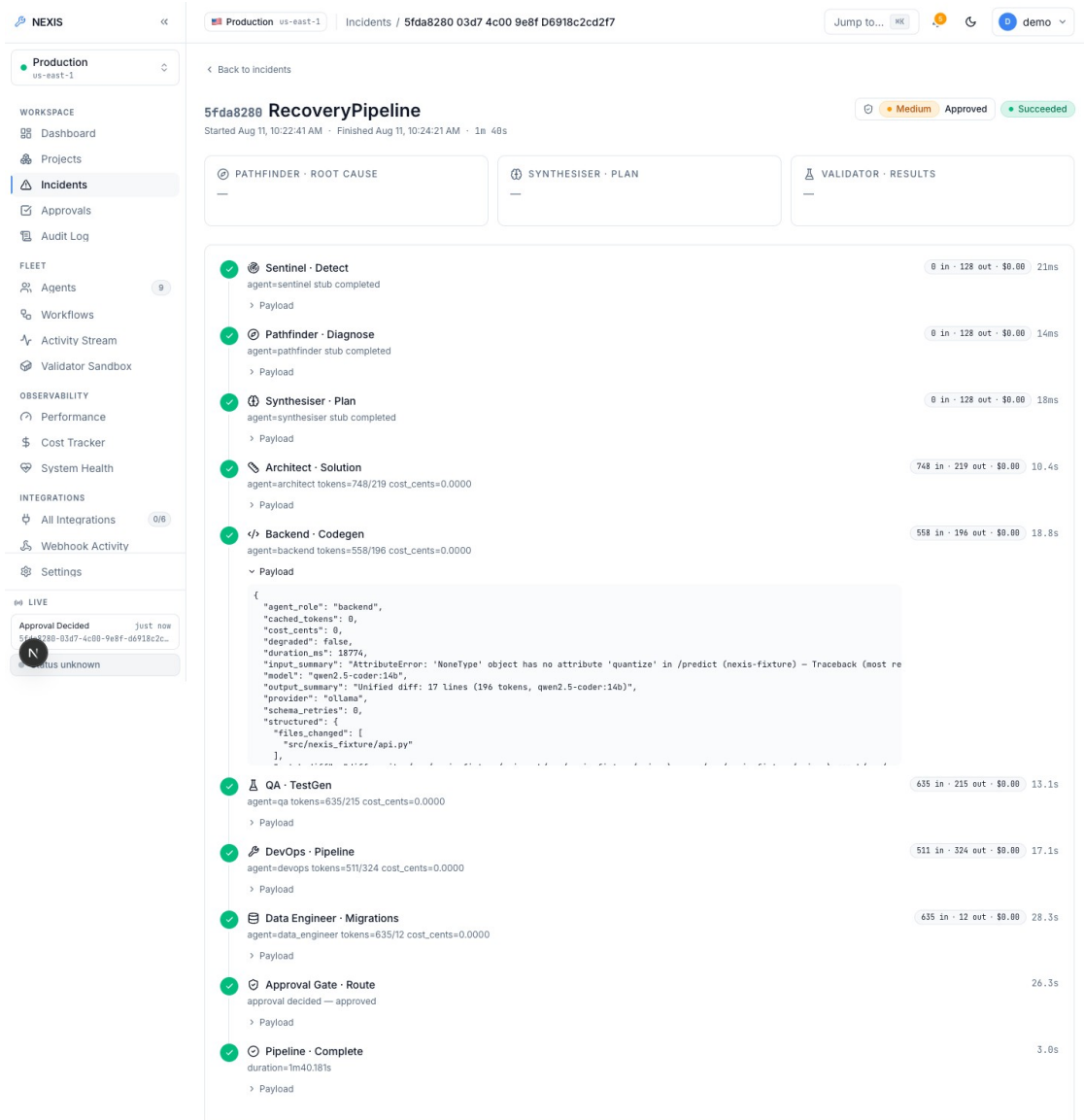


Figure 5.22: Backend Payload, Expanded. (raw JSON evidence of the qwen2.5-coder:14b-generated unified diff, provider="ollama").

5.2.5 Docker Preview-Deploy Engine (Ops Tab)

Each project detail page is organised into tabs. Overview (Figure 5.23) summarises the project, while Integrations (Figure 5.24) manages its provider connections, most importantly the GitHub repository binding. Recovery Policy (Figure 5.25) is where per-project self-healing behaviour gets configured, such as rollback on Service Level Objective (SLO) breach, and Activity (Figure 5.26) shows the project-scoped event history.

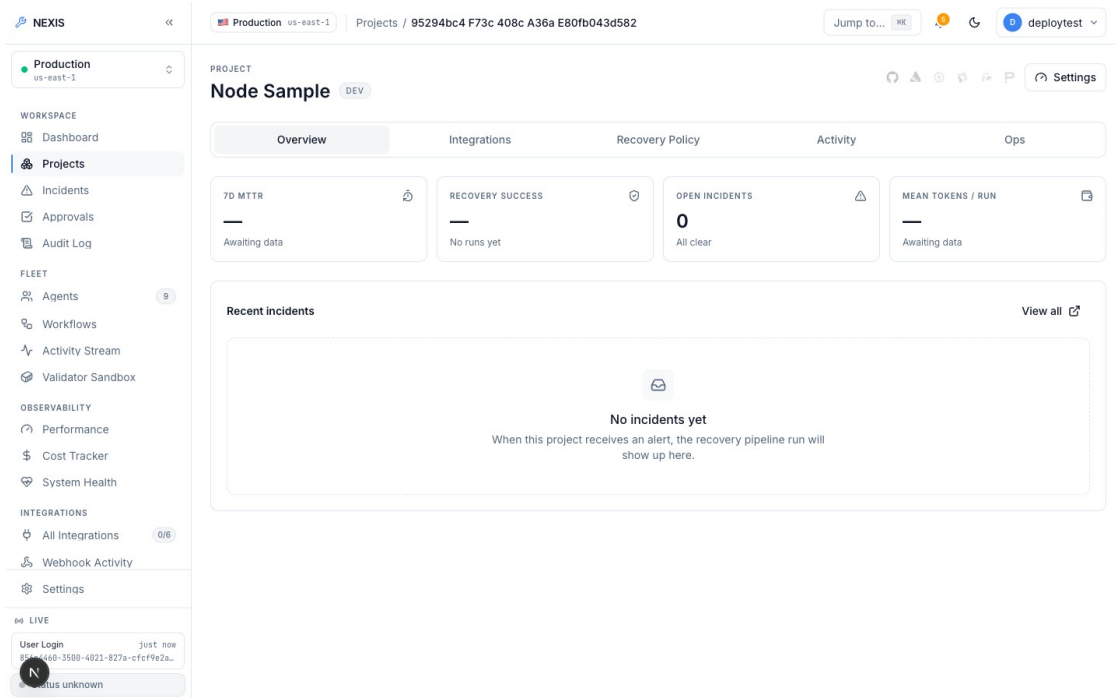


Figure 5.23: Project Overview Tab. (project summary and health).

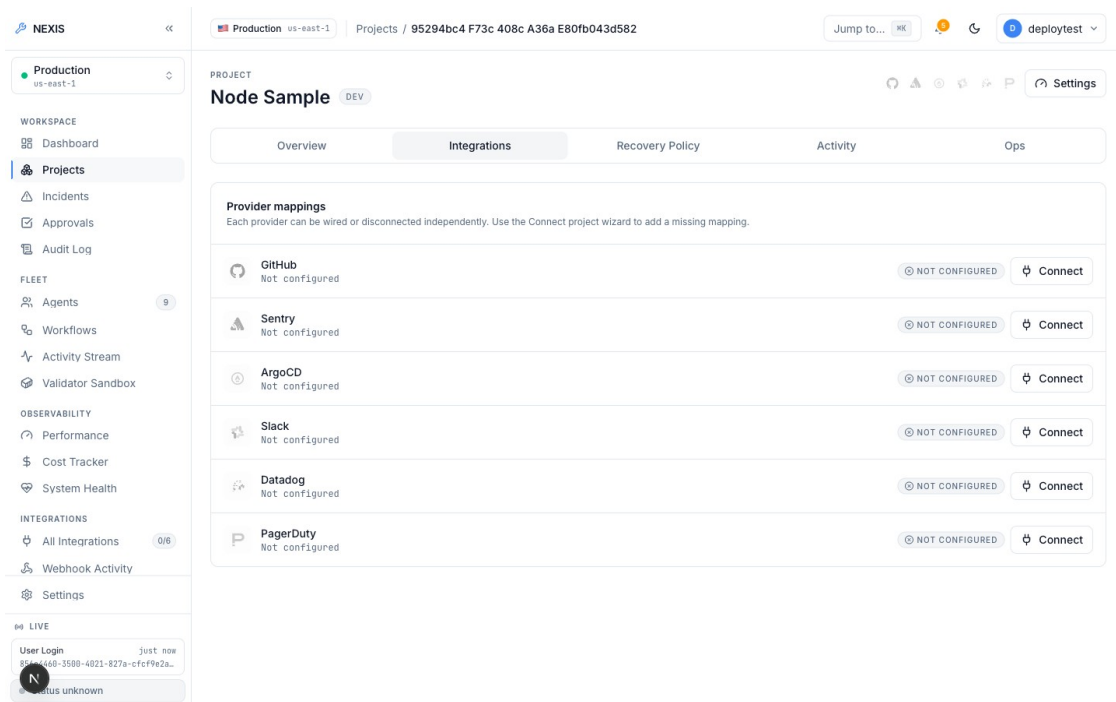


Figure 5.24: Project Integrations Tab. (provider connections including the GitHub repository binding).

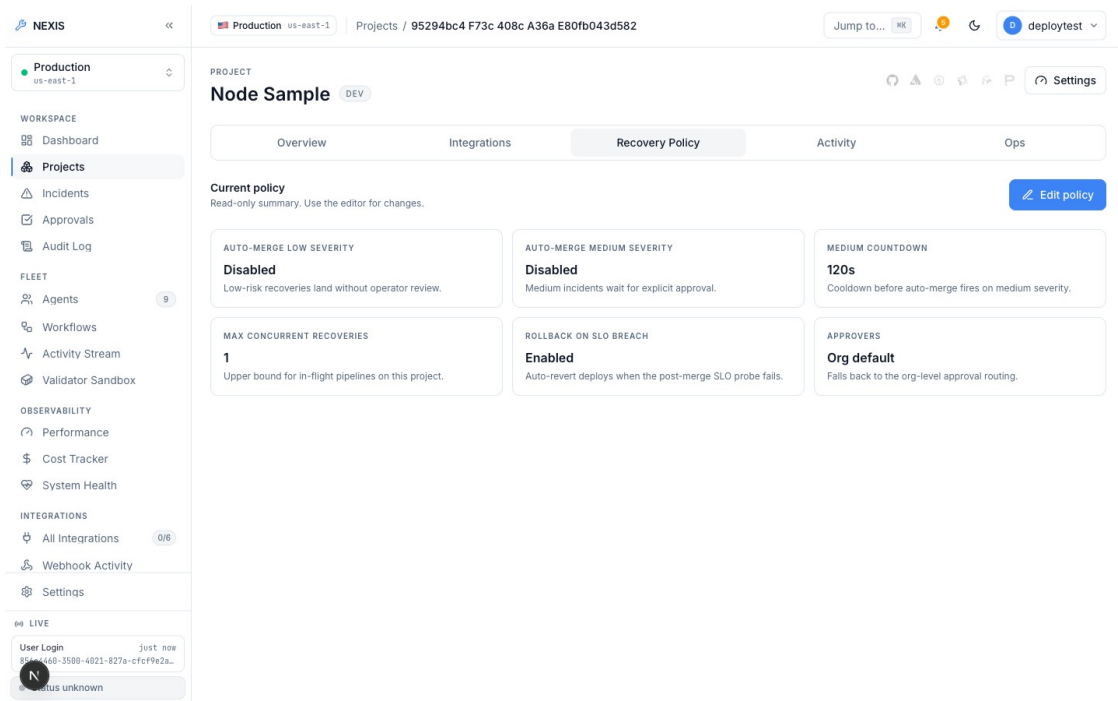


Figure 5.25: Project Recovery Policy Tab. (per-project self-healing configuration).

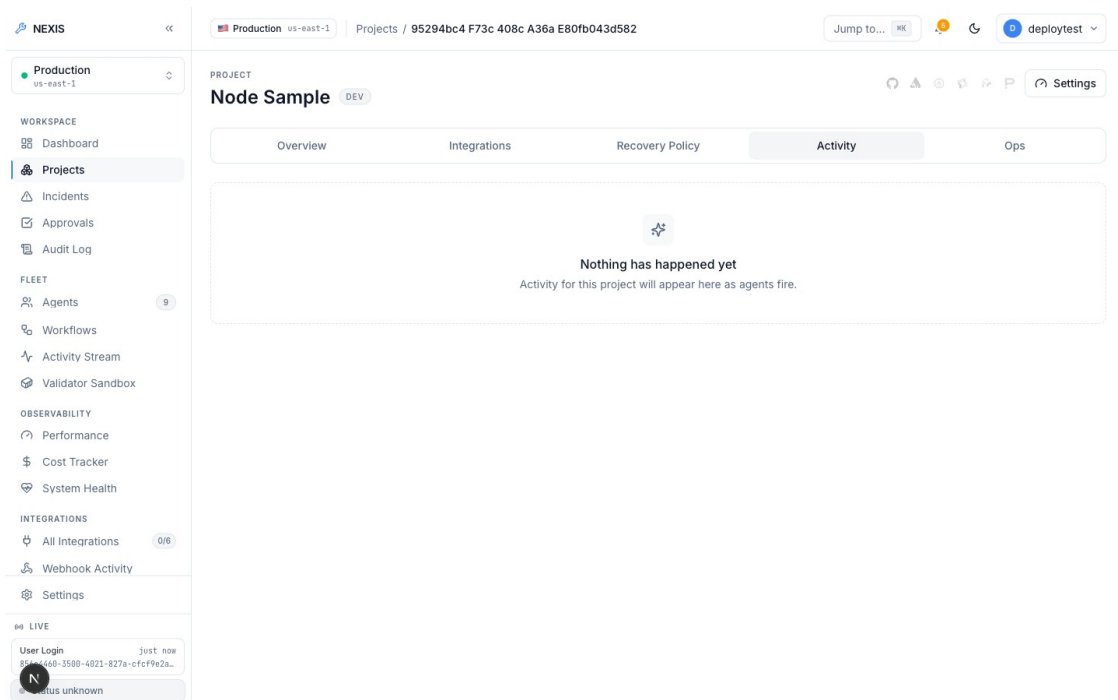


Figure 5.26: Project Activity Tab. (project-scoped event history).

The Ops tab is the frontend to the preview-deploy engine, and Figure 5.27 shows its empty state before any deployment has been made: the Deploy action, a live status area, and, once populated, the preview Uniform Resource Locator (URL) along with build/container logs and deployment history.

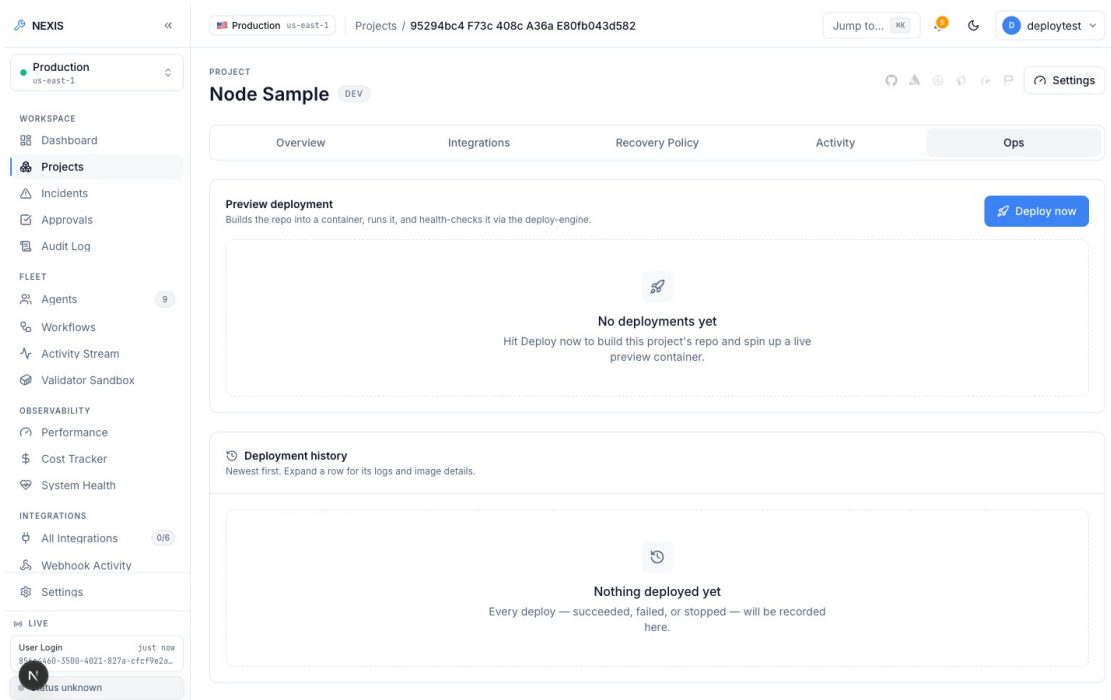


Figure 5.27: Project Ops Tab, Empty State. (the Deploy panel before any deployment).

Figure 5.28 shows what happened when Deploy was clicked on a project with no GitHub App connection in the evaluation environment: the real API returned the validation error “project has no github repository bound”, and the UI surfaces this verbatim. This capture does *not* show a successful deploy; what it shows is the real validation chain rejecting a deploy request that cannot be fulfilled, which is the correct behaviour here. The limitation is environmental, since no GitHub App credentials exist in the evaluation environment (as stated in Chapter 7’s limitations), rather than being a defect in the engine itself.

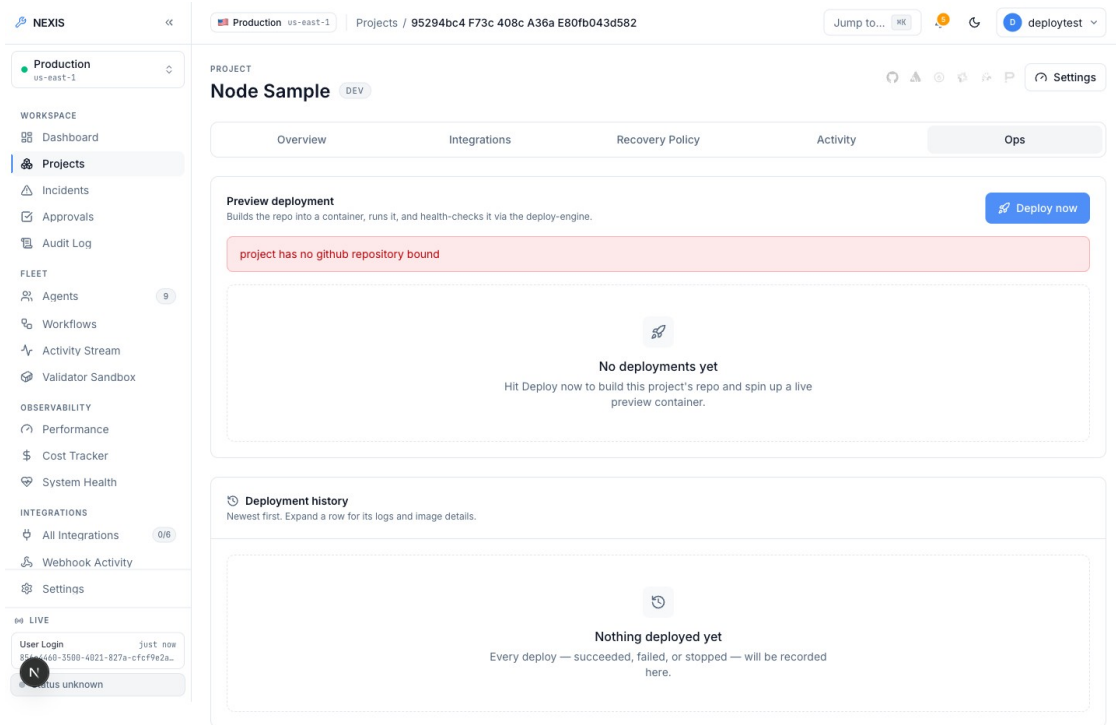


Figure 5.28: Project Ops Tab, Deploy Attempt. (a real validation error — “project has no github repository bound” — returned by the live API).

The engine itself, however, was proven live quite apart from that credential limitation. Furthermore, Listing 5.1 reproduces, verbatim, the verified end-to-end run recorded during this project. A real public repository (`heroku/node-js-getting-started`, which contains no Dockerfile) was cloned; its stack was detected as Node.js from marker files; a working Dockerfile was generated, the image built, and the container run under resource limits. A Hypertext Transfer Protocol (HTTP) request to the returned preview URL then served the application’s real rendered HyperText Markup Language (HTML) page, after which the container was stopped and removed cleanly.

```
POST /v1/deploy (repo: heroku/node-js-getting-started, no Dockerfile present)
-> 200 OK
  "status": "running", "dockerfile_source": "generated", "detected_stack":
"node",
  "url": "http://localhost:54671", "image_tag": "nexis-preview-test-
project:22d06076c357"
  build_log: "...npm ci... EXPOSE 3000... CMD [\"npm\", \"start\"]..."
  container_log: "Listening on 3000\nRendering 'pages/index' for route '/'"
```

```
GET http://localhost:54671/ -> real rendered HTML of the Node app (verified)
POST /v1/deploy/test-e2e-001/stop -> {"status":"stopped"}; container removed.
```

Listing 5.1: Verified live deploy-engine run.

Should a build or health check fail, the control plane inserts a real incident carrying source: "deploy_engine" into the same Sentinel-to-RecoveryPipeline path used by every other incident source. A broken build or an unrecognised stack thus becomes a self-healing target in exactly the same way as any production fault.

5.2.6 Projects, Agents, and Observability

Projects appear as cards on the Projects page (Figure 5.29), and new ones are created through the wizard shown in Figure 5.30, which derives sensible defaults from the project name. Each agent also has its own detail page (Figure 5.31 shows the Backend agent's), and the Recovery Pipeline page (Figure 5.32) presents the pipeline structure itself.

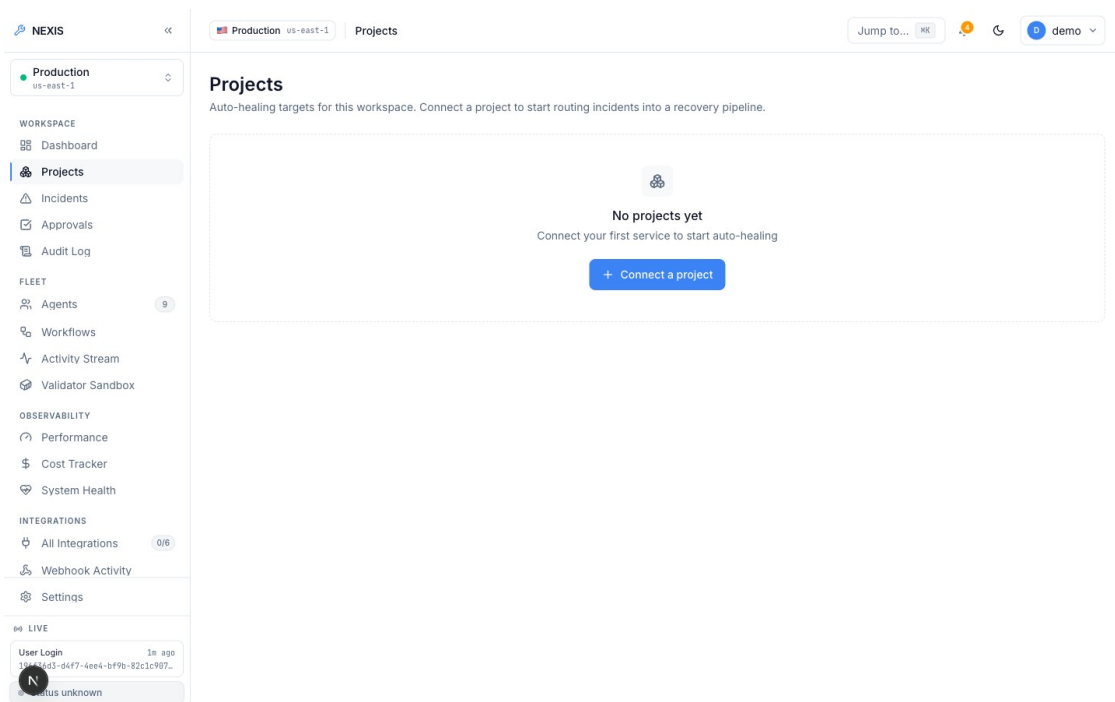


Figure 5.29: Projects Screen. (project cards with health indicators).

Productionus-east-1

Workspace

Dashboard

Projects

Incidents

Approvals

Audit Log

Fleet

Agents9

Workflows

Activity Stream

Validator Sandbox

Observability

Performance

Cost Tracker

System Health

Integrations

All Integrations0/6

Webhook Activity

Settings

LIVE

Approval Decided1m ago5584320-03d7-4c00-9e8f-d6918c2c...

Status unknown

Productionus-east-1

Projects / New

Jump to... KK

demo

Back to projects

1 Basics2 GitHub3 Sentry4 Monitoring5 Policy

STEP 1 OF 5

Project basics

Tell us about the service inside Production.

NAME

checkout-service

Slug preview: --

ENVIRONMENT

DevEngineering sandbox

StagingPre-production

ProdProduction traffic

DESCRIPTION

What this service does, who owns it, and how to reach the on-call.

OWNER USER ID

196f36d3-d4f7-4ee4-bf9b-82c1c907c3de

Defaults to you (demo@nexis.local).

Back

Next

Figure 5.30: New-Project Wizard. (guided project creation).

Productionus-east-1

Workspace

Dashboard

Projects

Incidents

Approvals

Audit Log

Fleet

Agents9

Workflows

Activity Stream

Validator Sandbox

Observability

Performance

Cost Tracker

System Health

Integrations

All Integrations0/6

Webhook Activity

Settings

LIVE

Approval Decided1m ago5584320-03d7-4c00-9e8f-d6918c2c...

Status unknown

Productionus-east-1

Agents / Backend

Jump to... KK

demo

Back to agents

BackendLTavailable

backend

Backend patch synthesis. Reads the codegraph + retrieval context, emits a unified diff.

TOTAL RUNS (7D)0

TOTAL TOKENS—In / out

TOTAL COST—

P95 DURATION—p50 —

LAST SEEN—

Run log

Every workflow run that invoked this agent — click a row to see the full event timeline and per-event payload.

2 total

STARTED	SCENARIO	SEVERITY	STATUS	DURATION	TOKENS IN / OUT	COST	SUMMARY
> 2m ago	nu11-deref	MEDIUM	Succeeded	18.8s	558 / 196	—	agent+backend tokens=558/196 cost_cents=0.0000
> 8m ago	nu11-deref	MEDIUM	Succeeded	13.8s	509 / 108	—	agent+backend tokens=509/108 cost_cents=0.0000

Showing 2 of 2

Figure 5.31: Agent Detail, Backend Agent. (per-agent configuration and run history).

81

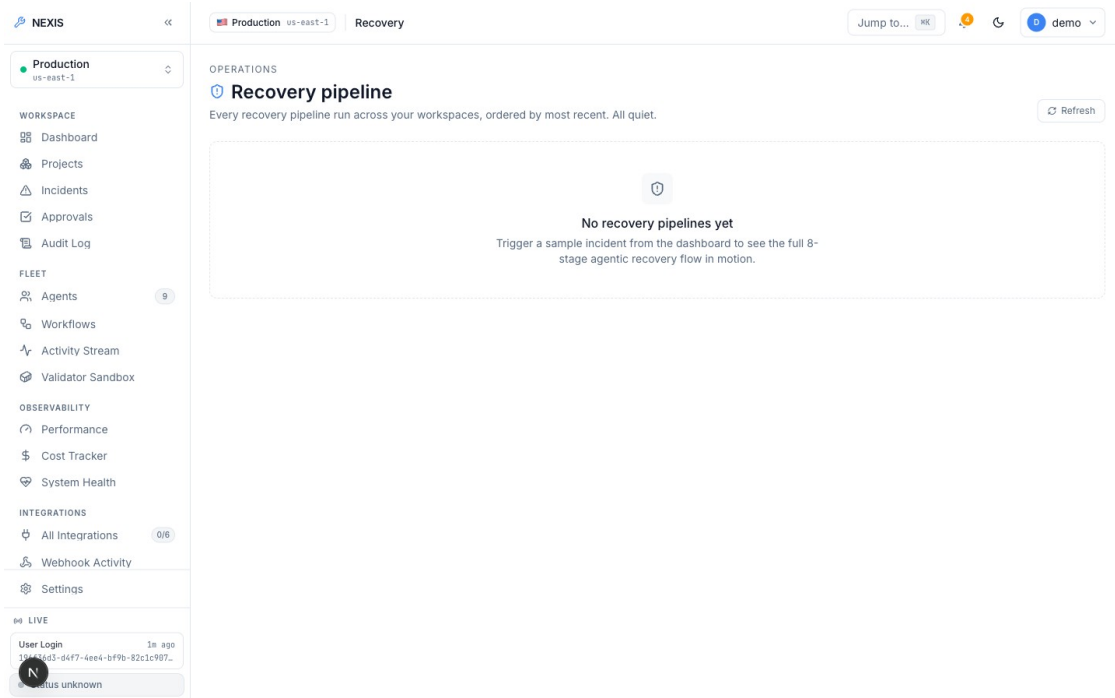


Figure 5.32: Recovery Pipeline Screen. (the nine-agent pipeline structure).

Observability is spread across four pages. Performance (Figure 5.33) charts pipeline latency and throughput, and the Cost Tracker (Figure 5.34) rolls up the per-organisation token ledger against a configurable daily budget. System Health (Figure 5.35) fans out parallel dependency probes against PostgreSQL, Redis, Neo4j, MinIO, and Temporal under a bounded timeout and reports each result, while the Evaluation (Eval) Harness (Figure 5.36) runs side-by-side OpenAI-versus-Ollama provider comparisons across fixture incidents, with persisted transcripts, cost accounting, and an administrator-only CSV export. The Knowledge Base page (Figure 5.37), finally, reports the status of the pgvector-backed retrieval store.

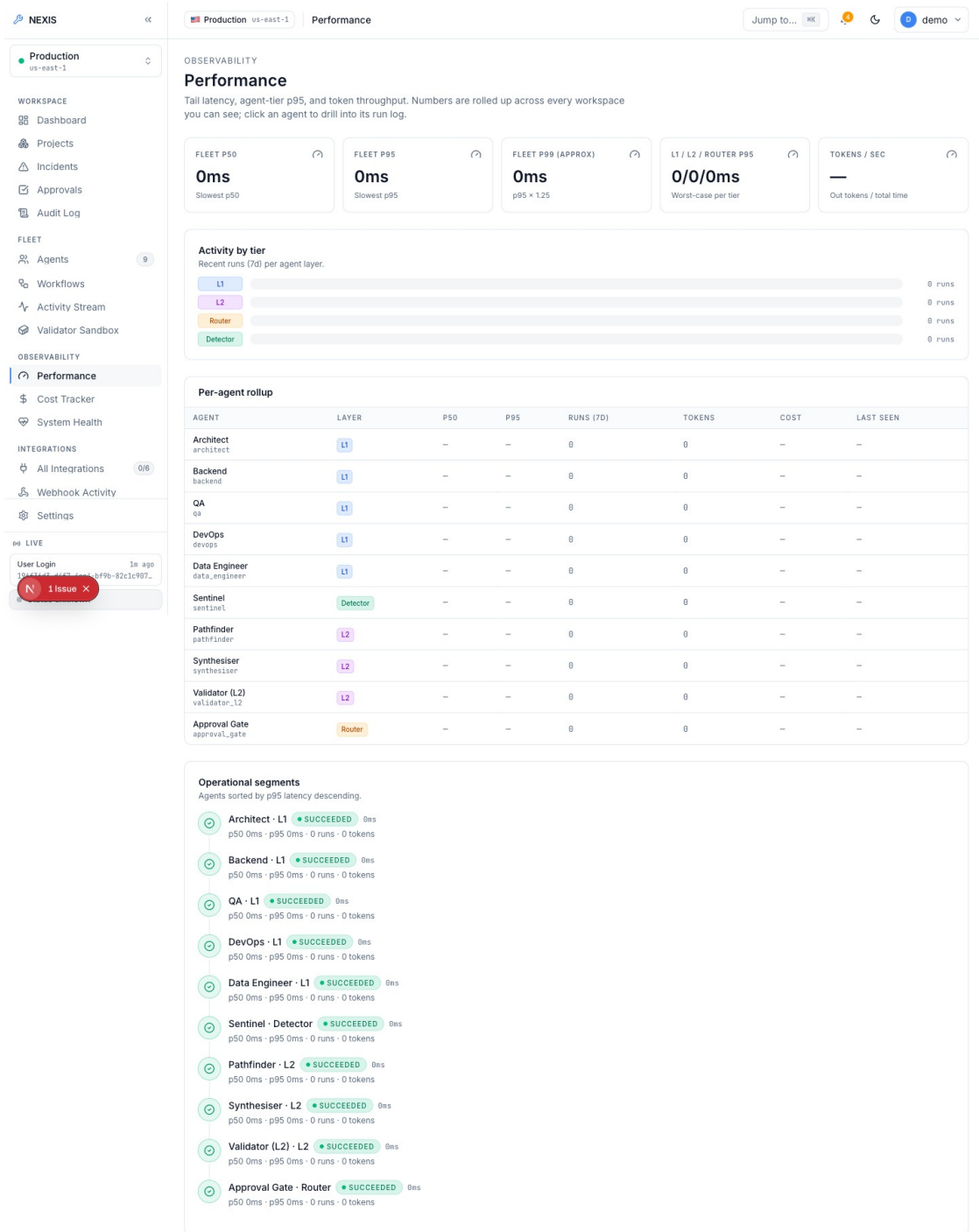


Figure 5.33: Performance Screen. (pipeline latency and throughput charts).

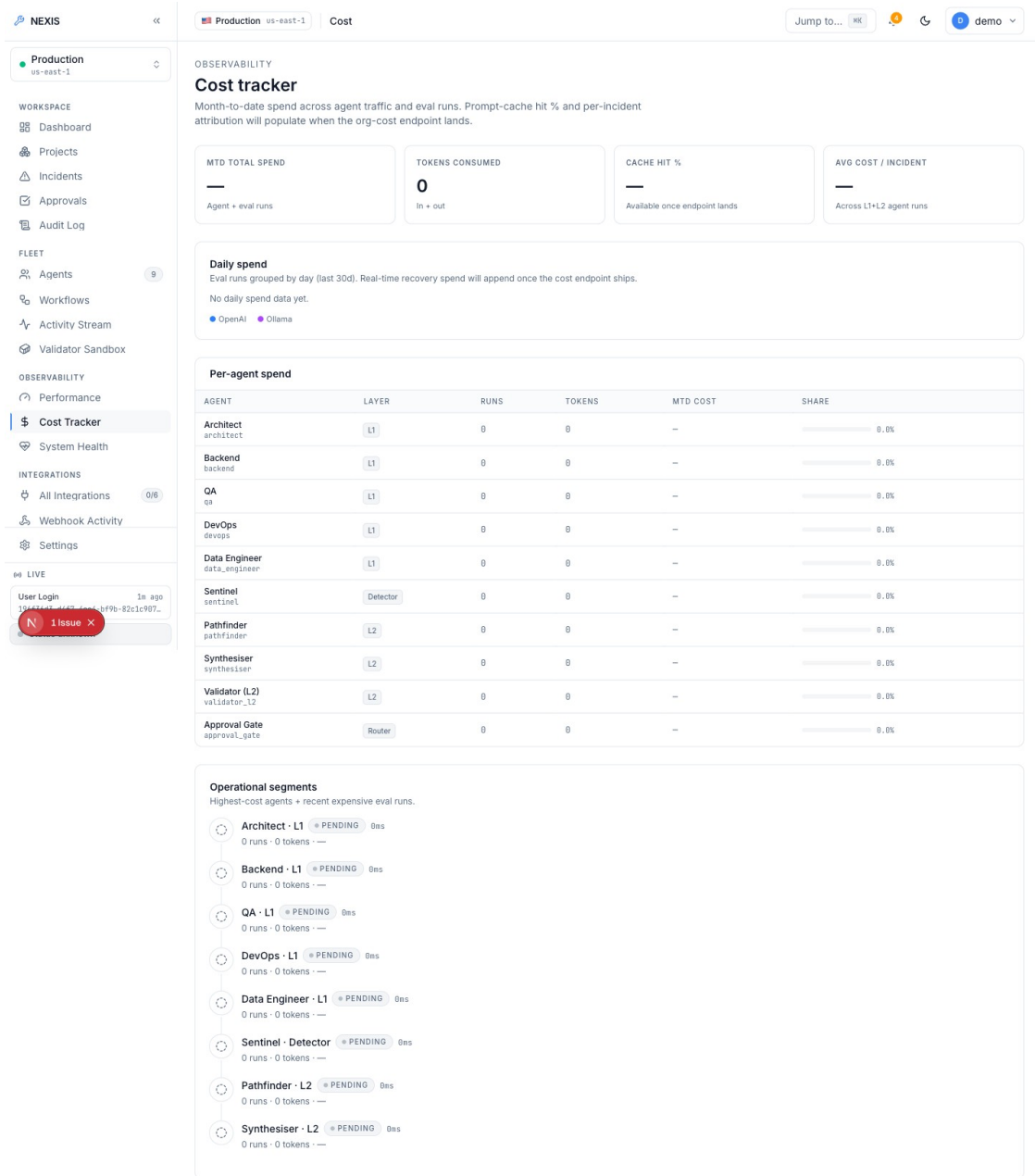


Figure 5.34: Cost Tracker. (per-organisation token ledger with daily budget).

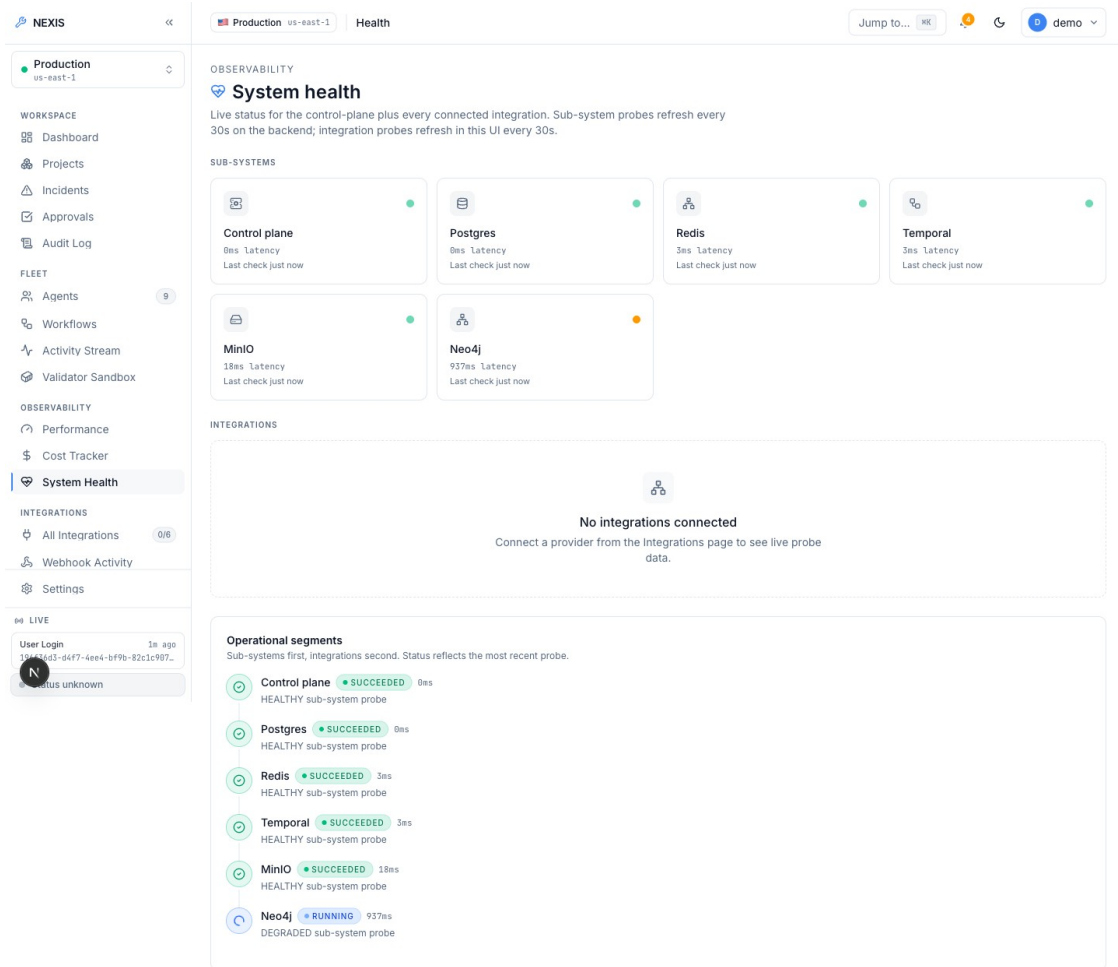


Figure 5.35: System Health. (parallel dependency probes: PostgreSQL, Redis, Neo4j, MinIO, Temporal).

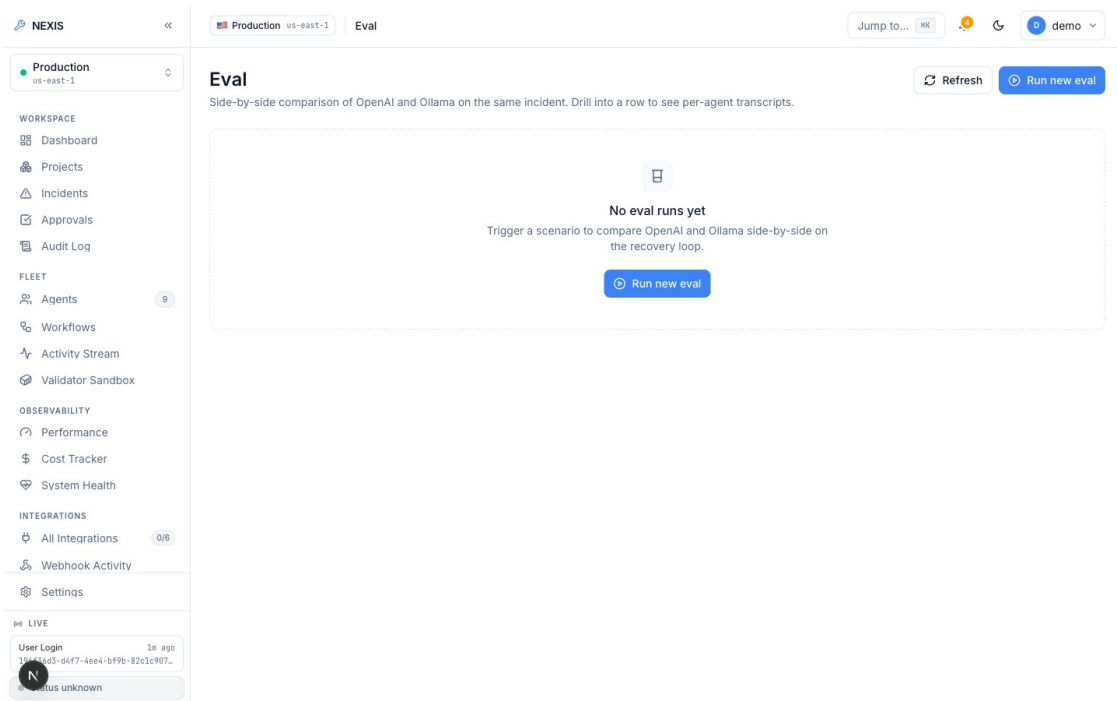


Figure 5.36: Eval Harness. (side-by-side OpenAI-versus-Ollama comparison over fixture incidents).

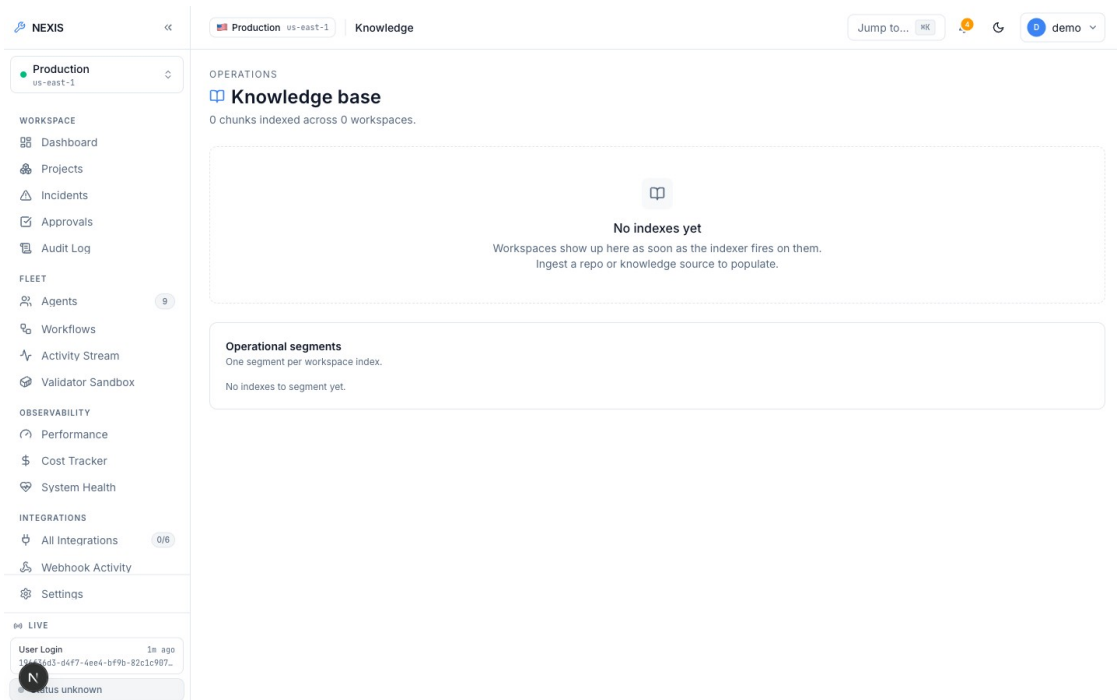


Figure 5.37: Knowledge Base. (pgvector-backed retrieval status).

5.2.7 Integrations

The platform's external providers are managed from the Integrations page (Figure 5.38): the GitHub App (covering installation, repository listing, and webhook-driven incident creation on merged pull requests), Sentry [14], Datadog [16], and PagerDuty [15] as webhook-driven incident sources, Slack [17] with OAuth install and interactive one-click approve/reject directly from a Slack message, and, beyond these, Argo CD and Stripe billing. Webhook Activity (Figure 5.39) lists inbound webhook deliveries for debugging purposes, while Connection Health (Figure 5.40) probes each configured integration and reports its status.

NEXIS

Production

us-east-1

Integrations

Jump to...

KK

demo

Production

us-east-1

Workspace

Dashboard

Projects

Incidents

Approvals

Audit Log

Fleet

Agents

Workflows

Activity Stream

Validator Sandbox

Observability

Performance

Cost Tracker

System Health

Integrations

All Integrations

Webhook Activity

Settings

LIVE

User Login

1m ago

Ym41c3-5df7-4ee4-bf9b-92c1c197e...

Status unknown

Integrations

Connect the third-party tools NEXIS uses to detect incidents, ship fixes, and roll back deployments.

Source Control

Where your code + PRs live.

GitHub

AVAILABLE

Disconnected

PR creation + repo metadata for code changes.

Configure

Check access

GitLab

ROADMAP

COMING SOON

Merge requests + repo metadata.

Unavailable

Bitbucket

ROADMAP

COMING SOON

Pull requests + branch operations.

Unavailable

1 live - 2 roadmap

Observability

Where incidents are detected.

Sentry

AVAILABLE

Disconnected

Error tracking + issue webhooks.

Configure

Check access

Datadog

AVAILABLE

Disconnected

Metrics, APM, and anomaly detection.

Configure

Check access

New Relic

ROADMAP

COMING SOON

Full-stack observability and APM.

Unavailable

2 live - 5 roadmap

Grafana Cloud

ROADMAP

COMING SOON

Hosted Grafana + Mimir/Loki/Tempo.

Unavailable

Prometheus

ROADMAP

COMING SOON

Metrics + Alertmanager webhook ingestion.

Unavailable

Honeycomb

ROADMAP

COMING SOON

Structured-event observability + BubbleUp.

Unavailable

Splunk

ROADMAP

COMING SOON

Log search + Observability Cloud alerts.

Unavailable

Incident Management

On-call routing + paging.

PagerDuty

AVAILABLE

Disconnected

On-call routing + paging.

Configure

Check access

Opsgenie

ROADMAP

COMING SOON

Atlassian on-call + escalation.

Unavailable

incident.io

ROADMAP

COMING SOON

Modern incident response platform.

Unavailable

1 live - 2 roadmap

Deploy & Infrastructure

Where rollbacks land.

ArgoCD

AVAILABLE

Disconnected

Deployment + rollback orchestration.

Configure

Check access

Flux CD

ROADMAP

COMING SOON

GitOps deploys + Helm controller.

Unavailable

Kubernetes

ROADMAP

COMING SOON

Direct cluster API: scale, restart, drain.

Unavailable

1 live - 5 roadmap

AWS CloudWatch

ROADMAP

COMING SOON

AWS metrics, alarms, and Lambda recovery.

Unavailable

GCP Monitoring

ROADMAP

COMING SOON

GCP alerts + Cloud Run/GKE recovery.

Unavailable

Azure Monitor

ROADMAP

COMING SOON

Azure alerts + AKS recovery.

Unavailable

Chat & Notifications

Where approvers get pinged.

Slack

AVAILABLE

Disconnected

Notify channels + DM approvers.

Configure

Check access

Microsoft Teams

ROADMAP

COMING SOON

Channel posts + approval actions.

Unavailable

Discord

ROADMAP

COMING SOON

Webhook notifications.

Unavailable

1 live - 2 roadmap

Data & Pipelines

Job and pipeline recovery.

Apache Spark

ROADMAP

COMING SOON

Job failure detection + retry orchestration.

Unavailable

Databricks

ROADMAP

COMING SOON

Cluster + workflow recovery.

Unavailable

Apache Airflow

ROADMAP

COMING SOON

DAG failure recovery + backfill triggers.

Unavailable

0 live - 6 roadmap

Figure 5.38: Integrations Screen. (provider catalogue: GitHub App, Sentry, Datadog, PagerDuty, Slack, Argo CD, Stripe).

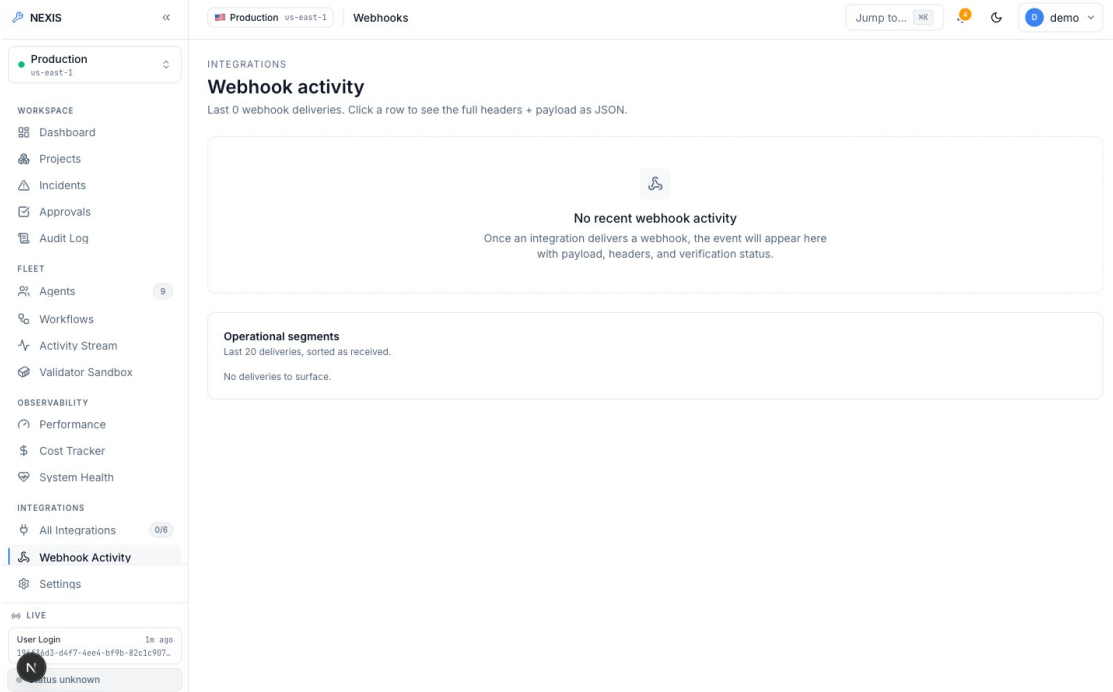


Figure 5.39: Webhook Activity. (inbound webhook delivery log).

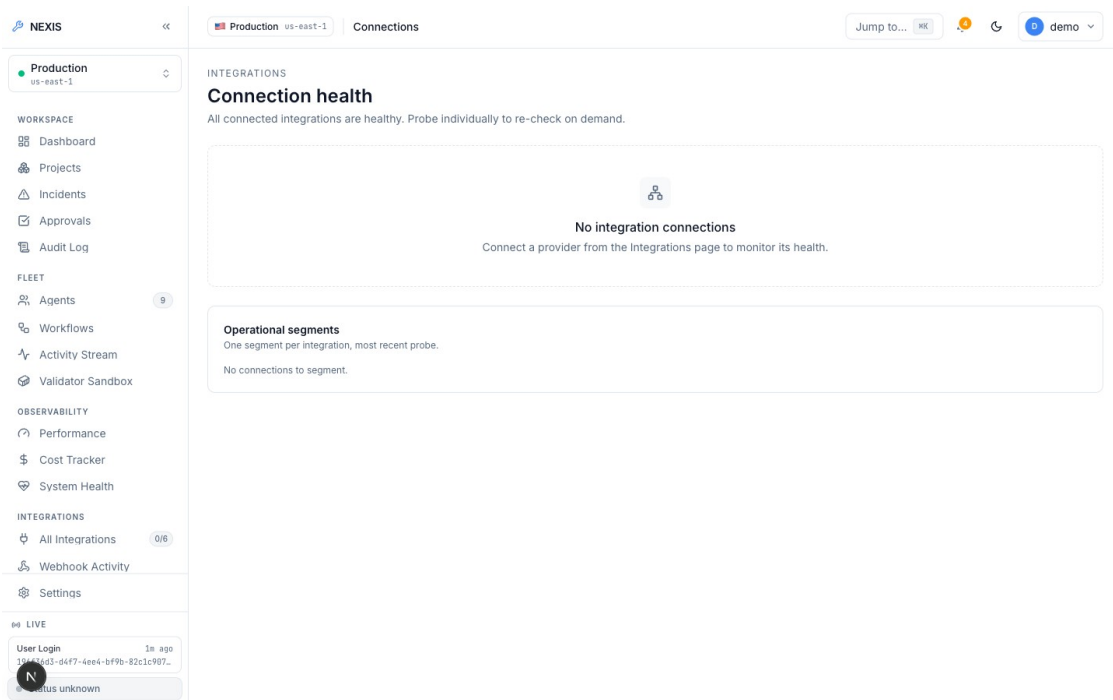


Figure 5.40: Connection Health. (per-integration health probes).

| 5.3 Database Schema

Nexis keeps its state in PostgreSQL 17, and the schema has grown incrementally across 33 sequential migrations, each one tracked in version control alongside the code that depends on it. Multi-tenancy is enforced inside the database itself rather than in application code. Moreover, nearly every table carries an organisation identifier, and PostgreSQL Row-Level Security policies [7] restrict every query to the calling organisation’s own rows. A request-scoped transaction binds `app.current_org_id` for each API request, which means that even a query that omits an explicit `WHERE` clause on the organisation still cannot read or write another tenant’s data.

Table 5.1 summarises the key tables. Organisation-scoping has two deliberate exceptions: `users` is a global identity table, since a person may belong to several organisations, with tenancy applied through `org_members`; and `sessions` is scoped to the owning user rather than to an organisation, because a login session belongs to a person and not to a tenant.

Table 5.1: Key tenant-scoped tables in the Nexis PostgreSQL schema.

Table	Purpose	RLS scope
<code>users</code>	Global account identity: credentials, Multi-Factor Authentication (MFA) enrolment, profile	Global (identity)
<code>org_members</code>	User–organisation membership and role (<code>owner</code> / <code>admin</code> / <code>member</code>)	Organisation-scoped
<code>projects</code>	Registered projects, GitHub repository binding, recovery policy	Organisation-scoped
<code>incidents_raw</code>	Every ingested fault event, from webhooks, fixtures, and	Organisation-scoped

Table	Purpose	RLS scope
	the deploy engine	
<code>workflow_runs</code>	One row per Temporal RecoveryPipeline execution, with per-step activity events	Organisation-scoped
<code>approval_decisions</code>	Terminal approval-gate outcomes: approve, reject, modify, timeout	Organisation-scoped
<code>deployments</code>	Preview-deploy engine builds/containers and GitOps deployments	Organisation-scoped
<code>lineage_events</code>	OpenLineage-compatible RunEvents [8] for migration propose/apply	Organisation-scoped
<code>audit_log</code>	Append-only, hash-chained record of privileged actions	Organisation-scoped
<code>feedback_examples</code>	RLHF training rows persisted from every terminal approval decision	Organisation-scoped
<code>role_recommendations</code>	Heuristic role-change recommendations with rationale and cool-down state	Organisation-scoped
<code>digest_reports</code>	Daily administrator digest aggregates (incidents, repairs, approvals, MTTR)	Organisation-scoped
<code>sessions</code>	Active login sessions: device, IP, user agent, last-seen	User-scoped
<code>api_keys</code>	Scoped bearer keys (<code>nx_live_...</code>) for	Organisation-scoped

Table	Purpose	RLS scope
	programmatic access	

Beyond relational state, three specialised stores complete the data layer. Neo4j holds the code dependency graph traversed by the Pathfinder agent, MinIO holds encrypted patch blobs referenced from `workflow_runs`, and pgvector columns inside PostgreSQL back the knowledge-base retrieval feature. Temporal, in addition, maintains its own durable persistence for workflow event histories; this complements the per-step records in `workflow_runs`, and in a recovery scenario, can reconstruct them.

Chapter 6: Testing and Results

The verification evidence for NEXIS is laid out here in the order it actually occurred during the final engineering cycle, not as a matrix of numbered test cases assembled after the fact. Three kinds of real evidence follow. Static verification comes first: the build, vet, lint, type-check, and automated test commands run against every service, along with what each one returned. Next are the live end-to-end integration runs carried out against the real, running system, namely two complete nine-agent recovery pipeline executions and one complete deploy-engine execution, all driven by a real local Large Language Model (LLM) rather than by stubs or mocks. The third kind is the set of defects that the testing process itself turned up, together with their root causes and the fixes applied. What was tested, and, just as importantly, what was not, is set out honestly at the close of the chapter.

| 6.1 Static Verification

Each component of the platform was verified through its own native toolchain, not a single uniform check. Across the four independent Go (Golang) modules, the full sequence of `go build ./...`, `go vet ./...` (the Go (Golang) toolchain’s built-in static analyser), and `go test ./...` was run, with a pass requiring zero build errors, zero vet findings, and zero failing tests. For the two Python components, `pytest` was the tool of choice: the causal-inference service’s suite runs 38 tests, and the validator’s Hypothesis-based sidecar [9] carries its own suite, which likewise passes in full. The Next.js frontend, for its part, was verified through the project’s three gate commands, namely TypeScript type checking, ESLint, and a full production build. Table 6.1 sets out the commands used against each component, along with the outcomes obtained.

Table 6.1: Static verification commands and results across all NEXIS components.

Component	Command	Result
services/control-plane — Go (Golang)	<code>go build ./... && go vet ./... && go test ./...</code>	Clean — build, vet, and all tests pass

Component	Command	Result
services/validator — Go (Golang)	go build ./... && go vet ./... && go test ./...	Clean — build, vet, and all tests pass
services/gitops — Go (Golang)	go build ./... && go vet ./... && go test ./...	Clean — build, vet, and all tests pass
services/deploy-engine — Go (Golang)	go build ./... && go vet ./... && go test ./...	Clean — build, vet, and all tests pass
services/causal-inference — Python	pytest	38 tests, all passing
Validator Hypothesis sidecar — Python	pytest	All tests passing
apps/web — Next.js	pnpm typecheck && pnpm lint && pnpm build	Clean — type check, lint, and production build all pass

The result column in this table is not aspirational. Every command listed was actually run against the final codebase and returned clean, and this includes the Go (Golang) suites, which now cover the JSON Web Token (JWT) clock-injection tests taken up in Section 6.4, tests that were silently failing earlier in the cycle and which genuinely pass at this point. Static verification, moreover, reaches every deployable unit of the platform; consequently, no service ships without first clearing its own build, its own static analysis, and its own test suite.

6.2 Live End-to-End Recovery Runs

Static verification, it may be noted, only establishes that each service is sound on its own; it says nothing about whether the nine agents, the Temporal workflow [2], the validator sandbox, the approval gate, and the live console actually work together as a system. Two complete recovery pipeline runs were carried out against the real running system to test exactly that: real PostgreSQL with Row-Level Security (RLS) active, real Temporal workflows, a real Neo4j dependency graph, and a real local LLM, Ollama [20] serving qwen2.5-coder:14b and gpt -

oss:20b, generating the actual plans, diffs, and tests. Nothing in either run was mocked or replayed. Chapter 5 reproduces the console evidence captured across both runs, from the pipeline mid-flight with live per-agent token counts and the fully green timeline awaiting approval, through to the honest timed-out terminal state of the first run and the complete Approved-to-Succeeded timeline of the second.

6.2.1 Run 1: Approval-Gate Timeout — the Safety Mechanism Under Real Timing Pressure

A fault fixture was injected in the first run, Sentinel picked it up, and the agent pipeline then ran through to completion: the Architect produced a structured plan, the Backend agent generated a unified-diff patch, the Quality Assurance (QA) agent wrote Hypothesis property tests, and the Validator ran the patch against those tests inside its locked-down Docker (Podman-compatible runtime) sandbox. Every step across Layer 1 and Layer 2 reached a green state on the live timeline. It was at the human approval gate that the candidate patch then came to rest, pending a decision; since none was made within the 120-second decision window, the workflow resolved the incident to a rejected terminal state once that window had elapsed.

In this regard, the outcome deserves to be read correctly: it counts as a *positive* test result, not a failure. The platform’s central safety invariant holds that no patch is ever deployed without passing validation and clearing the approval gate, and further, that the system fails closed on ambiguity, treating an absent decision as a refusal and never as consent. Real timing pressure was exactly what the first run exercised against this invariant, with a real generated patch sitting on the other side of the gate, and the system responded precisely as it had been designed to. Rather than ship an unreviewed change, it discarded the work, and the timeout-rejected outcome was recorded honestly and durably in the incident timeline. A demonstration environment could easily have hidden this particular run and shown only the success story; it is reported here deliberately, since a rejected run is, if anything, the stronger evidence that the approval gate is real.

6.2.2 Run 2: Approved and Succeeded in 1 Minute 40 Seconds

The same detection path was followed in the second run, except that the pending approval this time was acted on within the window. Classified at Medium severity, the incident produced a generated patch that passed sandbox validation; approval then came through the console's decision dialog, and the pipeline moved on through its deploy stage to a *Succeeded* terminal state. From fault detection through every agent step to that final succeeded state, the complete timeline spanned 1 minute 40 seconds.

This run is strong evidence rather than a staged demonstration. The accounting, to begin with, is real: the timeline records genuine per-agent token consumption and cost figures for every LLM-backed step, produced by the live Ollama provider as the run actually happened. The artefacts, moreover, are inspectable, since the raw JavaScript Object Notation (JSON) payload of the Backend agent's activity event shows the actual unified diff generated by `qwen2.5-coder:14b`, with `provider="ollama"` recorded in the event metadata. The patch that was validated, approved, and deployed is, in other words, the patch the model wrote and not a canned fixture.

Between them, the two runs cover both branches of the approval gate, refusal by timeout on one side and explicit approval on the other, against one and the same real pipeline. These are exactly the two behaviours a closed-loop recovery system must get right.

| 6.3 Live Deploy-Engine Verification

The deploy-engine was verified in its own live end-to-end run against a real public GitHub repository, `heroku/node-js-getting-started`, chosen deliberately for containing no `Dockerfile`. This is the engine's hardest path to exercise: it has to clone the repository, work out the stack from marker files, generate a working `Dockerfile` from nothing, build the image, run the container under resource limits, and only then prove liveness over Hypertext Transfer Protocol (HTTP). The actual request/response transcript of this run is reproduced in Listing 6.1.

Listing 6.1: Live deploy-engine end-to-end run: clone, generate Dockerfile, build, run, HTTP-verify, stop.

```
POST /v1/deploy (repo: heroku/node-js-getting-started, no Dockerfile present)
-> 200 OK
  "status": "running", "dockerfile_source": "generated", "detected_stack":
  "node",
  "url": "http://localhost:54671", "image_tag": "nexus-preview-test-
project:22d06076c357"
  build_log: "...npm ci... EXPOSE 3000... CMD [\"npm\", \"start\"]..."
  container_log: "Listening on 3000\nRendering 'pages/index' for route '/'"

GET http://localhost:54671/ -> real rendered HTML of the Node app (verified)

POST /v1/deploy/test-e2e-001/stop -> {"status":"stopped"}; container removed.
```

Each stage in the transcript genuinely took place. A Node.js stack was detected by the engine (`detected_stack: "node"`), and a Dockerfile was synthesised (`dockerfile_source: "generated"`) whose build log shows the generated `npm ci`, `EXPOSE 3000`, and `CMD` directives actually executing. The container log, in turn, shows the application booting and serving its index route, while an independent HTTP GET against the returned preview Uniform Resource Locator (URL) came back with the application's real rendered HyperText Markup Language (HTML); finally, the stop endpoint terminated and removed the container cleanly. Since the repository is public and the entire request sequence amounts to only three HTTP calls, this counts as a reproducible integration test and not merely a one-off observation.

For reproducibility, one environmental substitution is worth flagging. Docker was not available in the evaluation environment, so Podman [11] stood in as the container runtime, working through the same Docker-compatible Application Programming Interface (API) [10] that both the deploy-engine and the validator target. Since the engine's code path does not differ between the two runtimes, the run described above is proof enough that the Podman path works end-to-end. Unauthenticated public-repository access was what the clone in this run relied on; the authenticated GitHub App installation-token path is present in the code but was not exercised in this evaluation environment, a point also made in the limitations.

| 6.4 Defects Found and Fixed During This Engineering Cycle

More than merely confirmatory, the verification process turned out to surface two genuine defects, and both were root-caused and fixed within the same cycle. These are summarised in Table 6.2.

Table 6.2: Defects discovered by the verification process, with root causes, fixes, and verification.

Defect	Root Cause	Fix	Verification
13 JWT-related tests silently failing	Token <code>exp</code> claims were validated against the wall clock instead of an injected test clock, so time-sensitive assertions depended on when the suite happened to run	One-line change: inject the test clock via <code>jwt.WithTimeFunc</code>	All 13 tests now pass deterministically; production behaviour verified unchanged, because the production code's default clock is already identical to the JWT library's own default
Database migration failed to apply	<code>current_role</code> is a reserved PostgreSQL keyword and was used unquoted as a column name in a migration	Column renamed to <code>existing_role</code>	Migration applies cleanly; the Go (Golang) test suites over the affected schema remain green

Why the verification loop earns its cost is illustrated well by both defects. The JWT defect was *silent* in the sense that the affected tests kept failing without blocking anything visible, precisely the failure mode that erodes a test suite's value over time; it came to light, hence, only because the full `go test ./...` output was actually read rather than skimmed over. The migration defect was different in character, latent in a reserved-word collision that shows itself only at migration-apply time, the kind of bug a static review of the Structured Query Language (SQL) text can easily walk past.

One further pre-existing bug turned up during this cycle, for completeness, though it was deliberately left unfixed on the grounds that it is unrelated to the recovery pipeline and falls outside this project's scope: the workspace-onboarding progress poller requests

`/v1/workspaces/{id}/events` and gets back a 404, even though the workspace itself still provisions successfully on the server side regardless. Rather than being quietly left out, it is recorded here and again in the limitations.

| 6.5 Testing Summary

Put plainly, this is the evidence base for NEXIS. Static verification comes back 100% clean across every service in the platform: the four Go (Golang) modules pass their build, vet, and full test suites with zero errors between them; both Python services pass `pytest`, the causal-inference service's 38 tests included; and the frontend clears type checking, linting, and a full production build. Sitting on top of that foundation are three live integration runs, each observed directly against the real running system: two complete nine-agent recovery pipeline executions (one timeout-rejected in a way that demonstrates the fail-closed approval gate, the other Approved and Succeeded in 1 minute 40 seconds with real per-agent token and cost accounting), together with one complete deploy-engine run against a real public repository (clone, generated Dockerfile, build, run, HTTP-verified, stopped). Not one of these runs was simulated, and the LLM driving the recovery runs was a real local model rather than a stub.

What was *not* tested matters just as much. No formal load or performance testing took place: the platform has not been run under concurrent multi-tenant traffic, no throughput or latency figures under load were ever measured, and none, consequently, are claimed in this report. That single measured end-to-end duration of 1 minute 40 seconds is the observed time of one real run, nothing more, and should not be read as a statistical performance claim. Load testing under realistic multi-tenant concurrency, together with a larger sample of recovery runs from which meaningful Mean Time To Recovery (MTTR) distributions could actually be drawn, is named explicitly as future work in Chapter 7.

Chapter 7: Conclusion and Future Work

| 7.1 Overview

Existing Artificial Intelligence (AI) developer tools do not close the loop between detecting a software failure and deploying a verified fix for it; this project set out to build one that does, keeping a human in the process as a single decision point rather than as the pipeline itself. The delivered platform, Nexis, realises that goal as a working, verified system, not merely as a design proposal.

The central deliverable is the nine-agent closed-loop self-healing pipeline, durably orchestrated by Temporal [2]. A five-agent Execution Team, comprising Architect, Backend, Quality Assurance (QA), DevOps (Development and Operations), and Data Engineer, carries out the engineering work. Running alongside it, a four-agent Self-Healing Loop, made up of Sentinel, Pathfinder, Synthesiser, and Validator, detects faults, localises root causes over the Neo4j dependency graph [3], sandbox-validates every candidate patch, and routes the outcome through a severity-classified Approval Gate that offers Approve, Reject, and Modify decisions. Crucially, this pipeline was not merely implemented; it was exercised live. Chapter 6 reports two complete end-to-end recovery runs against a real local language model [20]. In one, the 120-second approval window genuinely elapsed and the run was timeout-rejected exactly as the safety design intends; in the other, a detected fault moved through planning, patch generation, property-based test validation, human approval, and deployment to a Succeeded terminal state in approximately one minute and forty seconds.

Complementing the recovery pipeline is the from-scratch Docker (Podman-compatible runtime) preview-deploy engine [10], the project's second major deliverable. Given a connected repository, it clones the code, detects the technology stack, and generates a working Dockerfile when none exists, then builds and runs the container under resource limits before returning a live preview Uniform Resource Locator (URL). Build and health-check failures, too, feed back into the same self-healing pipeline as any other incident source. This, too, was proven live: a real

public repository with no Dockerfile in it was cloned, containerised, built, served, verified over the Hypertext Transfer Protocol (HTTP), and cleanly stopped, all within a single recorded run.

Around these two engines sits a complete multi-tenant platform. Authentication runs on JavaScript Object Notation (JSON) Web Token (JWT) and Single Sign-On (SSO), backed by Multi-Factor Authentication (MFA) and session management [12], while Role-Based Access Control (RBAC) is enforced down to the database row via PostgreSQL Row-Level Security (RLS) [7]. Moreover, a heuristic role-recommendation engine turns audit-log evidence into explainable privilege suggestions, Stripe handles billing [13], and webhook-driven integrations tie in GitHub, Sentry, Datadog, PagerDuty, and Slack [14, 16, 15, 17]. On top of all this sits an observability surface, comprising dashboards, cost tracking, system health probes, and a hash-chained audit log, built on OpenTelemetry [5]. As documented in Chapter 6, all Go (Golang) modules, the Python causal-inference service, and the Next.js console pass their full build, vet, lint, and test suites.

| 7.2 Limitations of the Project

Academic integrity demands that the line between what was demonstrated and what was not be drawn plainly. The limitations listed below are real and known, each one a deliberate boundary rather than an oversight, and none of them is hidden behind softer wording.

- **No production credentials were exercised.** No real GitHub App or OpenAI production credentials existed in the evaluation environment used for this report. All the live evidence came instead from local Ollama models [20] and an unauthenticated clone of a public repository, so a fully credentialed production path, one ending in a real merged GitHub pull request, was not exercised end-to-end.
- **The platform is not yet live on its domain.** Although the `nexis.sh` domain has been registered, the platform itself is not yet deployed to it, so every piece of evidence in this report comes from local, native execution. Since Docker was unavailable in the build environment, Podman [11] was substituted as a runtime compatible with the Docker Application Programming Interface (API); it was shown to behave identically.

- **Fine-tuning is a manual step by design.** Real and already tested, the Reinforcement Learning from Human Feedback (RLHF) data-export pipeline persists every terminal approval decision, including engineer-edited diffs, in a form exportable as JSON Lines (JSONL). Actually submitting a fine-tuning job against that export, however, is a deliberate manual step, since it involves real money and a real external API call rather than an automated one.
- **One of twenty-seven demo scenarios is fully wired.** Twenty-seven curated fault scenarios are listed in the Live-Demo console, and of these, only one is wired end-to-end to a real fixture and classification path at present. As such, the remaining twenty-six are User Interface (UI)-complete but still backend-pending.
- **The causal ranking is a proxy, not a fitted causal model.** Root-cause candidates are ranked by the causal-inference sidecar through an honest statistical and graph-structural scoring method, with a transparent per-component confidence breakdown attached to each ranking. It is explicitly not a fitted DoWhy structural causal model [4]; no interventional production data exists against which such a model could be fitted, and claiming otherwise would be an overstatement.
- **A known unrelated bug was left unfixed.** During evaluation, a pre-existing bug turned up in the workspace-onboarding progress poller, a 404 returned from `/v1/workspaces/{id}/events`, and it was intentionally left unfixed as out of scope. The workspace itself, regardless, still provisions successfully on the server side.
- **No formal load testing was performed.** As noted in Chapter 6, no formal concurrent-load performance benchmark was run, and the latencies reported come from single-run live executions rather than a controlled load harness.

| 7.3 Future Work

Each item that follows is grounded in the existing architecture and reflects a concrete next step, not a generic aspiration.

- **Credentialed production deployment.** Deploy the platform to the registered `nexis.sh` domain behind a real GitHub App installation and a paid OpenAI account, so that the full

production path (authenticated clone, real pull-request creation, real merge-triggered redeploy) is exercised end-to-end with production credentials.

- **Complete the scenario library.** Wire the remaining twenty-six Live-Demo scenario cards to real fixtures and classification paths, which would extend the existing eleven-scenario fault-injection library across the application, database, deploy, observability, and security categories it already enumerates.
- **Finer-grained Sentinel baselines.** At present, the Sentinel’s statistical spike rule maintains its Exponentially Weighted Moving Average (EWMA) mean and variance baseline per organisation. Extending this to per-(organisation, service) baselines would thus stop a noisy service from masking a quiet one.
- **A fitted causal model.** Once production incident data with genuine interventions has accumulated, a real DoWhy structural causal model [4] should be fitted and evaluated against the current graph-evidence proxy, which was designed from the outset to be replaceable.
- **Execute a fine-tuning run.** When the accumulated RLHF export contains enough approve/reject/modify examples, the next step is to submit an actual fine-tuning job against it and see whether the tuned model reduces rejection and modification rates at the approval gate.
- **Real preview subdomains.** Adding a reverse-proxy layer (for example Traefik) in front of the deploy engine would let preview deployments receive stable per-project subdomains instead of `localhost` ports, a prerequisite for sharing preview URLs beyond the local machine.
- **Formal load benchmarking.** Run a controlled concurrent-load benchmark against the control plane and the recovery pipeline to characterise throughput, queue behaviour under simultaneous incidents, and Temporal worker saturation.
- **Human-subjects usability study.** The planned NASA Task Load Index (NASA-TLX) cognitive-workload study [18] with external participants is yet to be conducted; the survey instrument itself is already integrated into the post-incident flow, so what remains is recruitment and analysis rather than implementation.

| 7.4 Conclusion

The central claim of this report is deliberately narrow and deliberately strong: a software fault can travel from automatic detection, through multi-agent diagnosis, patch generation, and sandboxed validation, past a single human approval decision, and on to a deployed fix, all inside one continuous, durable, observable pipeline. That claim is not asserted; it is demonstrated with reproducible, live evidence. One recorded run shows the loop refusing to proceed when its human gate was not answered in time, while another shows that same loop carrying a real fault to an approved, deployed resolution in under two minutes, with every agent step, token count, and decision recorded in a durable timeline. The preview-deploy engine carries the same principle over to infrastructure, turning its own build failures into self-healing targets. Equally important, though, is what this report declines to claim. In this regard, Section 7.2 sets out that honest accounting in full: local models rather than production credentials, a registered but not yet live domain, a proxy in place of a fitted causal model, and one wired demo scenario out of twenty-seven. None of this is a list of apologies; it is part of the project's rigour, since the boundary of the evidence has been drawn exactly where the evidence stops. Within that boundary, Nexis stands as a complete, working demonstration that closed-loop fault recovery, gated by a single human decision point, is buildable today.

References

1. GitHub Copilot. AI pair-programming assistant for code completion and suggestion. Available at: <https://github.com/features/copilot> (Accessed: 11 August, 2026).
2. Temporal. Durable workflow orchestration engine for fault-tolerant distributed applications. Available at: <https://temporal.io/> (Accessed: 11 August, 2026).
3. Neo4j. Native graph database for storing and querying highly connected data. Available at: <https://neo4j.com/> (Accessed: 11 August, 2026).
4. DoWhy. Causal inference library from Microsoft Research / PyWhy for modelling and estimating causal effects. Available at: <https://github.com/py-why/dowhy> (Accessed: 11 August, 2026).
5. OpenTelemetry. Open-source observability framework for traces, metrics, and logs. Available at: <https://opentelemetry.io/> (Accessed: 11 August, 2026).
6. Argo CD. Declarative GitOps continuous delivery tool for Kubernetes. Available at: <https://argo-cd.readthedocs.io/> (Accessed: 11 August, 2026).
7. PostgreSQL Row Security Policies. Official documentation on row-level security for multi-tenant data isolation. Available at: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html> (Accessed: 11 August, 2026).
8. OpenLineage. Open standard for data lineage collection and analysis. Available at: <https://openlineage.io/> (Accessed: 11 August, 2026).
9. Hypothesis. Property-based testing library for Python. Available at: <https://hypothesis.readthedocs.io/> (Accessed: 11 August, 2026).
10. Docker. Container platform for building, shipping, and running applications. Available at: <https://www.docker.com/> (Accessed: 11 August, 2026).
11. Podman. Daemonless, Docker-API-compatible container engine. Available at: <https://podman.io/> (Accessed: 11 August, 2026).

12. WorkOS. Enterprise-ready authentication and single sign-on platform. Available at: <https://workos.com/> (Accessed: 11 August, 2026).
13. Stripe. Online payment processing and billing infrastructure. Available at: <https://stripe.com/> (Accessed: 11 August, 2026).
14. Sentry. Application error tracking and performance monitoring platform. Available at: <https://sentry.io/> (Accessed: 11 August, 2026).
15. PagerDuty. Incident response and on-call management platform. Available at: <https://www.pagerduty.com/> (Accessed: 11 August, 2026).
16. Datadog. Cloud-scale monitoring and observability service. Available at: <https://www.datadoghq.com/> (Accessed: 11 August, 2026).
17. Slack API. Platform APIs for building Slack integrations and interactive messages. Available at: <https://api.slack.com/> (Accessed: 11 August, 2026).
18. NASA Task Load Index (NASA-TLX). Subjective workload assessment instrument developed by NASA Ames Research Center. Available at: <https://humansystems.arc.nasa.gov/groups/tlx/> (Accessed: 11 August, 2026).
19. pgvector. Open-source vector similarity search extension for PostgreSQL. Available at: <https://github.com/pgvector/pgvector> (Accessed: 11 August, 2026).
20. Ollama. Tool for running large language models locally. Available at: <https://ollama.com/> (Accessed: 11 August, 2026).